第2章: TypeScript の基礎

この章では、TypeScript（タイプスクリプト：JavaScript に「型」の仕組みを追加したプログラミング言語。読みは「タイプスクリプト」）の基本を学びます。TypeScript は JavaScript（ジャバスクリプト：ブラウザで動く代表的なプログラミング言語）に「型（Type：データの種類のこと。例えば「文字」「数値」「真偽値」など）」という仕組みを追加した言語です。「型って何？」という方も安心してください。身近な例えを交えながら、一つずつ丁寧に解説します。

この章で学ぶこと

- **型（Type：タイプ）とは何か** — データの種類を指定する仕組み。「この箱には数字だけ入れていいよ」というラベルのようなもの
- **基本的な型** — `string`（ストリング：文字列を表す型）、`number`（ナンバー：数値を表す型）、`boolean`（ブーリアン：true/false の真偽値を表す型）など
- **インターフェース / 型エイリアス** — 自分だけのオリジナルの型を定義する方法。書籍管理アプリでは「Book型」を作ります
- **ジェネリクス**（Generics：ジェネリクス。「総称型」と訳される。型をパラメータ（後で決める変数のようなもの）として受け取る仕組み） — 型をパラメータ（引数）として受け取る柔軟な仕組み
- **よくあるエラーと対処法** — 初心者がつまづきやすいTypeScriptのエラーメッセージと、その読み方

なぜTypeScriptを学ぶの？ JavaScriptだけでもアプリは作れますが、TypeScriptを使うと「コードを書いている途中で間違いに気づける」「エディタ（コードを書くソフト）が賢く補完してくれる」というメリットがあります。最初は少し面倒に感じるかもしれませんが、慣れると「TypeScriptなしでは開発できない！」と思うようになります。

大事なポイント（先に伝えておくと安心）：

- **型は実行時には消える** — TypeScript の型情報は「人間と TypeScript コンパイラだけが見えるメモ」。プログラムが実際に動くときには影響しません（これを「型消去」と呼びます）。
- **`const` をデフォルトにする** — 値を入れ替える必要がないなら必ず `const`（再代入禁止）。「ここは入れ替わらない」とコードを読む人に伝えられて、バグも減ります。
- **`null` と `undefined`** — どちらも「値がない」を表しますが、`undefined`（アンディファインド：未定義）は「まだ何も入れていない」、`null`（ヌル：明示的な空）は「意図的に空にしている」を表します。
- **セミコロン `;` は省略可能** — 文末の `;` は本当は無くても動きますが、本書では分かりやすさのため付けます。

この章のゴールと地図

この章を読み終えたとき、次の5つができるようになっていれば合格です。

1. `const price: number = 2980;` のような**型注釈**を読んで、「この変数には数値しか入らない」と説明できる
2. VS Code の**赤い波線**やターミナルの**エラーメッセージ**を読んで、「どこが・なぜ」間違っているかを見つけられる
3. `interface` / `type` を使って、書籍管理アプリの `Book` 型のような**自分だけの型**を作れる
4. `string | null` のような**ユニオン型**を、`if` で**絞り込んで**から安全に使える
5. `<T>`（ジェネリクス）や `Promise` / `async` / `await` が出てきても、怖がらずに読める

章全体は、次の図のように「土台 → 型を組み立てる → 実戦で使う」の3段階で進みます。いま自分がどこを読んでいるのか迷ったら、この地図に戻ってきてください。

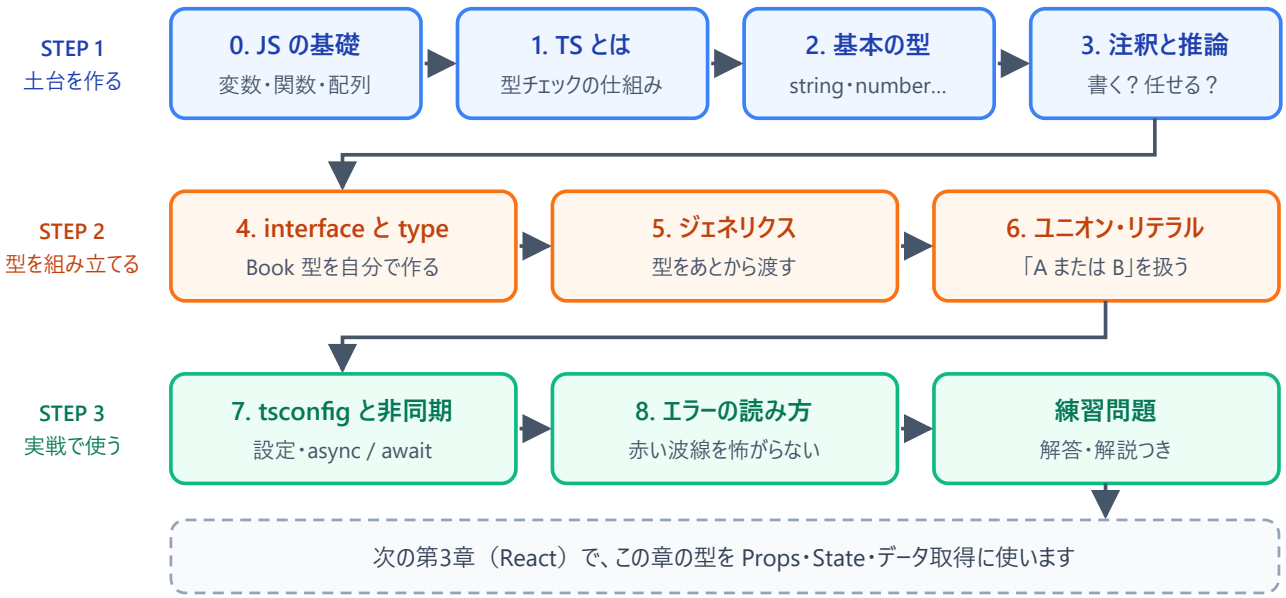


図2-1: 第2章の地図。上から順に「土台 → 型を組み立てる → 実戦で使う」と進む

> **進め方のコツ:** この章は「図 → 小さなコード → 実行結果 → 1行ずつの解説」の順に並んでいます。コードはできるだけ**自分の手で打ち込み**、慣れてきたら**わざと間違えて**エラーを出してみてください。エラーを出して直した回数だけ、TypeScript と仲良くなれます。

この章のサンプルを動かす準備

この章のサンプルの多くは、そのままファイルに保存して**実行**したり、**型チェック**したりできます。最初に一度だけ、練習用のフォルダを用意しましょう（VS Code のターミナルや PowerShell で入力します）。

```
# 練習用のフォルダを作って、その中へ移動する
mkdir ts-practice
cd ts-practice
# package.json（このフォルダの設定ファイル）を作る
npm init -y
# 型チェックの道具 tsc（TypeScript 本体）を、このフォルダに入れる
npm install -D typescript
```

準備ができたら、サンプルごとに `check.ts` のような名前でファイルを作り、次の2つのコマンドを使い分けます。

やりたいこと	コマンド	何が起きるか
実行する	`node check.ts`	型の部分を消して、JavaScript として動かす（型の間違いは調べない）
型をチェックする	`npx tsc --noEmit --strict --target esnext check.ts`	型の間違いだけを調べて報告する（プログラムは動かさない）

`tsc` のオプションの意味は次のとおりです。今は「おまじない」として毎回この形で打てば大丈夫です。

- `--noEmit`（ノーエミット）… 変換後の JavaScript ファイルを**作らない**（チェックだけしたいので）
- `--strict`（ストリクト）… **厳しいチェック**をすべて有効にする（Next.js のプロジェクトと同じ厳しさ）
- `--target esnext`（ターゲット）… 最新の JavaScript の機能（`Promise` など）がある前提でチェックする

> **注意:** `node ファイル名.ts` で TypeScript のファイルを直接実行できるのは、**Node.js 22.18 以降**（23.6 以降、24、25 など）です。`node --version` で確かめ、古い場合は第1章の手順で LTS 版に更新してください。バージョンによっては `ExperimentalWarning: Type Stripping is an experimental feature...`（型を消す機能はまだ実験中です）という警告が1行出ますが、動作には問題ありません。

> **注意:** `npx tsc` は「このフォルダに入れた TypeScript の `tsc` を動かす」という意味です。`npm install -D typescript` をする**前に** `npx tsc` を打つと、`tsc` という名前の**無関係なパッケージ**をインストールしようとしてしまいます。必ず先にインストールしてください。

> **注意:** 練習用フォルダには `tsconfig.json`（7節で学ぶ設定ファイル）を置かないでください。置いたまま `npx tsc ファイル名.ts` のようにファイル名を指定すると、新しい TypeScript では `error TS5112: tsconfig.json is present but will not be loaded if files are specified on commandline.` というエラーになります。

> **この章の実行結果について:** 実行結果とエラーメッセージは、Node.js 25.9 と TypeScript 7.0（`npm install -D typescript` で入る版）で実際に動かして貼り付けたものです。`C:\ts-practice` の部分は、あなたのフォルダの場所に置き換えて読んでください。Next.js のプロジェクトでは TypeScript 5 系が入ることがあり、エラーの「補足」部分の並び方が少し違いますが、読み方は同じです。

目次

0. 前提知識: JavaScriptの基礎の基礎
1. TypeScript とは

2. 基本的な型
3. 型注釈と型推論
4. インターフェースと型エイリアス
5. ジェネリクス
6. ユニオン型とリテラル型
7. TypeScript の設定（tsconfig.json）
8. よくあるエラーと対処法
9. 発展: アプリでは使っていない重要なTypeScript機能
10. 練習問題（解答・解説つき）

---

0. 前提知識: JavaScriptの基礎の基礎

TypeScript は JavaScript に「型」を追加した言語なので、JavaScript の文法をひと通り知っておくと話が早く進みます。ここでは TypeScript を読むうえで**最低限必要な JavaScript の文法**だけ、ぎゅっと圧縮して解説します。「もう知ってる！」という方は読み飛ばして 1 章へ。

> **動かしながら読みたい方へ:** 以下の例はどれも、ターミナルで `node` と入力して Enter を押すと開く対話モード（REPL）に貼り付けて実行できます。終了は `Ctrl+C` を2回押します。または、ブラウザの**開発者ツール → Console**（F12 で開ける）にそのまま貼り付けてもOKです。

0.1 コメント

コメントは「コンピュータには無視されるが、人間が読むためのメモ」です。

```
// からその行の終わりまでが「単一行コメント」。コンピュータは無視するので、人間用のメモとして自由に書ける。
// 行頭の
// この行はコメント。実行されない。
// 行末にも書ける
// let = 後から値を変えられる変数の宣言。x = 1 で変数 x に 1 を代入。文末の ; はセミコロン（文の終わりを示す）。// 以降はコメント。
let x = 1;
// 空行も自由に入れられる（コードの読みやすさのため）。

// /* で始めると複数行コメントの開始。行頭の * は飾り（書式の慣習）で意味はない。
// 文章中の /* や */ も「ただの文字」として扱われる。*/ で複数行コメントの終了。
/*
 * 複数行コメント。
 * /* と */ で囲む
 */
```

0.2 変数の宣言: `const` / `let`

値に名前を付けて保存する仕組みです。本書では原則 `const`（コンスト：constant の略。再代入できない変数）を使い、必要なときだけ `let`（レット：再代入できる変数）を使います。古い書き方の `var`（バー：variable の略。古い変数宣言）はバグの元なので使いません。

> **なぜ `const` をデフォルトにするのか？** 変数を見ただけで「これは中身が変わらない」と分かると、コードを読む人の負担が減ります。「もしかして他で書き換えてる？」と疑う必要がないからです。React/Next.js のコードではほぼ全ての変数を `const` で宣言します。

> **▼ このコードがやること（先に日本語で）**：値に名前を付けて保存する「変数」を、`const`（あとで変えられない）と `let`（あとで変えられる）の2通りで作ってみます。一番大事なのは「**変えない値は `const` を使う**」という習慣——うっかり書き換えてしまうバグを未然に防げます。詳しい1行ごとの意味はコード内のコメントを見てください。

```
// const = 一度入れたら変えられない
// const は「定数」宣言。name という名前の箱を作り、"太郎" を入れる。="代入演算子"。"..." で囲うと文字列リテラル。
const name = "太郎";

// 次の行はコメントアウト中。コメント記号(//)を外して再代入しようとすると
// 「Assignment to constant variable.（定数 name には再代入できない）」というエラーになる。
// name = "次郎";

// let = 後で書き換え可能
// let は「後で値を変えられる変数」の宣言。count という変数に初期値 0 を入れる。
let count = 0;
// ✅ OK
// count に「現在の count + 1」を代入。count は 0 → 1 になる。
count = count + 1;
// console.log は「コンソール（ターミナルやブラウザの出力欄）に値を表示する」関数。count の中身(=1) を表示する。
console.log(count);

// ▼ 実行結果
// 1
```

> **`console.log()` とは？**：ターミナル（Node.js の場合）またはブラウザのコンソール（ブラウザの場合）に値を表示する関数です。プログラムの動作確認に多用します。

0.3 基本のデータ型

データ型	例	説明
文字列（string）	`"hello"` `'太郎'` `` `テンプレ ${x}` ``	クォートで囲んだ文字
数値（number）	`42` `3.14` `-0.5`	整数も小数も同じ型
真偽値（boolean）	`true` `false`	ハイorロー
`null` / `undefined`	`null` `undefined`	値が「ない」ことを表す（null=明示的、undefined=未定義）
配列（array）	`[1, 2, 3]`	値の並び
オブジェクト（object）	`{ name: "太郎", age: 20 }`	キー：値の組

▼ このコードがやること（先に日本語で）：プログラムでよく使う基本的な「データの種類（型）」——文字列・数値・真偽値・配列・オブジェクト——を1つずつ変数に入れて、`console.log` でまとめて表示します。今は「こういう種類のデータがあるんだな」と眺めるだけでOKです。それぞれの値の書き方はコメントで説明しています。

```
// 文字列リテラル "hello"（"..." または '...' で囲ったもの）を const 変数 text に入れる。
const text = "hello";
// 数値リテラル 42（クォートで囲わない数字）を変数 num に入れる。整数も小数も同じ number 型。
const num = 42;
// 真偽値 true（true か false の2つしかない）を変数 isOk に入れる。命名で is... を付けると「真偽値」と分かりやすい慣習。
const isOk = true;
// 配列リテラル。[ ] の中に値を , で並べる。[0]=1, [1]=2, [2]=3 の順に並ぶ（添え字は 0 始まり）。
const list = [1, 2, 3];
// オブジェクトリテラル。{ キー: 値 } の組を , で並べる。.name や .age で値を取り出せる。
const user = { name: "太郎", age: 20 };

// console.log は引数を ", " 区切りでまとめて表示する。複数値の確認に便利。
console.log(text, num, isOk, list, user);

// ▼ 実行結果
// hello 42 true [ 1, 2, 3 ] { name: '太郎', age: 20 }
```

0.4 演算子（足し算・比較）

```
// 3（足し算）
// + は加算演算子。数値同士なら算術加算、文字列同士なら連結。
console.log(1 + 2);
// 7
// - は減算演算子。10 から 3 を引く。
console.log(10 - 3);
// 20
// * は乗算演算子（× の代わり）。
console.log(4 * 5);
// 3.3333333333333335
// / は除算演算子（÷ の代わり）。JavaScript の数値は浮動小数点なので小数になる。
console.log(10 / 3);
// 1（余り）
// % は剰余演算子。10 ÷ 3 = 3 余り 1 の「余り」を返す。
console.log(10 % 3);
// "abcd"（文字列の連結）
// + の左右が文字列だと、足し算ではなく連結になる。
console.log("ab" + "cd");

// true（等しい。型もチェック）
// === は「型も値も完全に同じか」を判定。同じなら true、違うなら false。
console.log(1 === 1);
// false（型が違うと false）
// 数値の 1 と文字列の "1" は型が違うので false。
console.log(1 === "1");
// true（異なる）
// !== は「型または値が異なるか」を判定。違うなら true。
console.log(1 !== 2);
// true
// > は「左が右より大きいか」。等号付きなら >=（以上）。
console.log(3 > 2);
// false（両方trueならtrue = AND）
// && は AND 演算子。両辺が true のときだけ true。
console.log(true && false);
// true  （片方trueならtrue = OR）
// || は OR 演算子。どちらか片方が true なら true。
console.log(true || false);
// false（反転）
// ! は否定演算子。true → false、false → true に反転する。
console.log(!true);
```

> **`==` ではなく `===` を使う:** 1個少ない `==` は型変換しながら比較するため、`1 == "1"` が `true` になります。バグの温床なので、本書では一貫して `===` を使います。

0.5 文字列とテンプレートリテラル

> ▼ **このコードがやること（先に日本語で）**：文字列に変数の値を埋め込む2つの方法——古い「`+` でつなげる」やり方と、新しい「**テンプレートリテラル**（バッククォート `` ` `` で囲み `${変数}` を埋め込む）」やり方——を見比べます。覚えてほしいのは後者の `` `〜${name}〜` `` の書き方で、こちらの方が読みやすくミスも減ります。

```
// 文字列リテラルを変数 name に代入。
const name = "太郎";

// 連結（古い書き方）
// + 演算子で「文字列＋変数＋文字列」をつなげる。冗長になりがち。
const msg1 = "こんにちは、" + name + "さん！";

// テンプレートリテラル（新しい書き方。バッククォート ` で囲む）
// `...` で囲んだ中に ${変数} を書くと、その場所に値が埋め込まれる。読みやすい。
const msg2 = `こんにちは、${name}さん！`;

// 1行目の文字列を表示。
console.log(msg1);
// 2行目の文字列を表示。
console.log(msg2);

// ▼ 実行結果
// こんにちは、太郎さん！
// こんにちは、太郎さん！
```

`${ ... }` の中には変数や式（しき：値を計算する記述。`1 + 2` や `name.length` など、評価すると1つの値になるもの）を入れられます。テンプレートリテラル（template literal：バッククォートで囲んだ文字列）は改行も含められて便利です。

0.6 if文（条件分岐）

> ▼ **このコードがやること（先に日本語で）**：点数に応じて「優・可・不可」を出し分ける**条件分岐**を `if / else if / else` で書きます。「もし〜なら、こうする。そうでなければ…」という考え方で、上から順に条件をチェックし、最初に当てはまったブロックだけが実行されます。

```
// 変数 score に数値 75 を入れる。
const score = 75;

// if は「もし～なら」の意味。( ) の中の条件式が true ならブロックを実行する。>= は「以上」。
if (score >= 80) {
  // score が 80 以上のとき表示。
  console.log("優");
// else if = 「上の条件に当てはまらず、こちらの条件に当てはまるなら」。score は 75 なので 80 未満かつ 60 以上 → ここが実行される。
} else if (score >= 60) {
  // 60 以上 80 未満なら表示。
  console.log("可");
// else = 「どの条件にも当てはまらないなら」。
} else {
  // 60 未満なら表示。
  console.log("不可");
// if 文の終わり。
}

// ▼ 実行結果
// 可
```

`{ ... }` で囲んだ範囲を**ブロック**（block：複数の文をひとまとめにした区切り。中括弧で囲む）と呼びます。条件に当てはまったブロックだけ実行されます。

> 「**文（statement）**」と「**式（expression）**」**の違い**: `if (...) { ... }` のように「処理を1つ実行する命令」が**文**。`1 + 2` や `score >= 80` のように「評価すると1つの値になる」のが**式**。`const x = 1 + 2;` は「`x` に `1+2`（式）を代入する文」のように、文の中に式が入ります。

0.7 for ループ・配列のループ

> **▼ このコードがやること（先に日本語で）**：同じ処理を**繰り返す**2つの書き方を学びます。①回数を数える `for` ループ、②配列の中身を1つずつ取り出す `for...of` ループです。配列を扱うときは ②の `for...of` が読みやすくおすすめです。

```
// 0, 1, 2 と3回繰り返す
// for ループ。( 初期化; 継続条件; 各回の最後の処理 ) の3つを ; で区切って書く。let i = 0 で開始、i < 3 が true の間繰り返し、毎回終わりに i++（i = i+1）。
for (let i = 0; i < 3; i++) {
  // ループの中身。テンプレートリテラルで現在の i を表示。
  console.log(`i = ${i}`);
// for ブロックの終わり。
}

// ▼ 実行結果
// i = 0
// i = 1
// i = 2

// 配列を1つずつ取り出す（モダンな書き方）
// 文字列の配列を作る。
const fruits = ["apple", "banana", "cherry"];
// for...of ループ。fruits の中身を 1つずつ fruit に取り出して繰り返す。const なので毎回新しい束縛。
for (const fruit of fruits) {
  // 取り出した値を表示。
  console.log(fruit);
// for ブロックの終わり。
}

// ▼ 実行結果
// apple
// banana
// cherry
```

0.8 関数の書き方

関数は「処理に名前を付けて何度でも呼び出せるようにしたもの」です。3 通りの書き方があります。本書では主に**3番のアロー関数**を使います。

> **▼ このコードがやること（先に日本語で）**：「2つの数を足す」という同じ処理を、関数の**4通りの書き方**で作って結果を見比べます。注目してほしいのは `(a, b) => a + b` という**アロー関数**——React/Next.js でほぼ毎回登場する書き方なので、ここで形に慣れておきましょう。

```
// 1. function 宣言
// function キーワードで関数を定義。add1 が関数名、(a, b) が「仮引数（受け取る値）」。
function add1(a, b) {
  // return = 計算結果を呼び出し元に返す。これがないと undefined を返す。
  return a + b;
// 関数本体の終わり。
}

// 2. function 式
// 「無名関数」を変数 add2 に代入する書き方。function の後に関数名がない。
const add2 = function (a, b) {
  // 中身は宣言版と同じ。
  return a + b;
// 式の終わりなので最後に ; が付く（宣言版は不要）。
};

// 3. アロー関数（=> を使う、Reactでよく使う）
// (引数) => { 処理 } の形がアロー関数。function キーワードが不要で短い。
const add3 = (a, b) => {
  // 本体は通常の関数と同じ。
  return a + b;
// アロー関数も「式」なので末尾に ;。
};

// 3'. アロー関数の省略形（return 1行のとき）
// 本体が return 1行だけのときは { } と return を省略可能。「a + b の値が自動で返る」と読む。
const add4 = (a, b) => a + b;

// 3
// 関数の呼び出し。add1 に 1 と 2 を渡し、返り値 3 を表示。
console.log(add1(1, 2));
// 3
// どの書き方でも結果は同じ。
console.log(add2(1, 2));
// 3
console.log(add3(1, 2));
// 3
console.log(add4(1, 2));
```

> **アロー関数の魅力:** `function` の文字を書かなくて済むので短く、コールバック関数（あとで呼んでもらう関数）として渡すときに見やすくなります。React/Next.jsのコードはほぼアロー関数で書かれています。

0.9 配列の便利メソッド: map / filter / forEach

これは超重要です。後の章で頻繁に登場します。

▼ **このコードがやること（先に日本語で）**：配列を扱う3つの超重要メソッド―― `forEach`（1つずつ処理）、`map`（1つずつ加工して**新しい配列**を作る）、`filter`（条件に合う要素だけ残す）――を使い分けます。鉄則は「`map` と `filter` は元の配列を変えず、**新しい配列を返す**」こと。React で一覧表示を作るとき毎回使うので、ここでしっかり押さえましょう。

```
// 数値の配列を用意。
const numbers = [1, 2, 3, 4, 5];

// forEach: 1つずつ処理する
// forEach は「配列の各要素に対して関数を実行する」メソッド。引数のアロー関数を1要素ずつ呼んでくれる。
numbers.forEach((n) => {
  // n には 1 → 2 → 3 → 4 → 5 と順に入る。
  console.log(n);
// forEach の終わり。返り値は無い（undefined）。
});

// ▼ 実行結果
// 1
// 2
// 3
// 4
// 5

// map: 1つずつ加工して新しい配列を作る
// map は「各要素を加工した新しい配列を返す」メソッド。元の配列は変わらない。(n) => n * 2 は省略形のアロー関数。
const doubled = numbers.map((n) => n * 2);
// 加工後の配列を表示。
console.log(doubled);

// ▼ 実行結果
// [ 2, 4, 6, 8, 10 ]

// filter: 条件を満たす要素だけ残した新しい配列を作る
// filter は「条件式が true になる要素だけ残した配列を返す」メソッド。n % 2 === 0 は「2 で割った余りが 0 ＝偶数」の意。
const evens = numbers.filter((n) => n % 2 === 0);
// 偶数だけ残った配列を表示。
console.log(evens);

// ▼ 実行結果
// [ 2, 4 ]
```

0.10 オブジェクトの読み書き

▼ **このコードがやること（先に日本語で）**：「名前・年齢…」のような複数の情報を**まとめて持つオブジェクト**の、値の取り出し・書き換え・追加、そして最後に「**分割代入**（`const { name, age } = user;` で一気に取り出す）」を学びます。特に分割代入は React のコードで頻出するので、形を覚えておくと後がラクです。

```
// オブジェクトを作る。name と age の2つのプロパティ（プロパティ＝オブジェクトのキーと値の組）を持つ。
const user = { name: "太郎", age: 20 };

// 値の取り出し（2つの書き方）
// "太郎"（ドット記法）
// .（ドット）でプロパティ名を直接指定する書き方。普通はこちらを使う。
console.log(user.name);
// [ ] にキー名を文字列で渡す書き方。変数でキーを指定したいとき便利（user[キー名変数]）。
// "太郎"（ブラケット記法）
console.log(user["name"]);

// 値の書き換え
// const で宣言されていても、オブジェクトの「中のプロパティ」は書き換え可能（const が禁止するのは user 自体の差し替えのみ）。
user.age = 21;
// 21
// age が 21 に変わった。
console.log(user.age);

// プロパティの追加
// 存在しないキーに代入すると、新しいプロパティが追加される。
user.email = "taro@example.com";
// user 全体を表示すると、email が追加されているのが分かる。
console.log(user);
// ▼ 実行結果
// { name: '太郎', age: 21, email: 'taro@example.com' }

// 分割代入（オブジェクトから一気に取り出す）
// { } の中に取り出したいキー名を書くと、同じ名前の変数を一気に作れる。name = user.name, age = user.age と同じ意味。
const { name, age } = user;
// "太郎" 21
console.log(name, age);
```

分割代入のコツ: React のコードでよく `const { title, author } = book;` のような書き方を見ます。これは「`book` オブジェクトから `title` と `author` を取り出して同名の変数を作る」省略記法です。慣れると非常に短く書けます。

0.10.1 オブジェクトは「住所」で受け渡される（参照とコピー）

上の例で「`const` で宣言したのに `user.age = 21` と書き換えられた」のを不思議に思った人もいるでしょう。その理由は、**オブジェクトや配列がメモリ（パソコンの記憶場所）の中でどう置かれているか**を知ると、すっきり分かります。

> **たとえ話:** 数値や文字列は、名札の付いたメモ用紙に**値そのもの**を書いておくイメージです。一方オブジェクトや配列は「大きな荷物」なので、別の倉庫に置いておき、変数のメモ用紙には**倉庫の住所**だけを書いておきます。`const` が禁止しているのは「メモ用紙を書き換えること（再代入）」だけ。住所の先にある荷物の中身は、`const` でも変えられるのです。

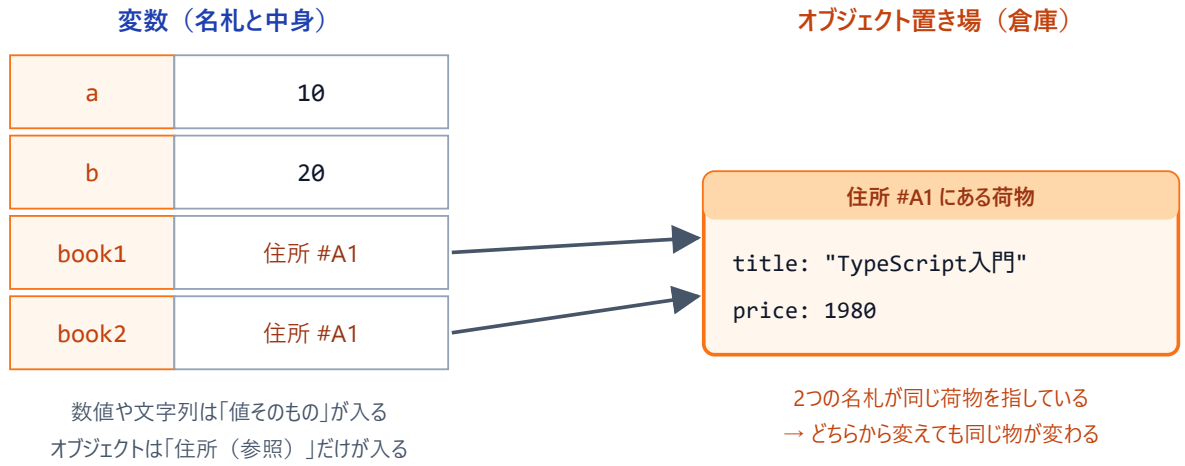


図2-2: メモリの中のイメージ。オブジェクトの変数には「荷物の住所」だけが入っている

参照とコピーの違いを確かめる

> **▼ このコードがやること（先に日本語で）:** 数値を別の変数に入れた場合と、オブジェクトを別の変数に入れた場合で、「片方を書き換えたらもう片方はどうなるか」を見比べます。最後に、スプレッド構文 `{ ...元 }` を使って**中身ごとコピー**する方法も試します。ファイル名は `copy.ts` です（中身は型注釈のない JavaScript そのままですが、TypeScript としてもそのまま動きます）。

```ts
// ① 数値（プリミティブ）を別の変数に入れる
let a = 10;
let b = a;
// b だけを 20 に変える
b = 20;
console.log("a =", a, "/ b =", b);

// ② オブジェクトを別の変数に入れる
const book1 = { title: "TypeScript入門", price: 2980 };
const book2 = book1;
// book2 の price だけを変えたつもり
book2.price = 1980;
console.log("book1:", book1);
console.log("book2:", book2);
console.log("book1 と book2 は同じもの？", book1 === book2);

// ③ スプレッド構文 { ...元 } で「中身」をコピーする
const book3 = { ...book1 };
book3.price = 500;
console.log("book1:", book1);
console.log("book3:", book3);
console.log("book1 と book3 は同じもの？", book1 === book3);
```

▼ 実行結果（`node copy.ts`）

```
a = 10 / b = 20
book1: { title: 'TypeScript入門', price: 1980 }
book2: { title: 'TypeScript入門', price: 1980 }
book1 と book2 は同じもの？ true
book1: { title: 'TypeScript入門', price: 1980 }
book3: { title: 'TypeScript入門', price: 500 }
book1 と book3 は同じもの？ false
```

▼ コードを1つずつ分解して解説

解説1: 数値は「値そのもの」がコピーされる

```ts
let a = 10;
let b = a;
b = 20;
```

- `let b = a;` の時点で、`b` のメモ用紙には `a` の**値（10）そのもの**が書き写されます。
- そのあと `b = 20` と書き換えても、`a` のメモ用紙は別の紙なので `10` のままです。実行結果の `a = 10 / b = 20` がその証拠です。

- 数値・文字列・真偽値・`null`・`undefined` のような「小さな値」は、いつもこの動き方をします。

> **用語: プリミティブ（primitive：基本の値）** … 数値・文字列・真偽値・`null`・`undefined` など、それ以上分解できない基本的な値のこと。変数には値そのものが入ります。

解説2: オブジェクトは「住所」がコピーされる

```
const book1 = { title: "TypeScript入門", price: 2980 };
const book2 = book1;
book2.price = 1980;
```

- `const book2 = book1;` で書き写されるのは、荷物（オブジェクト）そのものではなく**荷物の住所**です（図2-2）。
- そのため `book2.price = 1980` は「住所 #A1 にある荷物の価格を書き換える」という意味になり、同じ荷物を指している `book1` から見ても価格が `1980` に変わっています。
- `book1 === book2` が `true` になるのは、`===` がオブジェクトの場合「**同じ住所か**」を比べるからです。

> **用語: 参照（さんしょう：reference）** … 「その荷物がどこにあるか」を指し示す情報のこと。オブジェクトや配列の変数には、中身ではなく参照が入っています。

解説3: スプレッド構文 `{ ...元 }` で中身をコピーする

```
const book3 = { ...book1 };
book3.price = 500;
```

- `{ ...book1 }` は「`book1` の中身を全部取り出して、**新しい荷物**に詰め直す」という意味です。倉庫に新しい荷物（別の住所）ができます。
- だから `book3.price = 500` と書き換えても、`book1` の価格（1980）はそのままです。`book1 === book3` は「別の住所」なので `false` になります。
- 第3章の React では「state（画面の状態）は直接書き換えず、コピーを作って置き換える」というルールが出てきます。そのとき使うのがこの書き方です。

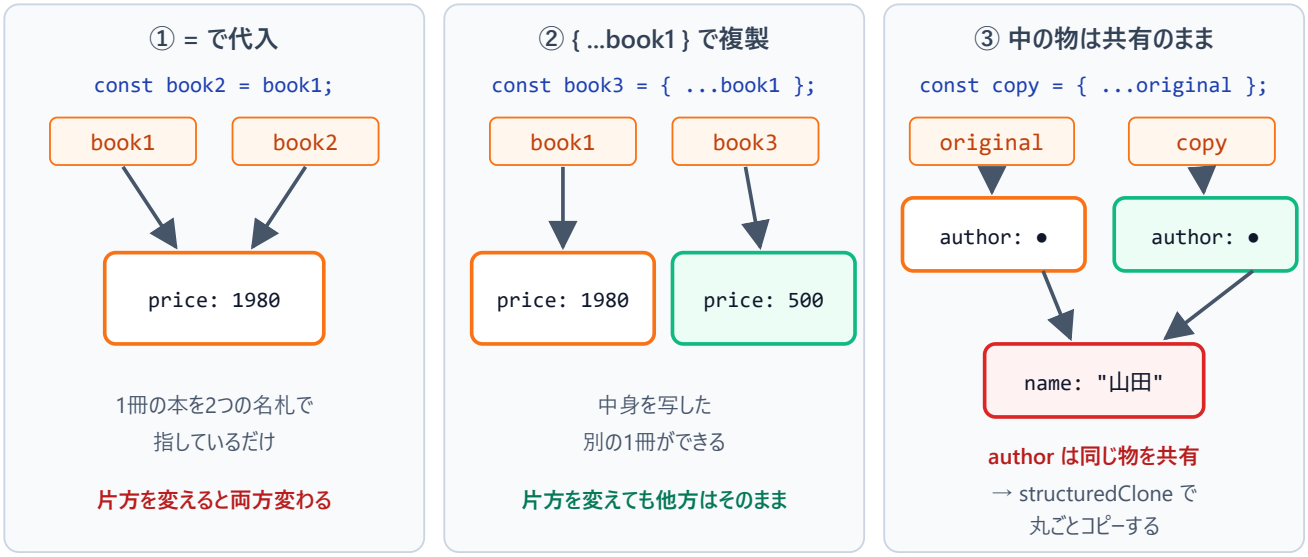


図2-3:「= で代入」「スプレッドで複製」「中のオブジェクトは共有のまま」の3パターン

> **つまずきポイント: スプレッド `{ ...元 }` は「1段目だけ」のコピー** — スプレッド構文がコピーするのは、いちばん外側の荷物だけです。荷物の中に**さらにオブジェクトが入っている**場合（図2-3の③）、その中身は住所が書き写されるだけなので、コピーと元で**共有**されたままになります。これを「浅いコピー（シャローコピー）」と呼びます。

NG 例（`shallow-ng.ts`）：コピーの著者名を変えたら、元の本の著者名まで変わってしまった

```
const original = {
  title: "TypeScript入門",
  author: { name: "山田太郎" },
};

// スプレッドでコピーしたつもり
const copy = { ...original };
// コピーの著者名だけを変えたつもり
copy.author.name = "佐藤花子";

console.log(original.author.name);
```

▼ 実行結果

```
佐藤花子
```

OK 例（`shallow-ok.ts`）：中のオブジェクトもコピーし直すか、`structuredClone` で丸ごとコピーする

```ts
const original = {
  title: "TypeScript入門",
  author: { name: "山田太郎" },
};

// 方法1: 中のオブジェクトもスプレッドでコピーし直す
const copy1 = { ...original, author: { ...original.author } };
copy1.author.name = "佐藤花子";

// 方法2: structuredClone で丸ごと（深く）コピーする
const copy2 = structuredClone(original);
copy2.author.name = "鈴木一郎";

console.log(original.author.name);
console.log(copy1.author.name);
console.log(copy2.author.name);
```

▼ 実行結果

```
山田太郎
佐藤花子
鈴木一郎
```

- 方法1の `{ ...original, author: { ...original.author } }` は、「外側をコピーしたうえで、`author` だけは**中身もコピーした新しい荷物**に差し替える」という意味です。
- 方法2の `structuredClone(値)`（ストラクチャード・クローン）は、ブラウザと Node.js に最初から入っている関数で、**何段入れ子になっていても丸ごとコピー**してくれます（深いコピー＝ディープコピー）。
- 元の `original` の著者名は `山田太郎` のまま守られています。

0.11 セミコロン `;` と改行

JavaScript/TypeScript では文末にセミコロン `;` を付ける習慣があります。実は省略しても大体動きますが（自動セミコロン挿入＝Automatic Semicolon Insertion, ASI という仕組みがあるため）、本書では**付ける派**です（VS Code が自動で付けてくれる場合も多い）。

> **省略すると稀にバグる例:** `return` のすぐ後で改行すると、自動で `;` が入って `return undefined;` と解釈されることがあります。曖昧さを避けるため、本書では一貫して `;` を付けます。

```
// const で a に 1 を入れる。文末の ; は「ここで1つの文が終わる」を示す。
const a = 1;
// 同様に b に 2 を入れる。
const b = 2;
// a + b は 3 になる。それを表示。
console.log(a + b);
```

これで JavaScript の超基礎は OK。次の節からいよいよ TypeScript 本編に入ります。

---

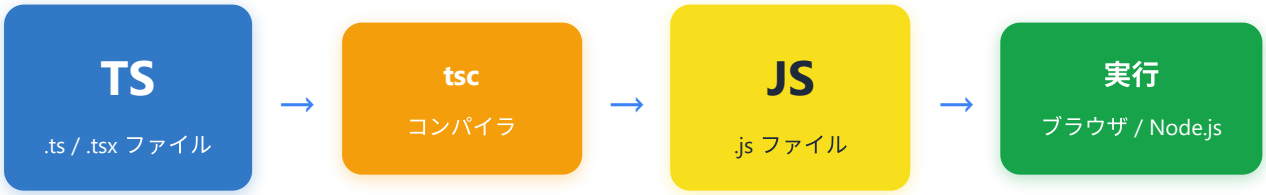
1. TypeScript とは

1.1 JavaScript との関係

TypeScript は、Microsoft（マイクロソフト：Windows や VS Code を作っている会社）が開発した **JavaScript のスーパーセット（superset：上位互換。JavaScriptの機能を全て含み、さらに追加機能がある言語）** です。すべての JavaScript コードは有効な TypeScript コードですが、TypeScript には「**型システム**」（Type System：タイプシステム。変数や関数に入るデータの種類を事前に定義し、チェックする仕組み）という強力な機能が追加されています。

TypeScript のコードは直接ブラウザや Node.js（ノードジェイエス：パソコン上で JavaScript を動かす実行環境）で実行できません。必ず **コンパイル**（Compile：コンパイル。人間が書いたプログラムを、コンピュータが実行できる形に変換すること。TypeScriptの場合はJavaScriptに変換する処理を指す。「トランスパイル（transpile）」とも呼ばれる）という変換処理を経て、JavaScript に変換されてから実行されます。

> **身近な例え：** TypeScriptとJavaScriptの関係は、「下書き用紙」と「清書用紙」に似ています。TypeScript（下書き）には赤ペンで注意書き（型情報）が書いてあり、間違いがあればその場で気づけます。最終的にコンパイルすると、注意書きを消した綺麗なJavaScript（清書）ができあがります。



この流れをもう少し詳しく見てみましょう。



ポイント（とても重要）：TypeScript の型情報は、コンパイル後の JavaScript には一切残りません。型はあくまで「開発時の安全ネット」です。これを **型消去（Type Erasure：タイプイレイジャー。型情報がコンパイルで削除される性質）** と呼びます。

例えば `const age: number = 25;` は、コンパイル後には `const age = 25;` になります。型注釈の `: number` の部分が消えるイメージです。つまり「実行時に型でエラーを止める」ことはできません。型チェックは**コーディング中の VS Code と、コンパイル時の `tsc` だけ**が担当します。

補足（最近の Node.js）：Node.js 22.18 以降は、`node ファイル名.ts` で TypeScript のファイルを**直接**実行できます。ただしこれは、Node.js が実行の直前に「型の部分を消して JavaScript にしている」だけです。つまり上の図の「型チェック」の段階は**飛ばされます**。次の「試してみよう」で、本当にそうなるかを確かめます。

試してみよう: node は型を見ない、tsc は型だけを見る

▼ このコードがやること（先に日本語で）：本の価格に送料を足すだけの小さな関数を作り、わざと「数値の代わりに文字列」を渡します。同じファイルを ① `node` で**実行**したときと、② `tsc` で**型チェック**したときで、結果がどう違うかを見比べます（準備は冒頭の「この章のサンプルを動かす準備」を参照）。ファイル名は `check.ts` です。

```ts
// 本の価格に送料 500 円を足して返す関数
// 引数 price は number（数値）だけ受け取る、と型で宣言している
function addShipping(price: number): number {
  return price + 500;
}

// 正しい呼び出し: 数値を渡している
console.log(addShipping(1000));

// 間違った呼び出し: 文字列 "1000" を渡している
console.log(addShipping("1000"));
```

▼ 実行結果（`node check.ts`）

```
1500
1000500
```

▼ 型チェックの結果（`npx tsc --noEmit --strict --target esnext check.ts`）

```
check.ts:11:25 - error TS2345: Argument of type 'string' is not assignable to parameter
of type 'number'.

11 console.log(addShipping("1000"));
                           ~~~~~~


Found 1 error in check.ts:11
```

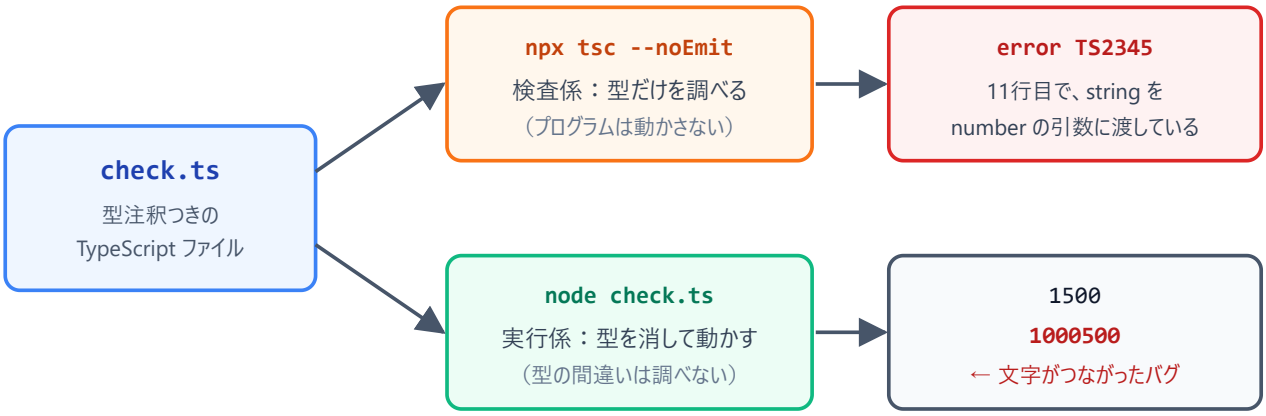


図2-4: tsc は「検査係」、node は「実行係」。node は型を消して動かすだけなので、型の間違いに気づかない

▼ コードを1つずつ分解して解説

解説1: 型注釈で「数値だけ受け取る」と約束する

```
function addShipping(price: number): number {
  return price + 500;
}
```

- `price: number` は「引数 `price` には数値だけを渡してね」という**約束（型注釈）**です。`): number` は「戻り値も数値です」という約束です。
- この約束は、人間と `tsc`（VS Code も中で同じ仕組みを使っています）が読むための**メモ**です。

解説2: node は型を消して、そのまま実行する

```
console.log(addShipping("1000"));
```

- `node check.ts` は、`: number` などの型の部分を消して、ただの JavaScript として実行します。型の約束は**見ていません**。
- そのため文字列 `"1000"` がそのまま関数に入り、`"1000" + 500` が計算されます。`+` の片方が文字列だと「足し算」ではなく「文字のつなげ算」になるので、結果は `1000500` という**おかしな値**になります。エラーで止まらないぶん、気づきにくいバグです。

解説3: tsc は実行せず、約束違反だけを報告する

```
check.ts:11:25 - error TS2345: Argument of type 'string' is not assignable to parameter
of type 'number'.
```

- `npx tsc --noEmit ...` は、プログラムを**動かさずに**型の約束が守られているかだけを調べます。
- `check.ts:11:25` は「`check.ts` の11行目・25文字目」、本文は「string 型の引数は、number 型の引数（パラメータ）に代入できません」という意味です。読み方は8節の冒頭でくわしく練習します。

> **つまずきポイント:「node で動いた」＝「型が正しい」ではない** — `node ファイル名.ts` は型を見ないので、型の間違いがあっても普通に動いてしまいます（上の `1000500` がその例）。型が正しいかは、VS Code の赤い波線か `npx tsc` で確かめる習慣を付けましょう。

NG 例: 型エラーが残ったまま `node` で動かして「動いたからOK」と思ってしまう（上の `check.ts`。結果は `1500` と `1000500`）

OK 例（`check-ok.ts`）：`tsc` のエラーを読んで、文字列を数値に変換してから渡す

```ts
function addShipping(price: number): number {
  return price + 500;
}

// 文字列は Number() で数値に変換してから渡す
console.log(addShipping(1000));
console.log(addShipping(Number("1000")));
```

▼ **実行結果**（`npx tsc --noEmit --strict --target esnext check-ok.ts` は何も表示されずに終わる＝エラーなし。そのあと `node check-ok.ts`）

```
1500
1500
```

> **ヒント:** Next.js では、`npm run dev`（開発サーバー）は型エラーがあっても画面を表示しますが、`npm run build`（公開用のビルド）は型チェックも行い、エラーがあると**ビルドが失敗**します。開発中こそ VS Code の赤い波線をこまめに消しておくのが近道です。

1.2 なぜ TypeScript を使うのか

TypeScript を使う最大の理由は **型安全性** です。以下の表で、JavaScript と TypeScript の違いを比較してみましょう。

観点	JavaScript	TypeScript
型チェック	実行時にエラー発覚	コンパイル時にエラー発覚
エディタ補完	限定的	強力な自動補完
リファクタリング	手動で確認が必要	型の変更で影響範囲を自動検出
ドキュメント性	コメントに頼る	型そのものがドキュメント
バグ発見のタイミング	ユーザーが使用中に発覚	開発中に発覚
学習コスト	低い	やや高い（型の学習が必要）
大規模開発	困難	適している

1.3 JavaScript vs TypeScript の比較コード例

例1: 関数の引数に誤った型を渡すケース

> ▼ **このコードがやること（先に日本語で）**：同じ「挨拶を作る関数」を JavaScript 版と TypeScript 版で書き比べ、「**間違った型の値を渡したとき何が起きるか**」を見ます。JavaScript は誤りに気づかず動いてしまうのに対し、TypeScript は実行する前にエラーで教えてくれる——これがTypeScriptを使う一番のメリットです。

```
// ---- JavaScript ----
// function 宣言。引数 name の型は書けない（書こうとすると構文エラー）。
function greet(name) {
  // 文字列連結で挨拶を作る。name が何の型でも文字列に変換されて連結されてしまう。
  return "こんにちは、" + name + "さん！";
}

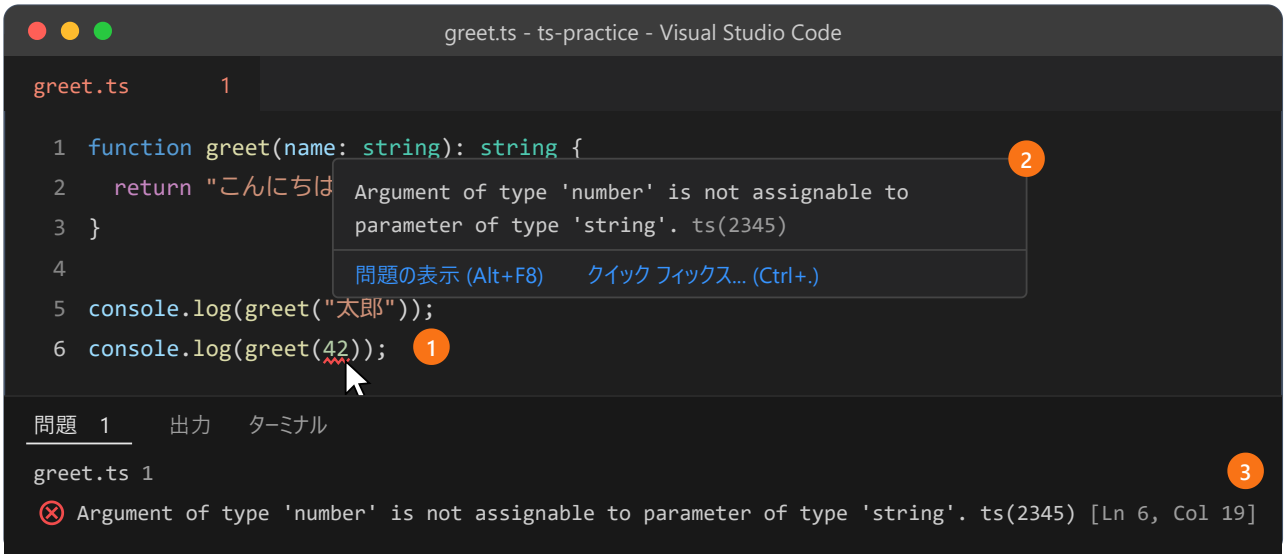
// 数値を渡してもエラーにならない（実行時まで気づけない）
// 引数に数値 42 を渡しても、JavaScript は何も警告しない。
console.log(greet(42));
// 42 が文字列 "42" に勝手に変換されて連結される。バグの温床。
// => "こんにちは、42さん！" ← 意図しない動作
```

```
// ---- TypeScript ----
// : string は「型注釈（Type Annotation）」。name の右の : string は「引数 name は string 型」、) の右の : string は「戻り値も string 型」を表す。
function greet(name: string): string {
  // ロジック自体は JavaScript と同じ。
  return "こんにちは、" + name + "さん！";
}

// コンパイル時にエラーが発生！
// ↑ ここで TypeScript が「string が必要なのに number を渡している」と警告。実行する前に気づける。
console.log(greet(42));
// エラー: Argument of type 'number' is not assignable to parameter of type 'string'
//         （number 型の引数は、string 型のパラメータに代入できません）

// 正しい使い方
// 文字列を渡すと OK。
console.log(greet("太郎"));
// => "こんにちは、太郎さん！"
```

VS Code では、`tsc` を打たなくても、コードを書いたその場で次のように知らせてくれます（図2-5）。



(1) 赤い波線 ＝ 型エラーの場所　(2) マウスを乗せると理由が出る　(3)「問題」パネルに一覧が出る

図2-5: VS Code の画面イメージ。赤い波線・ホバー表示・「問題」パネルの3か所でエラーを教えてくれる

- **(1) 赤い波線** … 型の約束が破られている場所に引かれます。ここでは `42` の下です。
- **(2) ホバー表示** … 波線にマウスを乗せると、理由（エラーメッセージ）が表示されます。最後の `ts(2345)` は**エラー番号**で、ターミナルの `tsc` が出す `error TS2345` と同じものです。番号で検索すると解説記事が見つかります。
- **(3)「問題」パネル** … `Ctrl + Shift + M` で開けます。開いているファイルのエラーが一覧になり、行をクリックするとその場所へジャンプできます。

> **ヒント:** 英語のエラーメッセージは「`Argument of type 'A' is not assignable to parameter of type 'B'`」＝「A 型の引数は、B 型の引数（の席）には座れません」と覚えると読みやすくなります。くわしくは8節の冒頭で練習します。

例2: オブジェクトのプロパティアクセス

> **▼ このコードがやること（先に日本語で）:** オブジェクトのプロパティ名を**タイプミス**したときの挙動を、JavaScript と TypeScript で比べます。JavaScript は黙って `undefined` を返すだけ（バグに気づきにくい）ですが、TypeScript は「そんなプロパティは無いよ」と赤線で即座に指摘し、正しい名前まで提案してくれます。

```javascript
// ---- JavaScript ----
// user オブジェクトを作る。
const user = {
  // name プロパティに文字列。
  name: "田中",
  // age プロパティに数値。, （カンマ）でプロパティを区切る。最後のカンマはあっても無くてもよい（trailing comma）。
  age: 25,
};

// タイプミスしてもエラーにならない（実行時に undefined になる）
// => undefined ← バグ！
// name と nmae（タイプミス）。JavaScript は存在しないプロパティを許し、結果は undefined。表示してから「あれ？」と気づくことが多い。
console.log(user.nmae);
```

```javascript
// ---- TypeScript ----
// 型注釈を書かなくても、TypeScript が自動で { name: string; age: number; } と「推論」してくれる。
const user = {
  name: "田中",
  age: 25,
};

// タイプミスするとコンパイル時にエラー！
// ↑ VS Code が即座に赤線。
console.log(user.nmae);
// エラー: Property 'nmae' does not exist on type '{ name: string; age: number; }'.
//         ('nmae' というプロパティは { name; age } 型には存在しません)
// 「もしかして name のこと？」とサジェストもしてくれる。
// Did you mean 'name'?

// 正しいアクセス
// => "田中"
console.log(user.name);
```

例3: 配列操作での型安全性

> **▼ このコードがやること（先に日本語で）**：「数値だけの配列」のつもりが、うっかり文字列を混ぜてしまったときの違いを比べます。JavaScript では計算結果が `"15six"` のように壊れてしまいますが、TypeScript では `number[]`（数値だけの配列）と宣言しておくことで、文字列を入れた瞬間にエラーで止めてくれます。

```
// ---- JavaScript ----
// 配列リテラル。JavaScript では配列の中身の型は問わない。
const numbers = [1, 2, 3, 4, 5];

// 文字列を追加してもエラーにならない
// ← バグの原因になる
// push は配列末尾に要素を追加するメソッド。文字列でも入ってしまう。
numbers.push("six");

// 後で計算しようとすると予期しない結果に
// reduce は配列を左から畳み込んで1つの値にするメソッド。第2引数 0 は初期値。a が累積値、b が現在の要素。
const sum = numbers.reduce((a, b) => a + b, 0);
// => "15six" ← 文字列連結になってしまう！
// 数値 + 文字列 → 文字列連結のルールが発動。"1+2+3+4+5=15" + "six" = "15six"。
console.log(sum);
```

```
// ---- TypeScript ----
// number[] は「number 型の配列」。これで「数値以外は入れられない」と TypeScript が保証してくれる。
const numbers: number[] = [1, 2, 3, 4, 5];

// 文字列を追加しようとするとコンパイルエラー！
// ↑ 「number しか入らない配列」に string を push しようとして即エラー。
numbers.push("six");
// エラー: Argument of type 'string' is not assignable to parameter of type 'number'

// 正しい使い方
// 6 は number なので OK。
numbers.push(6);
// a, b ともに number と推論されるので、+ は算術加算で動く。
const sum = numbers.reduce((a, b) => a + b, 0);
// => 21
// 1+2+3+4+5+6 = 21
console.log(sum);
```

2. 基本的な型

TypeScript には豊富な型が用意されています。ここでは、すべての基本型を詳しく見ていきます。

> **たとえ話**: 変数は「**名札の付いた箱**」、型は「**箱の口の形**」です。丸い口の箱には丸いもの（数値）しか入らず、細長い口の箱には細長いもの（文字列）しか入りません。形が合わないものを入れようとすると、TypeScript が「入りません！」と止めてくれます。これが型エラーの正体です。

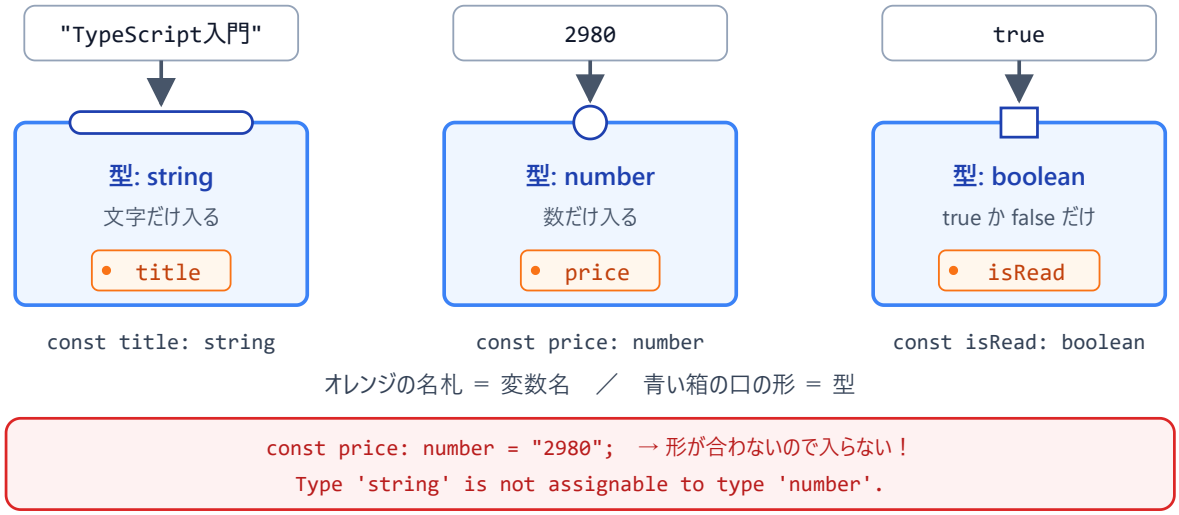
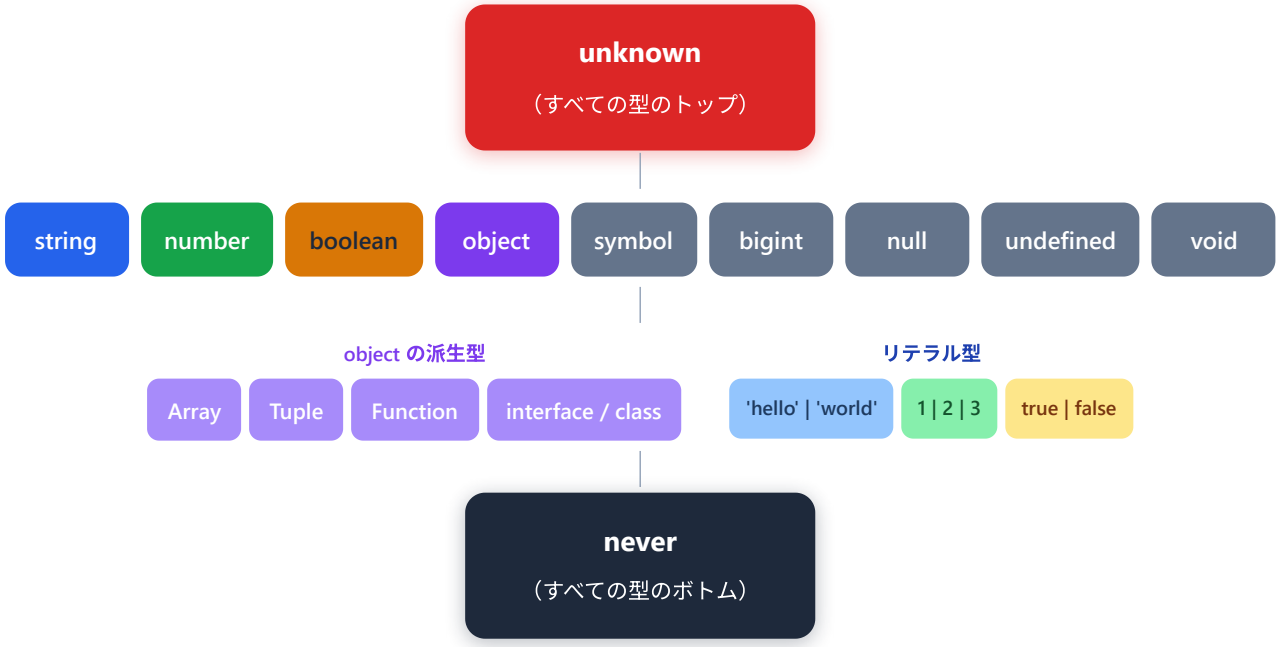


図2-6: 変数＝名札の付いた箱、型＝箱の口の形。形の合わない値は入れられない

TypeScript の型階層

まず、TypeScript の型システム全体を俯瞰してみましょう。



> **ポイント**: `unknown` はすべての型の「親」で、`never` はすべての型の「子」です。`unknown` にはどんな値も代入できますが、`never` にはどんな値も代入できません。

2.1 string（文字列型）

文字列を表す型です。シングルクォート、ダブルクォート、テンプレートリテラルのいずれでも使えます。

> ▼ **このコードがやること（先に日本語で）**：「文字列」を表す `string` 型の変数を、ダブルクォート・シングルクォート・テンプレートリテラルの3通りで作ります。覚えてほしいのは `const 変数名: string = 値` という**型注釈**の形 —— `: string` が「この箱には文字列しか入れません」という宣言です。

```ts
// ============================================================================
// string 型のサンプル – 「文字列」を意味する型
// ============================================================================
// const は「再代入禁止の変数宣言」。一度値を入れたら別の値で上書きできない。
// 変数名の右の : string が「型注釈」。「この変数は string 型ですよ」という宣言。
// = の右側が「初期値」。

// (1) ダブルクォート " で囲んだ文字列
//     greeting という名前の string 型変数に "こんにちは" を入れる。
const greeting: string = "こんにちは";

// (2) シングルクォート ' でも書ける（JavaScript/TypeScriptでは両者同じ意味）
//     チームのコーディング規約で「" に統一」「' に統一」を決めるのが普通。
const name: string = '太郎';

// (3) テンプレートリテラル: バッククォート ` で囲み、${変数名} で値を埋め込める
//     greeting と name の中身が連結されて入る。
//     文字列連結を + で書くより読みやすい（"こんにちは" + "、" + name + "さん！" よりずっと短い）
const message: string = `${greeting}、${name}さん！`;

// (4) console.log は「ターミナルやブラウザのコンソールに値を表示する」関数
//     第1引数で渡された文字列を1行表示する。
console.log(message);
// ▼ 実行結果（ターミナルにこう出る）
// こんにちは、太郎さん！


// (5) テンプレートリテラルは改行もそのまま含められる
//     これが文字列の中に改行コード(\n)として保持される。
const multiLine: string = `
  1行目
  2行目
  3行目
`;
console.log(multiLine);
// ▼ 実行結果
// （空行）
//   1行目
//   2行目
//   3行目
// （空行）
```

```
// ============================================================================
// string 型でエラーになる例 — VS Code が赤線で警告してくれる
// ============================================================================

// (1) 数値を string 型変数に入れようとしている → ❌
//     42 は数値リテラル。string が期待される場所には置けない。
const title: string = 42;
// ▼ エラーメッセージ
// Type 'number' is not assignable to type 'string'.
//    (number 型は string 型に代入できません)

// (2) 真偽値を string 型変数に入れようとしている → ❌
const flag: string = true;
// Type 'boolean' is not assignable to type 'string'.

// (3) null を string 型変数に入れようとしている → ❌
//     ※ tsconfig.json で "strictNullChecks": true のとき
//        null や undefined はそれぞれ独立した型として扱われる。
const nothing: string = null;
// Type 'null' is not assignable to type 'string'.
```

2.2 number（数値型）

整数・浮動小数点数の両方を含む数値型です。JavaScript と同様に、TypeScript の number はすべて浮動小数点数として扱われます。

> **▼ このコードがやること（先に日本語で）**：「数値」を表す `number` 型に入れられる色々な値——整数・小数・負の数・16進数・特殊な値（ `Infinity` や `NaN` ）など——を一通り作って表示します。今は「整数も小数も同じ `number` 型」という点だけ押さえれば十分で、特殊な記法はコメントを眺めるだけでOKです。

```ts
// ============================================================================
// number 型のサンプル – 「数値（整数も小数も）」を意味する型
// ============================================================================

// (1) 整数
const age: number = 25;

// (2) 小数（浮動小数点数）
const price: number = 1980.5;

// (3) 負の数
const negative: number = -10;

// (4) 16進数（0x で始まる）。Web開発で色コードを書くときに使う
//     0xff の f は 15 を意味する1桁の16進文字。ff = 15*16 + 15 = 255
// 256 - 1 = 255
const hex: number = 0xff;

// (5) 2進数（0b で始まる）。フラグ操作などで使う
//     0b1010 = 1*8 + 0*4 + 1*2 + 0*1 = 10
// = 10
const binary: number = 0b1010;

// (6) 8進数（0o で始まる）。Linuxのファイル権限で見る形
//     0o744 = 7*64 + 4*8 + 4*1 = 484
// = 484
const octal: number = 0o744;

// (7) 数値リテラル区切り（_ で読みやすくする）
//     1_000_000 と書いても 1000000 と書いても結果は同じ。実行時の値に _ は含まれない。
// = 1000000（百万）
const big: number = 1_000_000;

// (8) 特殊な数値
//     Infinity = 無限大、NaN = Not a Number（不正な数値計算の結果）
const inf: number = Infinity;
const notANumber: number = NaN;

console.log(age, price, negative, hex, binary, octal, big);
// ▼ 実行結果
// 25 1980.5 -10 255 10 484 1000000

// → Infinity（0で割ると無限大）
console.log(1 / 0);
// → NaN（0÷0 は数学的に未定義）
console.log(0 / 0);
// → NaN（文字列を数値として掛けると NaN）
// ※ これは JavaScript での動きを見せるための行。TypeScript ではこの行自体が型エラーになる:
//   The left-hand side of an arithmetic operation must be of type 'any', 'number',
```

```
'bigint' or an enum type.
//    （算術演算の左側は any・number・bigint・enum のどれかでなければなりません）
console.log("a" * 2);
```

```
// ============================================================================
// number 型でエラーになる例
// ============================================================================

// (1) 数字に見えるが文字列なのでダメ → ❌
const count: number = "5";
// Type 'string' is not assignable to type 'number'.

// (2) "100" + 50 は JavaScript的には string になる（"10050"になる）→ TS は型エラー扱い
const total: number = "100" + 50;
// Type 'string' is not assignable to type 'number'.
//
// ※ TypeScript はこれを書く前にエラーで止めてくれる。
//   JavaScript だけだと、後の計算で予想外のバグになる。

// (3) 数値型変数 a と文字列型変数 b を + するとどうなる?
//     → JavaScript では "10" + "20" のように文字列連結に化ける。
//     TypeScript は「これは number として使えませんよ」と止めてくれる。
const a: number = 10;
const b: string = "20";
const result: number = a + b;
// Type 'string' is not assignable to type 'number'.
```

2.3 boolean（真偽値型）

`true` または `false` のどちらかを持つ型です。

> **▼ このコードがやること（先に日本語で）**：「はい/いいえ」を表す `boolean` 型（`true` か `false` の2択）を扱います。直接 `true` / `false` を入れるだけでなく、`age >= 18` のような**比較式の結果**もそのまま `boolean` になる、という点に注目してください。条件判定の土台になる型です。

```ts
// ============================================================================
// boolean 型のサンプル – 「true か false」しか入らない型
// ============================================================================

// (1) 直接 true / false を入れる
// true = 「アクティブ」
const isActive: boolean = true;
// false = 「未完了」
const isCompleted: boolean = false;

// (2) 比較式の結果も boolean になる
//     >= は「以上」、=== は「等しい（型もチェック）」
//     式が成り立てば true、成り立たなければ false が返る。
// age が 25 なので true
const isAdult: boolean = age >= 18;
// name が "太郎" なので false
const isEmpty: boolean = name === "";

// (3) 論理演算子（&& と ||）
//     && (AND) : 両方 true なら true、片方でも false なら false
//     || (OR)  : 片方でも true なら true、両方 false なら false
//     ! (NOT)  : true ⇔ false を反転
// true && true = true
const canAccess: boolean = isActive && isAdult;

console.log(isActive, isCompleted, isAdult, isEmpty, canAccess);
// ▼ 実行結果
// true false true false true
```

```ts
// ============================================================================
// boolean 型でエラーになる例
// ============================================================================

// (1) 数字 1 は「真っぽい値」だが boolean 型ではない → ❌
const flag: boolean = 1;
// Type 'number' is not assignable to type 'boolean'.

// (2) "true" は文字列であって、真偽値ではない → ❌
const active: boolean = "true";
// Type 'string' is not assignable to type 'boolean'.

// (3) 0 も「偽っぽい値」だが boolean 型ではない → ❌
//     JavaScript では if (0) {...} は実行されない（falsy扱い）が、
//     boolean 型としては number の 0 と boolean の false は別物。
const truthy: boolean = 0;
// Type 'number' is not assignable to type 'boolean'.
```

2.4 array（配列型）

同じ型の要素を複数格納するコレクションです。書き方は2通りあります。

> ▼ **このコードがやること（先に日本語で）**：「同じ型の値を順番に並べたリスト」である**配列型**を作り、追加（ `push` ）・取り出し（ `[添え字]` ）・長さ（ `.length` ）といった基本操作を試します。書き方は `string[]` と `Array<string>` の2通りありますが、どちらも意味は同じ。よく使うのは前者の `型名[]` です。

```ts
// ============================================================================
// 配列型のサンプル – 「同じ型の値を順番に並べたリスト」
// ============================================================================

// ──────────────────────────── 書き方1: 型名 + [] ────────────────────────────
// string[] は「string が並んだ配列」の意味。一番一般的な書き方。

// (1) 文字列の配列
const fruits: string[] = ["りんご", "みかん", "バナナ"];
//                         [0]       [1]       [2]   ← 添え字（0から始まる）

// (2) 数値の配列
const scores: number[] = [85, 90, 78, 92];

// (3) 真偽値の配列
const flags: boolean[] = [true, false, true];


// ──────────────────────────── 書き方2: Array<型名> ──────────────────────────
// 結果は [] 記法と同じ。読み手の好みで選ぶ。

const colors: Array<string> = ["赤", "青", "緑"];
const ages: Array<number> = [20, 25, 30];


// ──────────────────────────── 配列の主な操作 ────────────────────────────────

// (4) push: 配列の末尾に要素を追加（破壊的に変更される）
//     型が一致する値だけ受け付ける（string配列に number は入らない）
// fruits = ["りんご", "みかん", "バナナ", "ぶどう"]
fruits.push("ぶどう");
// scores = [85, 90, 78, 92, 100]
scores.push(100);

// (5) [添え字] でアクセス。存在しない添え字は undefined（厳密モードでは型エラー）
// = "りんご"
const first: string = fruits[0];
console.log(first);
// ▼ 実行結果
// りんご

// (6) 配列の長さを取る
console.log(fruits.length);
// ▼ 実行結果
// 4

// (7) 空配列を作るときも型注釈を書いておくのが安全
//     書かないと、場所によって any[]（何でも入る）や never[]（何も入らない）と推論されてしまう。
const emptyStrings: string[] = [];
```

```
// OK
emptyStrings.push("hello");
console.log(emptyStrings);
// ▼ 実行結果
// [ 'hello' ]
```

▼ コードを1つずつ分解して解説

上のサンプルには、配列を扱ううえで初心者がつまずきやすいポイントがいくつかあります。順番に、**1つずつ**ていねいに見ていきましょう。

解説1: 配列の型の書き方（`型名[]`）

```
const fruits: string[] = ["りんご", "みかん", "バナナ"];
//                        [0]       [1]       [2]   ← 添え字（0から始まる）
```

- `const fruits` … `const`（再代入禁止の変数）で `fruits` という名前の箱を作っています。
- `: string[]` … これが配列の**型注釈**です。`string[]` は「**`string`（文字列）がいくつも並んだ配列**」という意味で、`型名` のうしろに `[]`（角カッコ）を付けると「その型の配列」を表します。
- `= ["りんご", ...]` … `[ ]` の中に値を `,`（カンマ）で並べると**配列リテラル**（配列そのものの書き方）になります。
- 各要素には先頭から `0, 1, 2, ...` と**添え字（インデックス）**が振られます。`fruits[0]` が `"りんご"`、`fruits[1]` が `"みかん"` です。**1番目が `0` から始まる**点が最初は戸惑いやすいので要注意です。

> **用語: 添え字（インデックス）** … 配列の中の「何番目か」を表す番号。プログラミングでは **0 から数える**のが基本です（1ではありません）。

解説2: もう1つの書き方（`Array<型名>`）

```
const colors: Array<string> = ["赤", "青", "緑"];
const ages: Array<number> = [20, 25, 30];
```

- `Array<string>` は、解説1の `string[]` と**まったく同じ意味**です。「文字列の配列」を別の書き方で表しているだけです。
- `< >`（山カッコ）の中に要素の型を書きます。これは後の「ジェネリクス」（5章）で出てくる書き方の一種です。
- どちらを使ってもOKですが、**よく使われるのは短い `string[]` のほう**です。「`Array<...>` という書き方もあるんだな」と知っておけば十分です。

解説3: 配列に要素を追加する（ `push` ）

```
// fruits = ["りんご", "みかん", "バナナ", "ぶどう"]
fruits.push("ぶどう");
// scores = [85, 90, 78, 92, 100]
scores.push(100);
```

- `.push(値)` は「**配列の末尾（いちばん後ろ）に値を1つ追加する**」メソッドです。
- 大事なのは、`push` は**元の配列そのものを書き換える**点です（これを「破壊的」と言います）。新しい配列を作るのではなく、`fruits` 自身が変化します。
- 型注釈のおかげで、`string[]` の配列には文字列しか `push` できません。`fruits.push(42)` のように数値を入れようとすると、TypeScript が即エラーで教えてくれます。

> **用語: メソッド** … 値（ここでは配列）が持っている「機能（関数）」のこと。`配列.push(...)` のように `.`（ドット）でつないで呼び出します。

解説4: 要素の取り出しと長さ

```
// = "りんご"
const first: string = fruits[0];
console.log(first);
// ▼ 実行結果
// りんご

console.log(fruits.length);
// ▼ 実行結果
// 4
```

- `fruits[0]` … `[ ]` に添え字を書くと、その位置の要素を取り出せます。`[0]` は1番目（`"りんご"`）です。
- `fruits.length` … `.length`（レングス）は「**その配列に要素が何個入っているか**」を表すプロパティです。解説3で `"ぶどう"` を追加したので、要素数は `4` になっています。
- `console.log(...)` は、カッコの中の値をターミナルやブラウザのコンソールに表示する関数です。

> **用語: プロパティ** … 値が持っている「データ（情報）」のこと。`.length` のように `.` でつないで読み取ります。`()` を付けないのがメソッドとの違いです。

配列は「**番号の付いたロッカーが横一列に並んだもの**」とイメージすると分かりやすくなります。番号（添え字）は 0 から始まり、最後の番号は「`length` − 1」です。

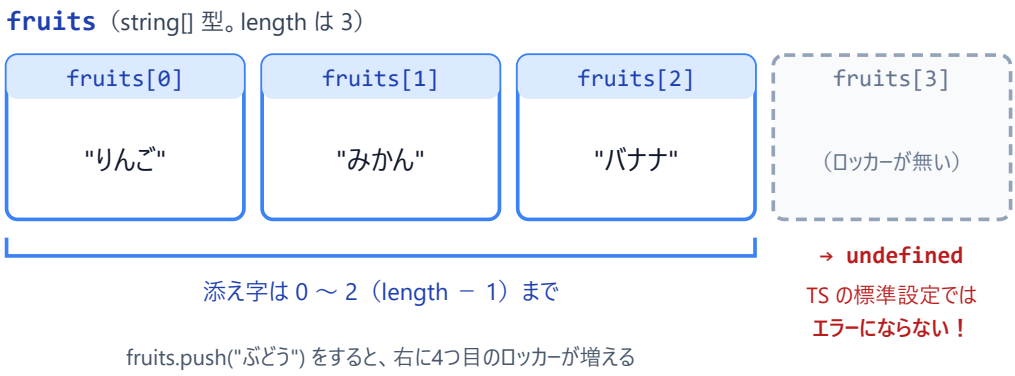


図2-7: 配列＝番号付きロッカー。存在しない番号を開けると undefined が返る

> **つまずきポイント: 範囲外の添え字は、TypeScript でも見逃される** — `fruits` の型は `string[]` なので、TypeScript は `fruits[3]` の型も `string` だと考えます。「その番号のロッカーが本当にあるか」までは、標準の設定ではチェックしません。その結果、実行して初めてクラッシュします。

NG 例（`index-ng.ts`）: 存在しない4番目を取り出して、そのまま文字列として使う

```ts
const fruits: string[] = ["りんご", "みかん", "バナナ"];

// 添え字 3 は存在しない（0, 1, 2 までしかない）
const fourth = fruits[3];
console.log(fourth);
console.log(fourth.toUpperCase());
```

`npx tsc --noEmit --strict --target esnext index-ng.ts` は**何も表示しない（エラーなし）**のに、実行すると落ちます。

▼ 実行結果（`node index-ng.ts`。node 内部の行は省略）

```
undefined
C:\ts-practice\index-ng.ts:6
console.log(fourth.toUpperCase());
                   ^

TypeError: Cannot read properties of undefined (reading 'toUpperCase')

Node.js v25.9.0
```

OK 例（`index-ok.ts`）: 取り出した値が `undefined` でないか確かめる／`??` で代わりの値を用意する

```ts
const fruits: string[] = ["りんご", "みかん", "バナナ"];

// 取り出したあと、undefined でないか確かめてから使う
const fourth = fruits[3];
if (fourth !== undefined) {
  console.log(fourth.toUpperCase());
} else {
  console.log("4番目の果物はありません");
}

// ?? で「無かったときの代わりの値」を用意しておく
const fifth = fruits[4] ?? "（なし）";
console.log(fifth);
```

▼ 実行結果

```
4番目の果物はありません
（なし）
```

- `TypeError: Cannot read properties of undefined (reading 'toUpperCase')` は「`undefined` の `toUpperCase` を読もうとしました」という意味の**実行時エラー**です。JavaScript で最もよく見るエラーの1つなので、形を覚えておきましょう。
- `??`（ヌル合体演算子）は「左が `null` か `undefined` なら右の値を使う」演算子です（2.11 と発展編でくわしく扱います）。

> **ヒント:** `tsconfig.json` に `"noUncheckedIndexedAccess": true` を足すと、`fruits[3]` の型が `string | undefined` になり、上の NG 例は `error TS18048: 'fourth' is possibly 'undefined'.`（`fourth` は undefined かもしれません）という型エラーで止まるようになります（`npx tsc` に `--noUncheckedIndexedAccess` を付けても試せます）。より安全になる代わりに、チェックを書く手間が増える設定です。

解説5: 空の配列には必ず型注釈を付ける

```ts
const emptyStrings: string[] = [];
// OK
emptyStrings.push("hello");
```

- 最初は空っぽ（`[]`）の配列を作りたいことがあります。
- このとき**型注釈 `: string[]` を必ず書く**のがコツです。中身が空だと、TypeScript は初期値から「何の配列か」を判断できないからです。

- 型注釈が無いと、書く場所によって推論結果が変わります。`const list = [];` のように**変数へ直接**入れた場合は、ひとまず `any[]`（何でも入る配列）として扱われ、型チェックが甘くなります。一方、`{ books: [] }` のように**オブジェクトの中**に書いた場合（や、第3章で学ぶ React の `useState([])`）は `never[]`（何も入れられない配列）と推論され、あとで `push` するとエラーになります（下のつまずきポイント）。
- 型注釈を書いておけば、「これは文字列を入れていく配列だ」と TypeScript に伝わり、`emptyStrings.push("hello")` が正しく通ります。

> **つまずきポイント: オブジェクトの中の空配列 `[]` は `never[]` になる** — 「本棚」のようなオブジェクトに、最初は空の配列を持たせておき、あとから本を `push` するのはよくある形です。ところが型注釈が無いと、TypeScript は `books` を「**何も入れてはいけない配列（`never[]`）**」だと推論してしまいます。

NG 例（`empty-ng.ts`）

```ts
// 本棚オブジェクト。books は最初は空っぽの配列
const shelf = {
  name: "本棚A",
  books: [],
};

shelf.books.push("TypeScript入門");
console.log(shelf);
```

▼ 型チェックの結果（`npx tsc --noEmit --strict --target esnext empty-ng.ts`）

```
empty-ng.ts:7:18 - error TS2345: Argument of type '"TypeScript入門"' is not assignable
to parameter of type 'never'.

7 shelf.books.push("TypeScript入門");
                   ~~~~~~~~~~~~~~~


Found 1 error in empty-ng.ts:7
```

（`node empty-ng.ts` で実行すると型を見ないので動いてしまい、`{ name: '本棚A', books: [ 'TypeScript入門' ] }` と表示されます。型エラーを見逃しやすい典型例です。）

OK 例（`empty-ok.ts`）：先に「形」を型で決めておく

```ts
// 本棚の「形」を先に型で決めておく
interface Shelf {
  name: string;
  books: string[];
}

// 型注釈を付けたので、books は「string の配列」だと分かる
const shelf: Shelf = {
  name: "本棚A",
  books: [],
};

shelf.books.push("TypeScript入門");
console.log(shelf);
```

▼ **実行結果**（`tsc` はエラーなし）

```
{ name: '本棚A', books: [ 'TypeScript入門' ] }
```

- エラー文の `parameter of type 'never'` は「この `push` の引数には **何も** 渡せません」という意味です。`never[]` を見かけたら「空配列に型を付け忘れたな」と思い出しましょう。
- `interface`（4節で学びます）の代わりに、`books: [] as string[]` と書いて「この空配列は文字列の配列です」と伝える方法もあります。
- 第3章の React では、`useState<Book[]>([])` のように `< >` で型を渡して同じ問題を防ぎます。

```ts
// --- エラーになる例 ---
// number[] と宣言しているのに "three" は string。混在は許されない。
const numbers: number[] = [1, 2, "three"];
// エラー: Type 'string' is not assignable to type 'number'

// string 専用配列。
const items: string[] = ["a", "b", "c"];
// 42 は number なので入れられない。
items.push(42);
// エラー: Argument of type 'number' is not assignable to parameter of type 'string'

// 異なる型が混在する配列を number[] として定義
// true (boolean) も "hello" (string) も number ではない → 2件エラーになる。
const mixed: number[] = [1, true, "hello"];
// エラー: Type 'boolean' is not assignable to type 'number'
// エラー: Type 'string' is not assignable to type 'number'
```

2.5 tuple（タプル型）

タプル（tuple：複数の値を順序付きで束ねた、長さが固定のデータ構造）型 は、**固定長で、各位置の型が決まっている配列** です。通常の配列と異なり、要素ごとに異なる型を持てます。

> **▼ このコードがやること（先に日本語で）：**「［名前，年齢］のように、**位置ごとに型と個数が決まった配列**」である**タプル型**を作って使います。普通の配列と違い「1番目は文字列・2番目は数値」と並びが固定される点がポイントです。座標［緯度，経度］のように、決まった組み合わせを表すのに便利です。

```
// --- 正しい例 ---

// [名前, 年齢] のタプル
// 型注釈の [string, number] が「タプル型」。1番目=string, 2番目=number と位置で型が固定される。
const person: [string, number] = ["田中", 30];

// [ID, 名前, アクティブフラグ] のタプル
// 3要素のタプル。位置ごとに型が決まっている。
const record: [number, string, boolean] = [1, "太郎", true];

// 要素へのアクセス（型が正しく推論される）
// [0] は1番目の要素を取り出す（配列・タプルともに添え字は 0 始まり）。TypeScript は「1番目は string」と知っているので型が string になる。
// "田中" ← string 型
const personName: string = person[0];
// 30 ← number 型
// [1] は2番目。number と推論される。
const personAge: number = person[1];

// 分割代入も可能
// 配列形式の分割代入。person[0] → name, person[1] → age に代入。型もそれぞれ string, number に決まる。
const [name, age] = person;
// name は string 型、age は number 型

// オプショナルなタプル要素
// 2要素目に ? を付けると「省略してもよい」タプル要素になる。number? は number | undefined と同じ意味。
const optionalTuple: [string, number?] = ["田中"];
// 2番目の要素はあってもなくてもよい
```

```
// --- エラーになる例 ---
// 1番目が string でなければならないのに number。2番目が number でなければならないのに string。位置の型と合わない。
const pair: [string, number] = [42, "田中"];
// エラー: Type 'number' is not assignable to type 'string'（1番目）
// エラー: Type 'string' is not assignable to type 'number'（2番目）

// タプル型は「長さも固定」。3要素必要なのに 2要素しかないとエラー。
const triple: [string, number, boolean] = ["太郎", 25];
// エラー: Source has 2 element(s) but target requires 3

// 型が異なる位置へのアクセス
// 1番目が string、2番目が number。
const data: [string, number] = ["hello", 42];
// [0] は string なのに number に代入しようとしているのでエラー。
const wrongType: number = data[0];
// エラー: Type 'string' is not assignable to type 'number'
```

2.6 object（オブジェクト型）

キーと値のペアを持つ構造化されたデータ型です。

> **▼ このコードがやること（先に日本語で）**：「キー：値」の組をまとめた**オブジェクト型**を、`{ name: string; age: number }` のように形を指定して作ります。あわせて、省略可能な `?`（オプショナル）、変更禁止の `readonly`、オブジェクトの入れ子（ネスト）も扱います。「どんなプロパティを持つか」を型で表せるのがポイントです。

```ts
// --- 正しい例 ---

// オブジェクトリテラル型
// { ... } の中の「キー: 型」の並びが「オブジェクト型」の宣言。3つの必須プロパティを持つことを意味する。区切りは ; (または , でもOK)。
const user: { name: string; age: number; email: string } = {
  // string プロパティ。
  name: "田中太郎",
  // number プロパティ。
  age: 30,
  // string プロパティ。
  email: "tanaka@example.com",
};

// オプショナルプロパティ（? をつける）
// description? の ? は「あってもなくてもよい」を表す。
const product: { name: string; price: number; description?: string } = {
  name: "TypeScript入門書",
  price: 2980,
  // description は省略可能
  // description は書かなくても OK。値は undefined になる。
};

// 読み取り専用プロパティ
// readonly キーワード = 「代入後は変更禁止」。config.apiUrl = ... と書くとエラーになる。
const config: { readonly apiUrl: string; readonly timeout: number } = {
  apiUrl: "https://api.example.com",
  timeout: 5000,
};

// ネストしたオブジェクト
// 型注釈の中にも { ... } を入れ子にできる。
const company: {
  name: string;
  // address 自体がオブジェクト型。
  address: {
    city: string;
    zipCode: string;
  };
} = {
  name: "サンプル株式会社",
  address: {
    city: "東京",
    zipCode: "100-0001",
  },
};
```

▼ コードを1つずつ分解して解説

オブジェクト型には、初心者がつまずきやすい書き方（オプショナル・`readonly`・ネスト）が詰まっています。順番に、**1つずつ**ていねいに見ていきましょう。

解説1: オブジェクト型の基本の書き方

```
const user: { name: string; age: number; email: string } = {
  name: "田中太郎",
  age: 30,
  email: "tanaka@example.com",
};
```

- `: { name: string; age: number; email: string }` の部分が**オブジェクトの型注釈**です。`{ }` の中に「**キー名: 型**」の組を並べて、「このオブジェクトはどんな項目を持つか」を宣言します。
- ここでは「`name`（文字列）・`age`（数値）・`email`（文字列）の3つを**必ず**持つ」という意味になります。区切りは `;`（セミコロン）でも `,`（カンマ）でもOKです。
- `= { name: "田中太郎", ... }` が実際の値（オブジェクトリテラル）。**型側の** `{ }` は「形の宣言」、**値側の** `{ }` は「中身そのもの」という違いに注意してください。
- 型で宣言した項目が1つでも欠けていたり、型が違ったりすると、TypeScript が即エラーで教えてくれます。

> **用語: プロパティ（キーと値）** … オブジェクトの中の「`name: "田中太郎"`」のような1組のこと。左の `name` を「キー（項目名）」、右の `"田中太郎"` を「値」と呼びます。

解説2: オプショナルプロパティ（`?`）

```
const product: { name: string; price: number; description?: string } = {
  name: "TypeScript入門書",
  price: 2980,
  // description は省略可能
};
```

- キー名のうしろに付いている `?`（`description?`）が**オプショナル**の印です。「この項目は**あってもなくてもよい**」という意味になります。
- そのため上の例では `description` を書いていませんが、エラーになりません。
- 省略した場合、`product.description` の値は `undefined`（値が無い状態）になります。
- 「必須にしたいなら `?` なし、任意にしたいなら `?` あり」と覚えておきましょう。

> **用語: オプショナル** … 「省略可能」という意味。`?` を付けると、その項目は書いても書かなくてもよくなります。

解説3: 読み取り専用プロパティ（`readonly`）

```
const config: { readonly apiUrl: string; readonly timeout: number } = {
  apiUrl: "https://api.example.com",
  timeout: 5000,
};
```

- キー名の前に付く `readonly`（リードオンリー）は「**一度値を入れたら、あとから変更できない**」という印です。
- 例えば `config.apiUrl = "別のURL"` と書こうとすると、TypeScript がエラーで止めてくれます。
- 「設定値（API の URL など）のように、途中で書き換わってほしくない項目」に付けると、うっかり変更によるバグを防げます。

> **用語: readonly** ... 「読み取り専用」。値を読むことはできるが、あとから書き換える（再代入する）ことはできません。

解説4: ネスト（入れ子）したオブジェクト

```
const company: {
  name: string;
  address: {
    city: string;
    zipCode: string;
  };
} = {
  name: "サンプル株式会社",
  address: {
    city: "東京",
    zipCode: "100-0001",
  },
};
```

- オブジェクトの中に、さらにオブジェクトを入れることを**ネスト（入れ子）**と呼びます。
- ここでは `company` の中の `address` が、それ自体「`city` と `zipCode` を持つオブジェクト」になっています。
- 型注釈の側も、値の側も、`{ }` の中に `{ }` を書く「入れ子」の形になっている点に注目してください。形（型）と中身（値）が対応しています。
- 取り出すときは `company.address.city`（東京）のように、`.`（ドット）をつなげて深い階層にアクセスします。

> **用語: ネスト（入れ子）** ... あるものの中に、同じ種類のものをさらに入れること。オブジェクトの中のオブジェクト、配列の中の配列、などを指します。

```ts
// --- エラーになる例 ---
// age が必須なのに...
const user: { name: string; age: number } = {
  name: "田中",
  // age がない！
  // 必須プロパティ漏れはエラー。
};
// エラー: Property 'age' is missing in type '{ name: string; }'

// 型で許可されているのは name と price のみ。
const item: { name: string; price: number } = {
  name: "ペン",
  price: 100,
  // 余分なプロパティ
  // 余計なプロパティ（型定義に無いキー）もエラー。これを「過剰プロパティチェック」と呼ぶ。
  color: "赤",
};
// エラー: Object literal may only specify known properties,
// and 'color' does not exist in type '{ name: string; price: number; }'

// readonly プロパティへの再代入
// readonly なので theme は読み取り専用。
const settings: { readonly theme: string } = { theme: "dark" };
// 後から書き換えようとするとエラー。
settings.theme = "light";
// エラー: Cannot assign to 'theme' because it is a read-only property
```

2.7 any 型

すべての型チェックを無効化する 型です。どんな値でも受け入れ、どんな操作も許可します。

> **▼ このコードがやること（先に日本語で）**：「何でもアリ」になってしまう `any` 型の挙動を体験します。`any` を使うとどんな値も代入でき、存在しないメソッド呼び出しすらエラーになりません——つまり TypeScript の安全装置が全部オフになります。**`any` は原則使わない**、と心に刻むための例です。

```ts
// --- any の使用例（非推奨だが動く）---
// any は「何でもアリ型」。型チェックを全部スキップする魔法のキーワード（=危険）。
let anything: any = "文字列";
// OK（number を代入）
// どんな型の値も再代入できる。
anything = 42;
// OK（boolean を代入）
// ↑同じく。
anything = true;
// OK（配列を代入）
// ↑同じく。
anything = [1, 2, 3];
// foo というメソッドは無いが、TypeScript は何も言わない。実際に動かすと TypeError で落ちる。
// OK（存在しないメソッドも呼べる → 実行時エラー！）
anything.foo();
// .bar が undefined、その .baz を読もうとして実行時にクラッシュ。
// OK（存在しないプロパティもアクセス可能 → 実行時エラー！）
anything.bar.baz;
```

> **警告**: `any` を使うと TypeScript を使う意味がほぼなくなります。**原則として `any` は使わないでください。**やむを得ない場合（外部ライブラリの型定義がない場合など）のみ、限定的に使います。

```ts
// --- なぜ any が危険か ---
// 引数 data の型を any にしてしまうと...
function processData(data: any) {
  // 型チェックが一切行われない
  // data が string でなければ実行時エラー
  // .toUpperCase() は string のメソッド。data が string でないと実行時に死ぬ。
  return data.toUpperCase();
}

// OK: "HELLO"
// 文字列なので OK。
processData("hello");
// 実行時エラー: data.toUpperCase is not a function
// 数値に toUpperCase は無い。
processData(42);
// 実行時エラー: Cannot read properties of null
// null の .toUpperCase を読もうとして即死。
processData(null);
```

2.8 unknown 型

`any` の安全な代替です。どんな値も代入できますが、**使う前に型チェックが必要** です。

▼ **このコードがやること（先に日本語で）：** `any` の安全版である `unknown` 型を使います。どんな値も入れられる点は `any` と同じですが、**使う前に「これは文字列？」と型を確かめないと触れない**のが違いです。`typeof` で中身の型を確認してから使う、という流れ（型ガード）に注目してください。

```
// --- unknown の正しい使い方 ---
// unknown 型変数を宣言。「中身は何か分からない」を明示する型。
let value: unknown = "こんにちは";
// OK（代入は自由）
// どんな型でも代入はできる（ここは any と同じ）。
value = 42;
// OK
value = true;

// ただし、そのまま使うことはできない
// ↑ unknown のままだとメソッド呼び出しが禁止される（安全性のため）。
// const upper: string = value.toUpperCase();
// エラー: 'value' is of type 'unknown'

// 型チェック（型ガード）を行ってから使う
// typeof は値の型を文字列で返す演算子。"string"|"number"|"boolean"|"undefined"|"object"|"function"|"symbol"|"bigint" のいずれか。
if (typeof value === "string") {
  // このブロック内では value は string 型として扱える
  // 条件式により TypeScript が「ここでは string と確定」と推論する。これを「型の絞り込み（narrowing）」と呼ぶ。
  // OK
  // string なので toUpperCase が呼べる。
  console.log(value.toUpperCase());
}

if (typeof value === "number") {
  // このブロック内では value は number 型として扱える
  // OK
  // .toFixed(2) は number のメソッドで「小数点以下2桁の文字列に変換」する。
  console.log(value.toFixed(2));
}
```

```ts
// --- any と unknown の比較 ---

// any: 危険（型チェックなし）
// 引数を any にすると...
function unsafeProcess(data: any): string {
  // 実行時に壊れる可能性あり
  // 何でも通るが安全性ゼロ。
  return data.toUpperCase();
}

// unknown: 安全（型チェック必須）
// unknown にすると...
function safeProcess(data: unknown): string {
  // 必ず型ガードを書かないと中身を使えない。
  if (typeof data === "string") {
    // 型チェック済みなので安全
    // string と確定したブロックなので OK。
    return data.toUpperCase();
  }
  // 型が string でなければデフォルト値を返す。
  return "不明なデータ";
}
```

2.9 never 型

決して発生しない値 の型です。主に以下の場面で使われます。

> **▼ このコードがやること（先に日本語で）:**「**決して値を返さない**」ことを表す `never` 型の使いどころを3つ見ます。①必ず例外を投げる関数、②終わらない無限ループ、③ `switch` で全パターンを処理したか確認する「網羅性チェック」です。最初は③が少し難しいので、①②で「正常に終わらない関数の型なんだな」とつかめれば十分です。

```ts
// --- 用途1: 絶対に値を返さない関数 ---
// 戻り値の型 : never は「決して return しない」関数の印。
function throwError(message: string): never {
  // throw = 例外を投げる文。new Error(...) で Error オブジェクトを作る。throw すると関数はここで終わるので、return されない。
  throw new Error(message);
  // この関数は必ず例外を投げるので、正常に値を返すことがない
}

// --- 用途2: 無限ループ ---
// ずっと終わらない関数も never。
function infiniteLoop(): never {
  // while (true) は「true の間ずっと繰り返し」= 無限ループ。
  while (true) {
    // 永遠に終わらない
  }
}

// --- 用途3: 到達不可能なコードの検出（網羅性チェック）---
// ユニオン型で3種類のリテラルを宣言。
type Shape = "circle" | "square" | "triangle";

// shape を受け取って面積を返す関数。
function getArea(shape: Shape): number {
  // switch 文：値ごとに分岐。
  switch (shape) {
    // shape が "circle" のとき。
    case "circle":
      // Math.PI は 円周率 π。半径10の円の面積を返す。
      return Math.PI * 10 * 10;
    case "square":
      // 1辺10 の正方形の面積。
      return 10 * 10;
    case "triangle":
      // 底辺10・高さ10の三角形の面積。
      return (10 * 10) / 2;
    default:
      // すべてのケースを処理済みなら、shape は never 型になる
      // 上の case で全てのリテラルを処理し尽くしているので、ここに来る可能性はなく shape の型は never に絞り込まれる。
      // never 型に shape を代入。これが「網羅性チェック」の魔法。
      const _exhaustiveCheck: never = shape;
      // never は number にも代入可（never はすべての型の部分型）。
      return _exhaustiveCheck;
  }
}
```

```ts
// --- never 型の重要性: 網羅性チェック ---

// Shape に新しい種類を追加した場合
// pentagon を追加
// ↑ あとから1種類追加した、と想像してください。
type Shape = "circle" | "square" | "triangle" | "pentagon";

function getArea(shape: Shape): number {
  switch (shape) {
    case "circle":
      return Math.PI * 10 * 10;
    case "square":
      return 10 * 10;
    case "triangle":
      return (10 * 10) / 2;
    default:
      // "pentagon" のケースが未処理なのでエラーになる！
      // default に来る可能性が「pentagon」として残っているので、shape は "pentagon" 型に絞り込まれる。
      // "pentagon" は never に代入できないのでエラー → 抜け漏れに気づける！
      const _exhaustiveCheck: never = shape;
      // エラー: Type '"pentagon"' is not assignable to type 'never'.
      // → "pentagon" の処理を追加し忘れたことに気づける
      return _exhaustiveCheck;
  }
}
```

2.10 void 型

値を返さない関数 の戻り値の型です。`never` と違い、関数自体は正常に終了します。

> ▼ **このコードがやること（先に日本語で）:「何も値を返さない関数」**の戻り値に付ける `void` 型を使います。`console.log` するだけ・`alert` を出すだけ、といった「処理はするが結果を返さない」関数の印です。`never` と違って関数自体はちゃんと正常に終わる、という点が違います。

```ts
// --- 正しい例 ---
// 戻り値 : void = 「値を返さない関数」。
function logMessage(message: string): void {
  // 文字列を表示するだけ。
  console.log(message);
  // return 文がない、または return; のみ
  // 暗黙的に undefined を返している扱い。
}

function showAlert(text: string): void {
  // alert はブラウザのポップアップ表示関数（Node.js には無い）。
  alert(text);
  // return; は OK（値を返さない）
  // return の後に値を書かなければ void と矛盾しない。
  return;
}

// void 型の変数には undefined のみ代入可能
// void 変数に入れられるのは undefined のみ。
const result: void = undefined;
```

```ts
// --- エラーになる例 ---
// void と宣言しているのに...
function greet(name: string): void {
  // 文字列を return しているので矛盾。
  return `こんにちは、${name}さん`;
  // エラー: Type 'string' is not assignable to type 'void'
  // void なのに値を返している
}

// void に文字列を入れられない。
const value: void = "hello";
// エラー: Type 'string' is not assignable to type 'void'

// void に数値も入れられない。
const num: void = 42;
// エラー: Type 'number' is not assignable to type 'void'
```

2.11 null と undefined

`null`（ヌル：明示的な「無」）は「値が意図的に空」であることを表し、`undefined`（アンディファインド：未定義）は「値が未定義」であることを表します。

使い分けの目安:

- `undefined` ：自然発生する「値がない」。変数を宣言しただけで初期化していない、関数が何も return しないとき、オブジェクトの存在しないプロパティを読んだとき。
- `null` ：プログラマが「ここは意図的に空ですよ」と明示するときに使う。例えば API で「ユーザーが見つからない」場合に `null` を返すなど。
- **迷ったら `undefined` を使う** のが TypeScript の流儀。`null` は外部から値が来る場合（API, DB など）に限定するのが一般的。

「値がない」にも種類があります。`0` や空文字 `""` は「値はある（中身が 0 や空）」、`null` は「わざと空にしてある」、`undefined` は「まだ何も入れていない」です。箱にたとえると次のようになります。



図2-8: 「0」「null」「undefined」の違い。どれも「何もない」ように見えるが、意味が違う

▼ このコードがやること（先に日本語で）: 「値が無い」を表す `null / undefined` を安全に扱う方法を学びます。`string | null` （文字列または空）のように型で「無いかも」と明示し、使う前に `if (user !== null)` で確認したり、`?.` （オプショナルチェイニング）や `??` （無いときの代替値）で安全に触ったりします。クラッシュを防ぐ大事なテクニックです。

```ts
// --- 正しい例 ---

// strictNullChecks が有効な場合（推奨）
// 明示的に null を許可
// string | null は「string または null」の意。| はユニオン型の区切り。
let nullableString: string | null = null;
// number または undefined を許可する変数。
let optionalValue: number | undefined = undefined;

// null チェック
// 戻り値が「string または null」の関数。
function findUser(id: number): string | null {
  // === は厳密等価。id が 1 のとき。
  if (id === 1) {
    return "田中太郎";
  }
  // ユーザーが見つからない場合
  // 1 以外は明示的に null を返す。
  return null;
}

// user の型は string | null と推論される。
const user = findUser(999);
// null でないことを確認する型ガード。
if (user !== null) {
  // null チェック後なので安全
  // このブロック内では user は string と絞り込まれる。
  console.log(user.toUpperCase());
}

// オプショナルチェイニング（?. ）
// user が null なら undefined を返す
// ?. は「左が null/undefined ならその場で undefined を返し、そうでなければ .length を読む」演算子。.length のクラッシュ回避用。
const length = user?.length;

// null 合体演算子（??）
// user が null/undefined なら "ゲスト"
// ?? は「左が null か undefined なら右の値を使う」演算子。|| と似ているが、|| は 0 や "" も false 扱いするのに対し ?? は null/undefined だけを判定する。
const displayName = user ?? "ゲスト";
```

```ts
// --- エラーになる例（strictNullChecks 有効時）---
// string 型に null は入らない（strictNullChecks: true の場合）。
let name: string = null;
// エラー: Type 'null' is not assignable to type 'string'

// number 型に undefined も入らない。
let age: number = undefined;
// エラー: Type 'undefined' is not assignable to type 'number'

// null チェックなしでのメソッド呼び出し
function findItem(id: number): string | null {
  // 三項演算子 ?:。 「条件 ? trueのとき : falseのとき」の形。
  return id > 0 ? "アイテム" : null;
}

// item の型は string | null。
const item = findItem(-1);
// チェックなしで .toUpperCase() を呼ぶとエラー。
console.log(item.toUpperCase());
// エラー: 'item' is possibly 'null'
// → item が null の可能性があるので、チェックなしでは使えない
```

基本型のまとめ表

型	説明	例	使用場面
`string`	文字列	`"hello"`, `'world'`	テキストデータ全般
`number`	数値	`42`, `3.14`	数値計算、ID
`boolean`	真偽値	`true`, `false`	フラグ、条件
`string[]`	文字列配列	`["a", "b"]`	リストデータ
`[string, number]`	タプル	`["太郎", 25]`	固定構造のペア
`object`	オブジェクト	`{ key: "value" }`	構造化データ
`any`	なんでも	すべて	**使用非推奨**
`unknown`	不明な型	すべて	安全な any の代替
`never`	発生しない	なし	網羅性チェック
`void`	返り値なし	`undefined`	関数の戻り値
`null`	空	`null`	意図的な空値
`undefined`	未定義	`undefined`	未初期化の値

3. 型注釈と型推論

3.1 明示的な型注釈

型注釈（Type Annotation：タイプアノテーション。変数や関数の引数・戻り値に「これは何の型か」を明示的に書くこと）とは、変数や関数のパラメータ・戻り値に明示的に型を記述することです。コロン（ `:` ）の後に型を書きます。

> **▼ このコードがやること（先に日本語で）**：変数・関数の引数・戻り値などに「これは何の型か」を**自分で明示的に書く**（型注釈）やり方を一通り見ます。共通ルールは「**名前のうしろに `:` を付けて型を書く**」こと——`bookTitle: string` 、`(price: number): number` のようにです。形に慣れるのが目的です。

```
// 変数の型注釈
// 変数名 : 型 = 値 の順。: string が型注釈。
const bookTitle: string = "TypeScript入門";
// : number で「数値型」を宣言。
const pageCount: number = 350;
// : boolean で「真偽値型」を宣言。
const isPublished: boolean = true;

// 関数のパラメータと戻り値の型注釈
// (引数1: 型, 引数2: 型): 戻り値の型 の形。
function calculateTotal(price: number, quantity: number): number {
  // price と quantity は両方 number なので、* の結果も number。
  return price * quantity;
}

// アロー関数の型注釈
// アロー関数版。( ): 戻り値の型 => 式 の形。
const add = (a: number, b: number): number => a + b;

// オブジェクトの型注釈
// { キー: 型; ... } の形でオブジェクト型を直接書ける。
const book: { title: string; author: string; pages: number } = {
  title: "TypeScript入門",
  author: "山田太郎",
  pages: 350,
};

// 配列の型注釈
// string[] = 「string の配列」。
const tags: string[] = ["プログラミング", "TypeScript", "入門"];
```

> **つまずきポイント: 分割代入の中に型を書いてしまう** — 引数を `{ title, price }` のように分割代入で受け取るとき、つい `{ title: string, price: number }` と書きたくなります。ところが**分割代入の `{ }` の中の `:` は「型」ではなく「別名（名前の付け替え）」**という意味なので、まったく違う動きになります。第3章の React で Props（部品に渡す値）を受け取るときに、本当によく起きる間違いです。

NG 例（`destructure-ng.ts`）

```ts
// 本の情報を受け取って1行の紹介文を作る関数（のつもり）
function describeBook({ title: string, price: number }) {
  return `${title} (${price}円) `;
}

console.log(describeBook({ title: "TypeScript入門", price: 2980 }));
```

▼ 型チェックの結果（`npx tsc --noEmit --strict --target esnext destructure-ng.ts`）

```
destructure-ng.ts:2:32 - error TS7031: Binding element 'string' implicitly has an 'any'
type.

2 function describeBook({ title: string, price: number }) {
                                 ~~~~~~

destructure-ng.ts:2:47 - error TS7031: Binding element 'number' implicitly has an 'any'
type.

2 function describeBook({ title: string, price: number }) {
                                                ~~~~~~

destructure-ng.ts:3:13 - error TS2304: Cannot find name 'title'.

3   return `${title} (${price}円) `;
              ~~~~~

destructure-ng.ts:3:22 - error TS2304: Cannot find name 'price'.

3   return `${title} (${price}円) `;
                       ~~~~~


Found 4 errors in the same file, starting at: destructure-ng.ts:2
```

▼ 実行結果（`node destructure-ng.ts`。node 内部の行は省略）

```
C:\ts-practice\destructure-ng.ts:3
  return `${title}（${price}円）`;
  ^

ReferenceError: title is not defined

Node.js v25.9.0
```

- `{ title: string }` は「`title` を取り出して、**`string` という名前の変数**に入れる」という意味になります。だから `Binding element 'string' ...`（`string` という変数の型が分からない）や、`Cannot find name 'title'`（`title` という変数は無い）と言われるのです。

OK 例（`destructure-ok.ts`）：型は分割代入の `{ }` の**外側**に、`:` を付けてまとめて書く

```ts
// 書き方1：分割代入の「外側」に、まとめて型を書く
function describeBook({ title, price }: { title: string; price: number }) {
  return `${title}（${price}円）`;
}

// 書き方2：型に名前を付けてから使う（React の Props でよく見る形）
interface BookSummary {
  title: string;
  price: number;
}

function describeBook2({ title, price }: BookSummary) {
  return `${title}（${price}円）`;
}

console.log(describeBook({ title: "TypeScript入門", price: 2980 }));
console.log(describeBook2({ title: "React実践ガイド", price: 3300 }));
```

▼ **実行結果**（`tsc` はエラーなし）

```
TypeScript入門（2980円）
React実践ガイド（3300円）
```

- 読み方は「`{ title, price }` を取り出す。取り出す元のオブジェクトの型は `{ title: string; price: number }`」です。**値の形と型**を `:` で区切って並べている、と覚えましょう。

3.2 型推論の仕組み

TypeScript は非常に賢い **型推論**（Type Inference：タイプインファレンス。初期値や使われ方から TypeScript が自動的に型を判断する機能）エンジンを持っています。多くの場合、型注釈を書かなくても TypeScript が自動的に型を判断

してくれます。

> **▼ このコードがやること（先に日本語で）**：型注釈を書かなくても、TypeScript が初期値から**自動で型を判断してくれる**（型推論）様子を見ます。`const message = "..."` と書けば勝手に `string` と分かる、という具合です。「明らかな型はわざわざ書かなくてよい」のがポイント。詳しい推論結果はコメントに書いてあります。

```ts
// --- TypeScript が自動で型を推論する例 ---

// 変数の初期化から推論
// string と推論
// 右辺が文字列リテラルなので、型注釈なしでも string と分かる。
const message = "こんにちは";
// number と推論
// 数値リテラル → number。
const count = 42;
// boolean と推論
// true/false → boolean。
const isValid = true;
// number[] と推論
// 配列の中身が全部 number なら number[] と推論。
const items = [1, 2, 3];

// 関数の戻り値も推論される
// 戻り値の型注釈を省略しても...
function multiply(a: number, b: number) {
  // 戻り値は number と推論される
  // a * b が number なので戻り値は number と分かる。
  return a * b;
}

// オブジェクトの構造も推論される
const user = {
  // name: string
  // 右辺の値から各プロパティの型が決まる。
  name: "田中",
  // age: number
  age: 25,
  // isActive: boolean
  isActive: true,
};
// user の型は { name: string; age: number; isActive: boolean } と推論

// 配列のメソッドから推論
// map の引数 n は配列の要素型から推論されて number。返す n * 2 も number。結果は number[]。
const doubled = [1, 2, 3].map((n) => n * 2);
// doubled は number[] と推論

// 条件式から推論
// 三項演算子 ?:。両辺が string なので結果は string と推論される。
const status = count > 10 ? "many" : "few";
// status は string と推論
```

```ts
// --- 型推論の注意点 ---

// let で宣言すると広い型に推論される
// string と推論（"red" リテラル型ではない）
// let は後から再代入されうるので「広めの型」(=string) に推論される。
let color = "red";
// OK（string なので他の文字列も代入可能）
color = "blue";

// const で宣言するとリテラル型に推論される
// "north" と推論（リテラル型）
// const は変わらないので「ピッタリの型」(="north") に推論される。
const direction = "north";
// エラー: const なので再代入不可
// direction = "south";

// 空配列は any[] に推論される（要注意）
// any[] と推論
// 中身がないので TypeScript は要素型を決められず any[] になる（ゆるい）。
const emptyList = [];
// → 型注釈をつけるべき
// 明示的に型を指定
// 必ず型注釈を書くのが安全。
const emptyStrings: string[] = [];
```

推論された型は、コメントを読まなくても **VS Code で確かめられます**。変数名にマウスを乗せる（ホバーする）だけで、TypeScript がその変数をどんな型だと判断したかが表示されます（図2-9）。

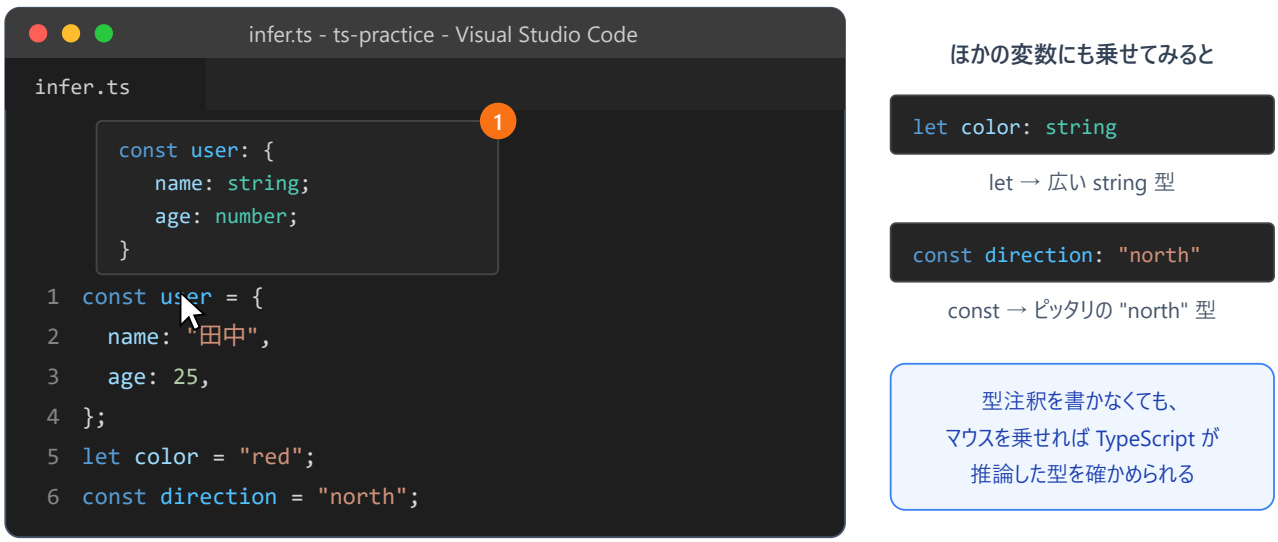


図2-9: VS Code のホバー表示（画面イメージ）。変数名にマウスを乗せると、推論された型が見える

ヒント:「この変数、何型になってるんだろう？」と迷ったら、まずホバーして確かめるのが一番の近道です。関数名に乗せれば引数と戻り値の型が、エラーの波線に乗せればエラーの理由が表示されます。

3.3 いつ型注釈を書くべきか



型注釈を書くべき場面

▼ このコードがやること（先に日本語で）:「型注釈を**自分で書いたほうがよい**5つの場面」を実例で示します。特に大事なのは①**関数の引数**——ここは TypeScript が推論できないので必ず型を書く、というルールです。空配列の初期化や複数の型を受け取る変数なども、書いておくと安全になります。

```ts
// 1. 関数のパラメータ（必須！推論できない）
// 引数は値が無い段階で書くので、TypeScript は推論できない。書かないと暗黙の any になる。
function greet(name: string, age: number): string {
  // テンプレートリテラルで挨拶文を生成。
  return `${name}さんは${age}歳です`;
}

// 2. 空配列を初期化する場合
// 型注釈がないと any[] になる
// 後で push する型を限定したいなら必須。
const users: string[] = [];

// 3. 関数の戻り値を明確にしたい場合（公開APIなど）
// Promise<T> は「将来 T を返す」を表す型。fetch は Promise を返すので戻り値も Promise になる。
function fetchUser(id: number): Promise<User> {
  // 戻り値の型が明確になり、実装ミスを防げる
  // fetch はサーバーに HTTP リクエストを送る関数。.then で結果を加工する。
  return fetch(`/api/users/${id}`).then((res) => res.json());
}

// 4. 複数の型を受け入れる場合
// | はユニオン型の区切り。「string か number のどちらか」。
let result: string | number;
// string なので OK。
result = "成功";
// number なので OK。
result = 404;

// 5. オブジェクトの構造を明確にしたい場合
// interface でオブジェクトの型を定義。
interface Config {
  apiUrl: string;
  timeout: number;
  retries: number;
}

// Config 型として宣言。3つのプロパティが必須。
const config: Config = {
  apiUrl: "https://api.example.com",
  timeout: 5000,
  retries: 3,
};
```

型推論に任せてよい場面

> **▼ このコードがやること（先に日本語で）**：逆に「型注釈を**書かなくてよい**（推論に任せてよい）場面」を示します。初期値を見れば型が一目瞭然なときは、わざわざ書くとかえって冗長になります。「明らかなら省く、曖昧なら書く」というバランス感覚をつかむのが目的です。

```
// 1. 初期値から型が明らかな場合
// 明らかに string
// : string と書かなくても推論される。冗長さを避けるために省くのも良い設計。
const name = "田中太郎";
// 明らかに number
const age = 30;
// 明らかに boolean
const isActive = true;

// 2. 配列リテラルで初期化する場合
// 明らかに string[]
// 中身があれば推論できる。
const fruits = ["りんご", "みかん"];

// 3. 関数の戻り値が単純な場合
// 戻り値の型を省略。
function add(a: number, b: number) {
  // 明らかに number を返す
  // number + number → number と推論。
  return a + b;
}

// 4. 変数の型が変わらない場合
// 明らかに number
// 計算結果から推論。
const total = price * quantity;
```

---

4. インターフェースと型エイリアス

4.1 interface の定義方法

`interface` はオブジェクトの「形（シェイプ）」を定義する方法です。

▼ **このコードがやること（先に日本語で）**：オブジェクトの「形（どんなプロパティを持つか）」を定義する `interface`（設計図）の基本を学びます。`User` という型を作っておけば、その形に合わないオブジェクトは即エラーになります。あわせて、省略可能な `?`、変更禁止の `readonly` も扱います。書籍管理アプリでもこの形で `Book` 型を作っていきます。

```ts
// ============================================================================
// interface の基本 - 「オブジェクトの設計図」を作る
// ============================================================================
// interface は「このオブジェクトは こういう形をしてないとダメだよ」という
// ルールを宣言する仕組み。クラスの設計図のようなものを軽量に書ける。
// 大文字始まりにするのが慣習（クラス名と同じ）。

// (1) User という名前の interface を定義
//     「User 型のオブジェクトには id, name, email, age の4つが必須」と宣言
interface User {
  // 数値型のID（必須）
  id: number;
  // 文字列型の名前（必須）
  name: string;
  // 文字列型のメールアドレス（必須）
  email: string;
  // 数値型の年齢（必須）
  age: number;
}

// (2) User 型の変数を作る
//     プロパティが1つでも欠けるとエラー、余計なプロパティもエラー（厳格チェック）
const user: User = {
  id: 1,
  name: "田中太郎",
  email: "tanaka@example.com",
  age: 30,
};

console.log(user);
// ▼ 実行結果
// { id: 1, name: '田中太郎', email: 'tanaka@example.com', age: 30 }


// ============================================================================
// オプショナルプロパティ「?」を使うと、あってもなくても良くなる
// ============================================================================

interface Product {
  // 必須
  id: number;
  // 必須
  name: string;
  // 必須
  price: number;
  // ←「?」付きなので省略可能。値が無いときは undefined
  description?: string;
}
```

```ts
// description を省略しても OK
const pen: Product = {
  id: 1,
  name: "ボールペン",
  price: 150,
  // description は書いていないが、? なのでエラーにならない
};

console.log(pen.description);
// ▼ 実行結果
// undefined


// ============================================================================
// readonly: 「あとから変えられないプロパティ」
// ============================================================================

interface Config {
  // readonly = 後で代入禁止
  readonly apiUrl: string;
  readonly timeout: number;
}

const config: Config = {
  apiUrl: "https://api.example.com",
  timeout: 5000,
};

// ← これを書くとエラー
// config.apiUrl = "https://other.com";
// ▼ エラー
// Cannot assign to 'apiUrl' because it is a read-only property.
//   ('apiUrl' は読み取り専用プロパティのため代入できません)
```

▼ コードを1つずつ分解して解説

`interface` を初めて見ると「クラス？型？」と混乱しがちです。順番に、**1つずつ**ていねいに見ていきましょう。

解説1: `interface` で「オブジェクトの設計図」を作る

```
interface User {
  // 数値型のID（必須）
  id: number;
  // 文字列型の名前（必須）
  name: string;
  // 文字列型のメールアドレス（必須）
  email: string;
  // 数値型の年齢（必須）
  age: number;
}
```

- `interface User { ... }` は「**`User` という名前の型（オブジェクトの設計図）を作る**」という宣言です。`interface`（インターフェース）は「このオブジェクトはこういう形をしていてね」というルールを定義する道具です。
- `{ }` の中に「**キー名: 型**」を1行ずつ並べます。ここでは `id`・`name`・`email`・`age` の4項目を持つと宣言しています。
- 各行のうしろには `;`（セミコロン）を付けますが、改行で区切るだけでも動きます。
- 型名は **大文字始まり**（`User`）にするのが慣習です。

> **用語: interface（インターフェース）** … オブジェクトの「形（どんな項目を持つか）」を定義するための、TypeScriptの道具のひとつ。`type` と並んでよく使われます。

解説2: 定義した型を使ってオブジェクトを作る

```
const user: User = {
  id: 1,
  name: "田中太郎",
  email: "tanaka@example.com",
  age: 30,
};
```

- `: User` という型注釈で、「この `user` は `User` 型の形をしていますよ」と宣言しています。
- これにより、`User` で決めた4項目を**すべて正しい型で持っていないとエラー**になります。
- 1つでも項目が欠けるとエラー（例: `age` を書き忘れる）。
- 型が違ってもエラー（例: `age: "30"` と文字列にする）。
- 余計な項目を足してもエラー（定義にないキーは許されません）。
- このように、一度 `interface` を作っておけば、同じ形のオブジェクトを**安全に何個でも**作れるのが利点です。

解説3: オプショナルプロパティ（`?`）

```
interface Product {
  // 必須
  id: number;
  // 必須
  name: string;
  // 必須
  price: number;
  // ← 「?」付きなので省略可能。値が無いときは undefined
  description?: string;
}
```

- キー名のうしろの `?`（`description?`）は「**この項目は省略してもよい**」という印（オプショナル）です。
- そのため `description` を書かずにオブジェクトを作ってもエラーになりません。省略した場合の値は `undefined`（値が無い状態）になります。
- 「必ず持っていてほしい項目には `?` なし、あってもなくてもよい項目には `?` あり」と使い分けます。

解説4: 読み取り専用プロパティ（`readonly`）

```
interface Config {
  // readonly = 後で代入禁止
  readonly apiUrl: string;
  readonly timeout: number;
}
```

- キー名の前に付く `readonly` は「**一度値を入れたら、あとから変更できない**」という印です。
- 例えば `config.apiUrl = "https://other.com"` と書き換えようとすると、TypeScript が「読み取り専用なので代入できません」とエラーで止めてくれます。
- API の URL や設定値のように「途中で書き換わってほしくない項目」に付けると、うっかり変更によるバグを防げます。

> **▼ このコードがやること（先に日本語で）**：既存の `interface` に項目を**追加した新しい型**を作る「継承（`extends`）」を学びます。`Dog extends Animal` と書けば「Animal の項目を全部受け継いだうえで、犬固有の項目を足す」という意味になり、同じ定義を何度も書かずに済みます。複数の型をまとめて継承することもできます。

```ts
// ============================================================================
// interface の継承（extends）— 既存の型に項目を追加した新しい型を作る
// ============================================================================

// (1) ベースとなる Animal 型
interface Animal {
  // 動物の名前
  name: string;
  // 年齢
  age: number;
}

// (2) Dog は Animal を「extends」している
//     → Animal の name, age を全て持ったうえで、breed と isVaccinated が追加される。
//     継承元の項目は書かなくてもOK（自動で引き継がれる）。
interface Dog extends Animal {
  // 犬種（Dog で追加）
  breed: string;
  // ワクチン接種済みか（Dog で追加）
  isVaccinated: boolean;
}

// (3) Dog 型のオブジェクトには合計4つのプロパティが必須
const myDog: Dog = {
  // Animal 由来
  name: "ポチ",
  // Animal 由来
  age: 3,
  // Dog 固有
  breed: "柴犬",
  // Dog 固有
  isVaccinated: true,
};
console.log(myDog.name, myDog.breed);
// ▼ 実行結果
// ポチ 柴犬


// ============================================================================
// 複数の interface を同時に継承（カンマ区切り）
// ============================================================================
// 「ID を持つ」「タイムスタンプを持つ」のような共通の特徴を細かく分けて、
// 必要な組み合わせを extends で混ぜるのが現代的な設計。

// (1) 「ID を持つ」だけを表現する interface
interface HasId {
  id: number;
}
```

```ts
// (2) 「作成日時・更新日時を持つ」だけを表現する interface
interface HasTimestamp {
  // Date は JavaScript 標準の日時オブジェクト
  createdAt: Date;
  updatedAt: Date;
}

// (3) BlogPost は HasId と HasTimestamp の両方を継承し、独自フィールドを追加
interface BlogPost extends HasId, HasTimestamp {
  // タイトル
  title: string;
  // 本文
  content: string;
  // 著者
  author: string;
}

// (4) BlogPost 型のオブジェクトには6つのプロパティが必須
//     new Date("2025-01-01") は「2025年1月1日 0:00 UTC」を表す Date オブジェクト
const post: BlogPost = {
  id: 1,
  createdAt: new Date("2025-01-01"),
  updatedAt: new Date("2025-06-15"),
  title: "TypeScript入門",
  content: "TypeScriptの基礎を学びましょう",
  author: "田中太郎",
};

console.log(post.title, "投稿日:", post.createdAt.toISOString());
// ▼ 実行結果
// TypeScript入門 投稿日: 2025-01-01T00:00:00.000Z
```

▼ このコードがやること（先に日本語で）： `interface` には、データだけでなく「**メソッド（関数）**を持つ」という形も定義できます。ここでは四則演算の関数4つを持つ `Calculator` 型を作り、それに合わせて実装します。「この型のオブジェクトは、こういう関数を必ず持っていてね」と約束させられる、という点がポイントです。

```ts
// interface でメソッドを定義
// 「メソッド（関数プロパティ）4つを持つオブジェクト」の型定義。
interface Calculator {
  // add メソッドの型。引数2つ、戻り値 number。
  add(a: number, b: number): number;
  // 同様に subtract（引き算）。
  subtract(a: number, b: number): number;
  // multiply（掛け算）。
  multiply(a: number, b: number): number;
  // divide（割り算）。
  divide(a: number, b: number): number;
}

// Calculator 型として4つのメソッドを実装。
const calc: Calculator = {
  // 各メソッドはアロー関数で実装。引数の型は interface から推論される。
  add: (a, b) => a + b,
  subtract: (a, b) => a - b,
  multiply: (a, b) => a * b,
  // 割り算は 0 除算チェックがあるので { } で複数行に。
  divide: (a, b) => {
    // b が 0 のとき例外を投げる。throw は実行を中断する文。
    if (b === 0) throw new Error("0で割ることはできません");
    // 正常時は割った結果を返す。
    return a / b;
  },
};
```

4.2 type の定義方法

`type`（型エイリアス）はあらゆる型に名前をつける方法です。

> **▼ このコードがやること（先に日本語で）：** `type`（型エイリアス）を使って、あらゆる型に**自分で名前を付ける**方法を学びます。`interface` がオブジェクトの形専用なのに対し、`type` は文字列の別名・ユニオン型（「AまたはB」）・タプル・関数型など、何にでも名前を付けられるのが強みです。ここでは代表的なパターンをまとめて見ます。

```ts
// 基本的な type エイリアス
// type で型に別名を付ける。UserName は単に string の別名。
type UserName = string;
// Age は number の別名。意味のある名前を付けるとコードが読みやすい。
type Age = number;
// IsActive = boolean。
type IsActive = boolean;

// オブジェクト型
// type でオブジェクト型を定義（interface でもほぼ同じ）。
type User = {
  id: number;
  name: string;
  email: string;
  age: number;
};

// User 型として変数を作る。
const user: User = {
  id: 1,
  name: "田中太郎",
  email: "tanaka@example.com",
  age: 30,
};

// ユニオン型（interface ではできない）
// 文字列リテラルのユニオン。3つの値のいずれかに限定。
type Status = "active" | "inactive" | "pending";
// string か number のどちらでも入る型。
type Id = string | number;

// "active" は許可された値なので OK。
const userStatus: Status = "active";
// string が許可されているので OK。
const userId: Id = "user_123";

// タプル型
// 2要素タプル。[緯度, 経度] のように使う。
type Coordinate = [number, number];
// [名前, 年齢] のタプル。
type NameAge = [string, number];

// 東京の座標
// Coordinate 型として代入。
const point: Coordinate = [35.6762, 139.6503];

// 関数型
// 「(a: number, b: number) => number」は関数型の表記。「引数2つ受け取って number を返す関数」を意味する。
```

```ts
type MathOperation = (a: number, b: number) => number;

// MathOperation 型として add を実装。引数の型は型から推論される。
const add: MathOperation = (a, b) => a + b;
// 同じ型から別の関数を作れる（型の再利用）。
const subtract: MathOperation = (a, b) => a - b;
```

▼ コードを1つずつ分解して解説

`type` は「オブジェクトの形」以外にも色々な型に名前を付けられるのが強みです。代表的なパターンを、**1つずつ**ていねいに見ていきましょう。

解説1: 基本型に別名を付ける

```ts
type UserName = string;
type Age = number;
type IsActive = boolean;
```

- `type 名前 = 型;` の形で、**既存の型に自分で好きな名前（別名）を付けられます。**
- `type UserName = string` は「`UserName` は `string` の別名」という意味で、中身はただの文字列です。ただ「`string`」と書くより `UserName` と書くほうが「これは利用者名なんだな」と意図が伝わります。
- このように、意味のある名前を付けるとコードが読みやすくなる、というのが `type` の使いどころの1つです。

> **用語: 型エイリアス（type alias）** … 既存の型に付ける「別名（あだ名）」のこと。`type` キーワードで作ります。

解説2: オブジェクト型に名前を付ける

```ts
type User = {
  id: number;
  name: string;
  email: string;
  age: number;
};

const user: User = {
  id: 1,
  name: "田中太郎",
  email: "tanaka@example.com",
  age: 30,
};
```

- `type` でも、`interface` と同じように**オブジェクトの形**を定義できます。`{ }` の中に「キー名: 型」を並べる書き方は `interface` とほぼ同じです。
- 違いは書き方だけで、`type User = { ... }` の `=`（イコール）が付く点が `interface User { ... }` との見た目の差です。
- 一度 `User` 型を作れば、`const user: User = { ... }` のように使い回せます。やっていることは `interface` の例とまったく同じです。

解説3: ユニオン型に名前を付ける（`type` ならでは）

```
type Status = "active" | "inactive" | "pending";
type Id = string | number;

const userStatus: Status = "active";
const userId: Id = "user_123";
```

- `|`（パイプ）で型を並べると「**A または B**」という**ユニオン型**になります。これは `interface` ではできず、`type` ならではの機能です。
- `type Status = "active" | "inactive" | "pending"` は「`Status` 型には、この3つの文字列のどれかしか入らない」という意味です。`"actve"`（`active` の打ち間違い）のような値を入れると即エラーになります。
- `type Id = string | number` は「`Id` 型には文字列か数値のどちらかが入る」という意味です。

> **用語: ユニオン型** … `|` で複数の型をつないだ「どれか1つ」を表す型。「`string` または `number`」のように、複数の候補を許したいときに使います。

解説4: タプル型・関数型に名前を付ける

```
type Coordinate = [number, number];
type NameAge = [string, number];

// 東京の座標
const point: Coordinate = [35.6762, 139.6503];

type MathOperation = (a: number, b: number) => number;

const add: MathOperation = (a, b) => a + b;
const subtract: MathOperation = (a, b) => a - b;
```

- `[number, number]` は**タプル型**（位置ごとに型と個数が固定された配列）です。`Coordinate` という名前を付けて「座標（緯度・経度）」を表しています。

- `(a: number, b: number) => number` は**関数型**で、「数値を2つ受け取って数値を返す関数」という形を表します。これに `MathOperation` という名前を付けています。
- 一度 `MathOperation` 型を作れば、`add` と `subtract` のように**同じ型から複数の関数を作れる**（型を使い回せる）のが利点です。引数 `a`・`b` の型は型注釈から自動で推論されるので、関数側に書く必要はありません。

> **用語: 関数型** … 「どんな引数を受け取り、何を返す関数か」を表す型。`(引数: 型) => 戻り値の型` という形で書きます。

> **▼ このコードがやること（先に日本語で）：** `type` ならではの、少し進んだ型の組み合わせ方を見ます。`&`（交差型）で「**両方の性質を合わせ持つ**型」を作ったり、条件によって型を変える「条件付き型」、既存の型を一括変換する「マップ型」を扱います。最初は難しく感じる部分なので、まずは `&` で型を合体できる、という点だけ押さえればOKです。

```ts
// 交差型（Intersection Type）
// 「name を持つ」だけを表す型。
type HasName = {
  name: string;
};

// 「age を持つ」だけを表す型。
type HasAge = {
  age: number;
};

// & は交差型（intersection）の記号。両方の性質を合わせ持つ型。「name と age を両方持つ」になる。
type Person = HasName & HasAge;
// Person は { name: string; age: number; } と同じ

const person: Person = {
  name: "田中",
  age: 30,
};

// 条件付き型（Conditional Type）
// T が string 型に含まれるなら "yes"、そうでないなら "no" を返す型。T extends X は「T が X 型の部分型か」の判定。
type IsString<T> = T extends string ? "yes" : "no";

// "yes"
// T = string → "yes"。
type A = IsString<string>;
// "no"
// T = number → "no"。
type B = IsString<number>;

// マップ型（Mapped Type）
// T の全プロパティを readonly に変換した型を作る。
type Readonly<T> = {
  // [P in keyof T] は「T のキーをひとつずつ P として取り出す」記法。T[P] はそのキーに対応する値の型。前に readonly を付けるとすべて読み取り専用に。
  readonly [P in keyof T]: T[P];
};

// User の全プロパティが readonly になった型ができる。
type ReadonlyUser = Readonly<User>;
// すべてのプロパティが readonly になる
```

4.3 interface vs type の使い分け

機能	interface	type
オブジェクトの形を定義	OK	OK
継承（extends）	OK	交差型（ `&` ）で代替
実装（implements）	OK	OK
宣言のマージ（Declaration Merging）	OK	不可
ユニオン型	不可	OK
タプル型	不可	OK
プリミティブ型のエイリアス	不可	OK
マップ型	不可	OK
条件付き型	不可	OK
計算されたプロパティ	不可	OK

▼ このコードがやること（先に日本語で）： `interface` だけが持つ「**宣言のマージ**」という機能を見ます。同じ名前で `interface` を2回書くと、TypeScript が自動で1つに合体してくれます（ライブラリの型を後から拡張するときに使う）。一方 `type` で同名を再定義するとエラーになる、という違いも確認します。

```
// --- interface でしかできないこと: 宣言のマージ ---
// 同じ名前の interface を複数書くと、TypeScript が自動でマージしてくれる。
interface Window {
  title: string;
}

interface Window {
  appVersion: number;
}

// 2つの宣言がマージされる
// 結果は両方のプロパティを持つ型。ライブラリの型を後から拡張する用途で使う。
// Window = { title: string; appVersion: number; }

// type では同じ名前で再定義するとエラーになる
// 1つ目の定義。
type Animal = { name: string };
// エラー: Duplicate identifier 'Animal'
// 同じ名前で再定義しようとするとエラー。
type Animal = { age: number };
```

> **▼ このコードがやること（先に日本語で）**：逆に `type` だけができることを並べて見ます。ユニオン型（「AまたはB」）、プリミティブ型への別名付け、タプル型、関数型、条件付き型――これらは `interface` では書けません。「オブジェクトの形以外も名前を付けたいときは `type`」と覚えるための一覧です。

```
// --- type でしかできないこと ---

// ユニオン型
// | で複数のリテラルを並べて型を作るのは type ならでは。
type Result = "success" | "error" | "loading";

// プリミティブのエイリアス
// プリミティブ型に別名を付けるのも type のみ。
type ID = string | number;

// タプル型
// タプル型のエイリアスも type で書く。
type Point = [number, number];

// 関数型
// 関数型のエイリアスも type で書く。
type Formatter = (input: string) => string;

// 条件付き型
// 条件付き型も type 限定。T が null か undefined なら never、そうでなければ T 自体。
type NonNullable<T> = T extends null | undefined ? never : T;
```

使い分けの指針:

- **interface を使う場面**: オブジェクトの形状を定義する場合（特にクラスが実装する場合や、ライブラリの型を拡張したい場合）
- **type を使う場面**: ユニオン型、タプル型、関数型、プリミティブ型エイリアスなど、interface では表現できない型を定義する場合

> **実務でのコツ**: チーム内で統一するのが最も重要です。迷ったら **interface をデフォルトで使い、interface で表現できない場合のみ type を使う** という方針が一般的です。

4.4 書籍管理アプリで使う型の定義例

実際のアプリケーション開発を想定して、書籍管理アプリに必要な型を定義してみましょう。**初心者でもわかるように、ほぼ全行に解説コメントを付けています。**

▼ **このコードがやること（先に日本語で）**：これまで学んだ型の道具（ユニオン型・`interface`・継承・ジェネリクスなど）を総動員して、これから作る**書籍管理アプリ全体で使う「データの形」**をまとめて定義します。ここで型を一度作っておくと、アプリのあらゆる場所で同じ型を再利用でき、間違いを TypeScript が指摘してくれます。長いですが、1つずつコメントを追えば大丈夫です。

```ts
// ============================================================================
// 書籍管理アプリの型定義（詳細コメント版）
// ============================================================================
// このファイルはアプリ全体で使う「データの形」を集めた辞書のようなもの。
// ここで型を1度作っておくと、関数の引数や useState の中身など
// あらゆる場所で同じ型を再利用でき、間違いを TypeScript が指摘してくれる。
// ============================================================================


// ----------------------------------------------------------------------------
// (1) 「読書ステータス」の型
// ----------------------------------------------------------------------------
// 「ユニオン型」（| で区切って列挙）を使い、値を3つの文字列のどれかに限定する。
// こうすると status = "yomu" のような typo が即座にエラーになる。
type ReadingStatus = "want_to_read" | "reading" | "completed";
//                        ↑読みたい      ↑読書中     ↑読了


// ----------------------------------------------------------------------------
// (2) 「評価」の型 – 1〜5 の数値リテラルだけを許す
// ----------------------------------------------------------------------------
// number だけだと 0 や 100 も入ってしまう。1〜5 のいずれかに限定したいので、
// 数値リテラル型のユニオンで宣言する。
type Rating = 1 | 2 | 3 | 4 | 5;


// ----------------------------------------------------------------------------
// (3) カテゴリの型 – 列挙したいときの定石
// ----------------------------------------------------------------------------
// 各値の右側のコメントは「人間用のメモ」。実行時には消える。
type BookCategory =
  // 小説
  | "fiction"
  // ノンフィクション
  | "non-fiction"
  // 技術書
  | "technology"
  // ビジネス
  | "business"
  // 自己啓発
  | "self-help"
  // その他
  | "other";


// ----------------------------------------------------------------------------
// (4) 著者情報の型 – interface で「オブジェクトの形」を定義
// ----------------------------------------------------------------------------
// `?` を付けたプロパティは「あってもなくてもいい」（オプショナル）
```

```ts
interface Author {
  // 著者のユニークID（必須）
  id: string;
  // 著者名（必須）
  name: string;
  // 国籍（省略可能。? が付く）
  nationality?: string;
}



// ---------------------------------------------------------------------------
// (5) 書籍の基本情報
// ---------------------------------------------------------------------------
// `author` プロパティは前で定義した Author 型を再利用している。
// このように型を入れ子にできるのが TypeScript の便利なところ。
interface Book {
  // 書籍のユニークID
  id: string;
  // タイトル
  title: string;
  // 著者（Author 型のオブジェクト）
  author: Author;
  // ISBN番号（書籍の世界共通ID）
  isbn: string;
  // カテゴリ（前述の6種類のどれか）
  category: BookCategory;
  // 総ページ数
  pages: number;
  // 出版日。ISO 8601 形式 "2025-01-15"
  publishedDate: string;
  // 表紙画像URL（省略可能）
  coverImageUrl?: string;
  // 説明文（省略可能）
  description?: string;
}



// ---------------------------------------------------------------------------
// (6) 読書記録の型 - Book を「継承」して項目を追加
// ---------------------------------------------------------------------------
// `extends Book` は「Book のすべての項目をそのまま受け継いだうえで、追加項目を持つ」
// という意味。継承を使うと「Book にこれだけ足したやつ」という意図が明確になる。
interface ReadingRecord extends Book {
  // 読書ステータス（必須）
  status: ReadingStatus;
  // 評価（読了後にのみ設定するためオプショナル）
  rating?: Rating;
  // 読み始めた日
  startDate?: string;
  // 読み終わった日
```

```
  endDate?: string;
  // メモ
  memo?: string;
  // 現在のページ（読書中のみ意味を持つ）
  currentPage?: number;
}


// ---------------------------------------------------------------------------
// (7) ユーティリティ型 Omit / Partial の活用
// ---------------------------------------------------------------------------
// 同じことを毎回手書きすると面倒なので、TypeScript の組み込みユーティリティを使う。

// `Omit<T, K>` = T から K プロパティを抜いた型を作る。
// 新規作成時は id がまだ存在しないので、Book から id を抜く。
type CreateBookInput = Omit<Book, "id">;
// 結果として「id 以外の全フィールドが必須」の型ができる。

// `Partial<T>` = T のすべてのプロパティを「オプショナル」にした型。
// 更新時は「変えたい項目だけ」送れば良いので、全項目を ? 付きにしたい。
type UpdateBookInput = Partial<Omit<Book, "id">>;
// 結果として「id 以外の全フィールドが省略可」の型ができる。


// ---------------------------------------------------------------------------
// (8) フィルター条件の型 — 検索・並び替えのオプション
// ---------------------------------------------------------------------------
// すべて省略可（?）にして、ユーザーが指定したものだけ使うパターン。
interface BookFilter {
  // ステータスで絞る
  status?: ReadingStatus;
  // カテゴリで絞る
  category?: BookCategory;
  // タイトルまたは著者名で検索
  searchQuery?: string;
  // 並び替え基準
  sortBy?: "title" | "publishedDate" | "rating";
  // asc=昇順（1→9, A→Z）, desc=降順（9→1, Z→A）
  sortOrder?: "asc" | "desc";
}


// ---------------------------------------------------------------------------
// (9) ジェネリクスを使った API レスポンス型
// ---------------------------------------------------------------------------
// `<T>` は「あとで決まる型のプレースホルダー」。
// 例えば `ApiResponse<Book>` と書けば T = Book になり、
// `ApiResponse<Author>` と書けば T = Author になる。
// 「成功・失敗の枠組みは同じだが、データの中身は呼び出し側で決まる」を表現するのに最適。
interface ApiResponse<T> {
```

```typescript
  // 取得したデータ本体（型は呼び出し側次第）
  data: T;
  // 成否
  success: boolean;
  // メッセージ（省略可）
  message?: string;
}


// ---------------------------------------------------------------------------
// (10) ページネーション付きレスポンス
// ---------------------------------------------------------------------------
// 一覧取得APIは「データ配列＋ページ情報」をセットで返すのが定番。
// data: T[] の T[] は「T の配列」を意味する記法。
interface PaginatedResponse<T> {
  // 1ページ分のデータ配列
  data: T[];
  // 全件数
  total: number;
  // 現在のページ番号（1始まり）
  page: number;
  // 1ページあたりの件数
  perPage: number;
  // 総ページ数
  totalPages: number;
}

// ===== 使用例 =====

// 書籍の作成
const newBook: CreateBookInput = {
  title: "TypeScript実践ガイド",
  author: {
    id: "author_001",
    name: "山田太郎",
    nationality: "日本",
  },
  isbn: "978-4-XXXX-XXXX-X",
  category: "technology",
  pages: 400,
  publishedDate: "2025-06-01",
  description: "TypeScriptの基礎から実践まで学べる一冊",
};

// 読書記録
const myReading: ReadingRecord = {
  id: "book_001",
  title: "TypeScript実践ガイド",
  author: {
    id: "author_001",
```

```
    name: "山田太郎",
  },
  isbn: "978-4-XXXX-XXXX-X",
  category: "technology",
  pages: 400,
  publishedDate: "2025-06-01",
  status: "reading",
  startDate: "2025-07-01",
  currentPage: 150,
  memo: "第5章まで読了。ジェネリクスの説明がわかりやすい。",
};

// フィルターを使った検索
const filter: BookFilter = {
  status: "reading",
  category: "technology",
  sortBy: "title",
  sortOrder: "asc",
};

// API レスポンスの型適用
const response: ApiResponse<ReadingRecord> = {
  data: myReading,
  success: true,
  message: "書籍情報を取得しました",
};

// ページネーション付きレスポンス
const listResponse: PaginatedResponse<ReadingRecord> = {
  data: [myReading],
  total: 1,
  page: 1,
  perPage: 10,
  totalPages: 1,
};
```

4.4.1 Omit と Partial で「入力用の型」を作る（動かして確認）

上の (7) で出てきた `Omit` と `Partial` は、書籍管理アプリの「新規登録」と「更新」で実際に使う考え方です。文字だけだとイメージしにくいので、まず図で「型がどう変形するか」を見てみましょう。

> **たとえ話:** `Book` 型を「記入欄が4つある申込用紙」だとすると、`Omit<Book, "id">` は「id 欄を切り取った用紙」、`Partial<...>` は「すべての欄に『空欄でも可』と書き足した用紙」です。元の用紙（ `Book` ）はそのまま残り、コピーを加工して新しい用紙を作るイメージです。

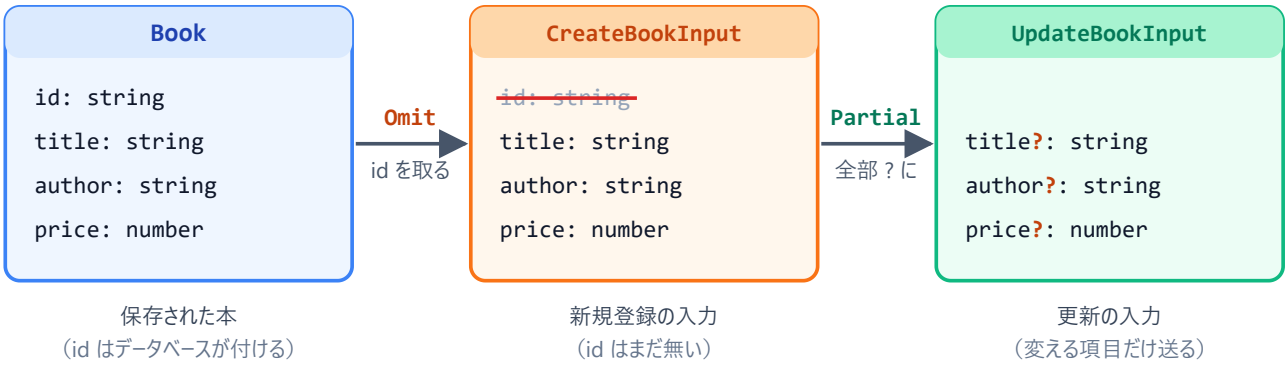


図2-10: Omit で id を取り除き、Partial で全項目を省略可能（?）にする。元の Book 型は変わらない

入力用の型を使って本を登録・更新する

▼ **このコードがやること（先に日本語で）**： `Book` 型から「新規登録の入力」と「更新の入力」の2つの型を作り、それを使って本を1冊作り、価格だけを変更します。更新のときは**変えたい項目だけ**を渡せばよいこと、元の本は壊れないことを確かめます。ファイル名は `utility.ts` です。

```ts
// アプリで保存される「本」の形（id はデータベースが付ける番号）
interface Book {
  id: string;
  title: string;
  author: string;
  price: number;
}

// 新規登録の入力: まだ id が無いので、Book から id を取り除いた形
type CreateBookInput = Omit<Book, "id">;

// 更新の入力: 変えたい項目だけ送ればよいので、全部を省略可能にした形
type UpdateBookInput = Partial<CreateBookInput>;

// 入力に id を付けて「本」を完成させる関数
function createBook(input: CreateBookInput, newId: string): Book {
  return { id: newId, ...input };
}

// 元の本に「変えたい項目」だけを上書きした、新しい本を返す関数
function updateBook(book: Book, changes: UpdateBookInput): Book {
  return { ...book, ...changes };
}

const input: CreateBookInput = {
  title: "TypeScript入門",
  author: "山田太郎",
  price: 2980,
};

const created = createBook(input, "book-001");
console.log(created);

// 価格だけを変更する（ほかの項目は書かなくてよい）
const updated = updateBook(created, { price: 1980 });
console.log(updated);

// 元の created は変わっていない
console.log(created.price);
```

▼ 実行結果（`node utility.ts`。`tsc` はエラーなし）

```
{ id: 'book-001', title: 'TypeScript入門', author: '山田太郎', price: 2980 }
{ id: 'book-001', title: 'TypeScript入門', author: '山田太郎', price: 1980 }
2980
```

▼ コードを1つずつ分解して解説

解説1: Omit で「id の無い型」を作る

```
type CreateBookInput = Omit<Book, "id">;
```

- `Omit<元の型, "取り除くキー">`（オミット：省く）は、元の型から指定したプロパティを取り除いた**新しい型**を作ります。
- 新しく本を登録するとき、`id` はまだありません（データベースが保存するときに付けます）。だから「`id` 以外の3項目が必須」の型が欲しいのです。
- `Book` を書き換えたわけではないので、`Book` 自体には `id` が残っています。

解説2: Partial で「全部省略できる型」を作る

```
type UpdateBookInput = Partial<CreateBookInput>;
```

- `Partial<型>`（パーシャル：部分的な）は、すべてのプロパティに `?` を付けた型を作ります。
- 更新のときは「価格だけ」「タイトルだけ」のように、**変えたい項目だけ**を送りたいので、全部を省略可能にしておきます。`updateBook(created, { price: 1980 })` が通るのはこのためです。

解説3: スプレッドを重ねて「上書きした新しい本」を作る

```
function updateBook(book: Book, changes: UpdateBookInput): Book {
  return { ...book, ...changes };
}
```

- `{ ...book, ...changes }` は「まず `book` の中身を全部並べ、そのあとに `changes` の中身を並べる」という意味です。同じキーがあると**後ろが勝つ**ので、`price` だけが `1980` に上書きされます。
- 0.10.1 で学んだとおり、スプレッドは**新しいオブジェクト**を作るので、元の `created` の価格は `2980` のままです（実行結果の3行目）。

型が守ってくれる様子も確かめておきましょう。次のファイル（`utility-ng.ts`）では、わざと2か所を間違えています。

```ts
interface Book {
  id: string;
  title: string;
  author: string;
  price: number;
}
type CreateBookInput = Omit<Book, "id">;
type UpdateBookInput = Partial<CreateBookInput>;

// ① 新規登録の入力に id を書いてしまった
const input: CreateBookInput = {
  id: "book-001",
  title: "TypeScript入門",
  author: "山田太郎",
  price: 2980,
};

// ② 更新の入力で price に文字列を入れてしまった
const changes: UpdateBookInput = { price: "1980" };
```

▼ 型チェックの結果（`npx tsc --noEmit --strict --target esnext utility-ng.ts`）

```
utility-ng.ts:12:3 - error TS2353: Object literal may only specify known properties,
and 'id' does not exist in type 'CreateBookInput'.

12   id: "book-001",
     ~~


utility-ng.ts:19:36 - error TS2322: Type 'string' is not assignable to type 'number'.

19 const changes: UpdateBookInput = { price: "1980" };
                                      ~~~~~


  utility-ng.ts:5:3 - The expected type comes from property 'price' which is declared
here on type 'Partial<CreateBookInput>'
    5   price: number;
        ~~~~~

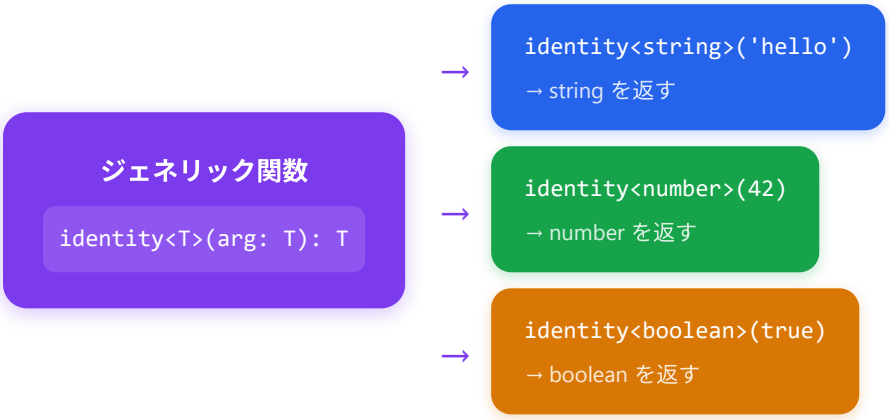

Found 2 errors in the same file, starting at: utility-ng.ts:12
```

- 1つ目は「オブジェクトには型に書かれたプロパティしか書けません。`id` は `CreateBookInput` に存在しません」という意味です。`Omit` でちゃんと `id` が取り除かれていることが分かります。
- 2つ目は「`string` は `number` に代入できません」。その下の補足（`The expected type comes from ...`）は「`number` という期待は、5行目の `price: number` から来ています」と、**型が決められた場所**まで教えてくれています。`Partial` で省略可能になっても、型そのもの（`number`）は守られているのです。

5. ジェネリクス

5.1 基本概念

ジェネリクス（Generics） は、型を「パラメータ化」する仕組みです。関数やクラス、インターフェースを定義する時点では型を決めず、使う時点で具体的な型を指定できます。



▼ このコードがやること（先に日本語で）：ジェネリクスの**ありがたみを実感するための「悪い例」**を先に見ます。「受け取った値をそのまま返すだけ」の関数を作りたいのに、`any` を使うと型情報が失われ、型ごとに関数を作ると同じコードが量産されてしまう——という困りごとを示します。次のブロックでこれをジェネリクスが解決します。

```
// --- ジェネリクスなしの場合（問題あり）---

// 方法1: any を使う → 型安全性が失われる
// any を使えば何でも受け取れるが...
function identityAny(arg: any): any {
  return arg;
}
// result1 は any 型（string 情報が失われる）
// 結果も any 扱いになり、型安全性が消える。
const result1 = identityAny("hello");

// 方法2: 型ごとに関数を作る → コードの重複
// string 専用版。
function identityString(arg: string): string {
  return arg;
}
// number 専用版。型が増えるごとに関数を量産する必要がある。
function identityNumber(arg: number): number {
  return arg;
}
```

▼ **このコードがやること（先に日本語で）**：いよいよ**ジェネリクス**の登場です。`<T>` という「あとで決まる型の入れ物」を使い、**1つの関数であらゆる型に対応**させます。`identity<string>(...)` のように型を指定したり、値から自動推論させたりできます。カギは「`T` は型の変数（プレースホルダー）」という考え方。使うときに具体的な型が当てはまります。

```ts
// ============================================================================
// ジェネリクスを使った場合 − 「型を変数化」して1つの関数で全てに対応
// ============================================================================

// `<T>` の T は「Type」の頭文字。慣習的によく使われる。
// 「この関数は何かの型 T を受け取り、同じ T を返すよ」という宣言。
//
// arg: T  → 引数の型は T
// : T     → 戻り値の型も T（引数と同じ型）
function identity<T>(arg: T): T {
  // 受け取ったものをそのまま返す
  return arg;
}

// ────────── 使用例(1): 型を明示的に指定する ──────────
// `<string>` のように山カッコで型を渡すことを「型引数を渡す」と呼ぶ。
const str = identity<string>("hello");
//          ↑ T = string と決まるので、引数は string 限定、戻り値も string
console.log(str);
// ▼ 実行結果
// hello
// ▼ str の型: string
//  (number を渡そうとするとコンパイルエラー: identity<string>(42) は ❌)

const num = identity<number>(42);
console.log(num);
// ▼ 実行結果
// 42
// ▼ num の型: number

const bool = identity<boolean>(true);
console.log(bool);
// ▼ 実行結果
// true
// ▼ bool の型: boolean


// ────────── 使用例(2): 型を省略する（型推論にお任せ） ──────────
// 引数の値 "world" を見て、TypeScript が「あ、T = string ですね」と自動推論する。
// 多くの場合、型引数を書かなくてOK。
const inferred = identity("world");
console.log(inferred);
// ▼ 実行結果
// world
// ▼ inferred の型: string（自動推論された）
```

▼ コードを1つずつ分解して解説

ジェネリクスは初心者が最初にとまどう概念の代表です。`<T>` という見慣れない書き方を、**1つずつ**ていねいに見ていきましょう。

解説1: ジェネリック関数の定義（`<T>`）

```
function identity<T>(arg: T): T {
  // 受け取ったものをそのまま返す
  return arg;
}
```

- `function identity<T>(...)` の `<T>` が**ジェネリクス**の正体です。これは「**あとで決まる型を入れておく箱（型の変数）**」で、関数を定義する時点では「何型か」を決めません。
- `arg: T` … 引数 `arg` の型を `T` にしています。「`arg` の型は、あとで決まる `T` と同じ」という意味です。
- `: T` … 戻り値の型も `T` にしています。つまり「**受け取った型と同じ型を返す**」関数だと宣言しています。
- `T` という名前は「Type（型）」の頭文字で、慣習でよく使われるだけです。`<U>` でも `<Item>` でも動きますが、まずは `T` に慣れましょう。

> **用語: ジェネリクス（総称型）** … 型を「あとで決める変数」のように扱う仕組み。`<T>` のように山カッコで書き、1つの関数や型をあらゆる型に対応させられます。

> **用語: 型パラメータ** … `<T>` の `T` のこと。関数の引数（パラメータ）が「値の入れ物」であるのに対し、型パラメータは「型の入れ物」です。

解説2: 使うとき型を明示的に指定する

```
const str = identity<string>("hello");
const num = identity<number>(42);
const bool = identity<boolean>(true);
```

- 関数名のうしろに `<string>` のように書くと、「**今回の `T` は `string` ですよ**」と型を渡せます。これを「型引数を渡す」と言います。
- `identity<string>("hello")` では `T = string` に決まるので、引数は文字列しか受け付けず、戻り値（`str`）の型も `string` になります。
- 同様に `identity<number>(42)` なら `T = number`、`identity<boolean>(true)` なら `T = boolean` になります。**1つの関数が、渡した型に応じて姿を変える**のがジェネリクスの威力です。
- もし `identity<string>(42)` のように型と中身が食い違うと、TypeScript が即エラーで教えてくれます。

解説3: 型を省略して自動推論にまかせる

```
const inferred = identity("world");
// ▼ inferred の型: string（自動推論された）
```

- 多くの場合、`<string>` のような型引数は**書かなくてもかまいません。**
- `identity("world")` のように値だけ渡すと、TypeScript が引数 `"world"` を見て「あ、`T` は `string` だな」と**自動で判断（型推論）**してくれます。
- そのため `inferred` の型はちゃんと `string` になります。実務では、このように型を省いて書くことが多いです。

> **用語: 型推論** … 型注釈を書かなくても、TypeScript が値から「これは何型か」を自動で判断してくれる機能のこと。

ここまでを、いつもの「値の引数」と並べて図にすると次のようになります。関数の引数 `x` が「あとで入る**値**の置き場所」なら、`<T>` は「あとで入る**型**の置き場所」です。**ジェネリクスとは「型を受け取る引数」**だと考えると、ぐっと分かりやすくなります。

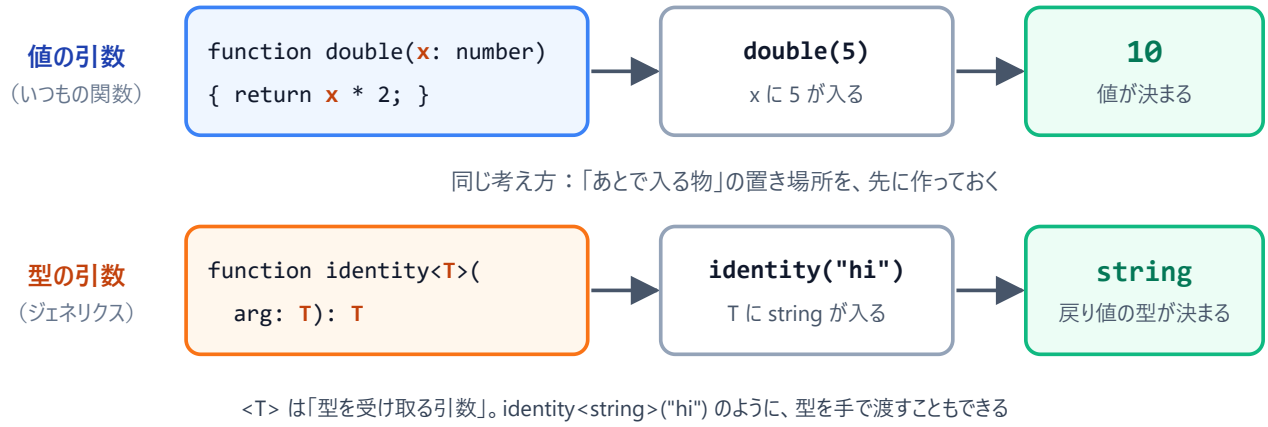


図2-11: ジェネリクスは「型の引数」。値の引数 x と同じく、呼び出したときに中身（型）が決まる

> **ジェネリクスのありがたみ:** 上の `identity` 関数は、もし型を毎回手書きだったら `identityString` `identityNumber` `identityBoolean` … と無限に増えてしまう。`<T>` 1つで全てに対応できる。配列メソッドの `Array.prototype.map<U>(callback)` などはまさにこの仕組みで動いている。

> **▼ このコードがやること（先に日本語で）**：型パラメータは1つだけでなく、`<T, U>` のように**複数**持てます。ここでは2つの値をペア（タプル）にして返す `pair` 関数を作ります。あわせて `T・U・K・V` といった「型パラメータ名のよくある慣習」も紹介します（意味は普通の変数名と同じく自由ですが、慣習に従うと読みやすい）。

```
// --- ジェネリクスの慣習的な型パラメータ名 ---
// T: Type（一般的な型）
// U: 2番目の型パラメータ
// K: Key（オブジェクトのキー）
// V: Value（オブジェクトの値）
// E: Element（要素）
// R: Return（戻り値）

// 複数の型パラメータ
// <T, U> で2種類の型パラメータを宣言。タプル [T, U] を返す。
function pair<T, U>(first: T, second: U): [T, U] {
  // 引数2つをタプルにして返す。
  return [first, second];
}

// [string, number]
// 型を明示。T=string, U=number。
const p1 = pair<string, number>("hello", 42);
// [string, boolean]（型推論）
// 引数から自動推論。
const p2 = pair("田中", true);
```

5.2 実用的な例

例1: 配列操作のユーティリティ関数

> ▼ **このコードがやること（先に日本語で）**：ジェネリクスの実用例として、「どんな型の配列でも使える」便利関数を3つ作ります——最初の要素を取る `getFirst`、最後の要素を取る `getLast`、中身をシャッフルする `shuffle` です。`<T>` のおかげで、文字列の配列でも数値の配列でも、**同じ関数1つで型安全に**扱えます。

```ts
// 配列の最初の要素を取得する関数
// <T> を導入。引数の T[] で「T の配列」を受け取る。戻り値は T かもしくは undefined（空配列のとき）。
function getFirst<T>(arr: T[]): T | undefined {
  // [0] は1要素目。配列が空なら undefined になる。
  return arr[0];
}

// string | undefined
// T = string と推論。
const firstFruit = getFirst(["りんご", "みかん"]);
// number | undefined
// T = number と推論。
const firstNum = getFirst([10, 20, 30]);

// 配列の最後の要素を取得する関数
// 同じく <T> を使う汎用関数。
function getLast<T>(arr: T[]): T | undefined {
  // 三項演算子。長さが0より大きければ最後の要素、そうでなければ undefined。
  return arr.length > 0 ? arr[arr.length - 1] : undefined;
}

// 配列をシャッフルする関数
// 配列を受け取り、シャッフルした新しい配列を返す。
function shuffle<T>(arr: T[]): T[] {
  // ... はスプレッド構文。「配列の中身を全部展開する」演算子。[...arr] で「元配列を1段コピーした新しい配列」を作れる。元を破壊しないため。
  const result = [...arr];
  // 末尾から先頭手前まで逆順にループ（Fisher-Yates シャッフル）。
  for (let i = result.length - 1; i > 0; i--) {
    // Math.random() は 0以上1未満の乱数。* (i+1) で 0..i の小数、Math.floor で切り捨てて整数に。
    const j = Math.floor(Math.random() * (i + 1));
    // 分割代入による値の入れ替え。左右を同時に評価して交換する。
    [result[i], result[j]] = [result[j], result[i]];
  }
  // シャッフル後の配列を返す。
  return result;
}

// string[]
const shuffledFruits = shuffle(["りんご", "みかん", "バナナ"]);
// number[]
const shuffledNums = shuffle([1, 2, 3, 4, 5]);
```

▼ コードを1つずつ分解して解説

3つの関数それぞれにジェネリクスの使いどころが詰まっています。順番に、**1つずつ**ていねいに見ていきましょう。

解説1: 最初の要素を取る `getFirst`

```
function getFirst<T>(arr: T[]): T | undefined {
  return arr[0];
}

// string | undefined
const firstFruit = getFirst(["りんご", "みかん"]);
// number | undefined
const firstNum = getFirst([10, 20, 30]);
```

- `getFirst<T>(arr: T[])` … `<T>` で型を変数化し、引数を `T[]`（`T` の配列）にしています。これで「**どんな型の配列でも受け取れる**」関数になります。
- 戻り値の型は `T | undefined` です。なぜ `undefined` が付くかというと、もし**空の配列**を渡されたら `arr[0]`（1番目の要素）が存在せず `undefined` になるからです。「無いかもしれない」を型で正直に表しています。
- `getFirst(["りんご", "みかん"])` では `T = string` と自動推論され、戻り値は `string | undefined` になります。数値の配列を渡せば `number | undefined` です。**同じ関数1つで型ごとに正しく対応**できるのがジェネリクスの利点です。

解説2: 最後の要素を取る `getLast`

```
function getLast<T>(arr: T[]): T | undefined {
  return arr.length > 0 ? arr[arr.length - 1] : undefined;
}
```

- こちらも `<T>` を使った汎用関数で、配列の**最後の要素**を返します。
- `arr.length > 0 ? ... : ...` は**三項演算子**です。「`条件 ? 条件が真のときの値 : 偽のときの値`」という形で、ここでは「要素が1個以上あるなら最後の要素、なければ `undefined`」という意味になります。
- `arr[arr.length - 1]` … `arr.length` は要素数です。添え字は0始まりなので、最後の要素の添え字は「要素数 - 1」になります（例: 3個なら添え字は2）。

> **用語: 三項演算子** … `条件 ? A : B` の形で「条件が真なら A、偽なら B」を返す式。短い条件分岐を1行で書けます。

解説3: 配列をシャッフルする `shuffle`

```
function shuffle<T>(arr: T[]): T[] {
  const result = [...arr];
  for (let i = result.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [result[i], result[j]] = [result[j], result[i]];
  }
  return result;
}
```

- `const result = [...arr]` … `...`（ドット3つ）は**スプレッド構文**で、「元の配列の中身を全部展開して、新しい配列にコピーする」書き方です。**元の `arr` を壊さない**ために、コピーを作ってから加工しています。
- `for (let i = ...; i > 0; i--)` … 末尾から先頭手前まで逆向きに繰り返すループです。これは「フィッシャー・イェーツ法」というシャッフルの定番アルゴリズムです。
- `Math.floor(Math.random() * (i + 1))` … `Math.random()` は「0以上1未満の乱数」を返します。`* (i + 1)` で範囲を広げ、`Math.floor`（切り捨て）で整数にして、`0` から `i` までのランダムな添え字 `j` を作っています。
- `[result[i], result[j]] = [result[j], result[i]]` … **分割代入を使った値の入れ替え**です。一時変数を使わずに、`result[i]` と `result[j]` の中身を一度に交換しています。
- 戻り値の型は `T[]`。シャッフル後の新しい配列を返します。

> **用語: スプレッド構文（`...`）** … 配列やオブジェクトの中身を「展開」する書き方。`[...arr]` で配列を1段コピーでき、元を壊さずに加工するのに使います。

例2: ジェネリックなインターフェース

> **▼ このコードがやること（先に日本語で）**: ジェネリクスは関数だけでなく `interface` にも使えます。ここでは「成功・失敗の枠組みは共通だが、**中身のデータの型は使う側で決める**」API レスポンス型 `ApiResponse<T>` を作ります。`ApiResponse<User>` ならデータは User、`ApiResponse<Book[]>` なら書籍の配列、というふうに型を差し替えられるのがポイントです。

```ts
// API レスポンスの汎用型
// ジェネリックなインターフェース。T は使う側が決める型。
interface ApiResponse<T> {
  // data の中身の型は T 次第。
  data: T;
  // HTTP ステータス (200, 404, 500, ...)
  status: number;
  message: string;
  // Date は JavaScript 標準の日時オブジェクト。
  timestamp: Date;
}

// ユーザーデータ用のレスポンス
interface User {
  id: number;
  name: string;
  email: string;
}

// T = User として ApiResponse を使う。data は User 型に決まる。
const userResponse: ApiResponse<User> = {
  data: {
    id: 1,
    name: "田中太郎",
    email: "tanaka@example.com",
  },
  status: 200,
  message: "成功",
  // new Date() は「今この瞬間の日時」を表す Date オブジェクトを作る。
  timestamp: new Date(),
};

// 書籍データ用のレスポンス
interface Book {
  id: string;
  title: string;
}

// T = Book[]（書籍の配列）として使う。data は Book[] に決まる。
const bookResponse: ApiResponse<Book[]> = {
  data: [
    { id: "1", title: "TypeScript入門" },
    { id: "2", title: "React実践ガイド" },
  ],
  status: 200,
  message: "成功",
  timestamp: new Date(),
};
```

例3: 制約付きジェネリクス（extends）

> **▼ このコードがやること（先に日本語で）**：「どんな型でもOK」ではなく「**ある条件を満たす型だけ**受け取りたい」ときに使う、制約付きジェネリクス（ `<T extends ...>` ）を学びます。例えば「 `length` を持つものだけ」「そのオブジェクトに実在するキーだけ」のように絞り込めます。少し難しい所なので、 `extends` で「型に条件を付けられる」とつかめれば十分です。

```ts
// T は必ず length プロパティを持つ型に制限
// <T extends X> は「T は X 型の部分型でなければならない」という制約。ここでは「length: number を持つ何か」に制限。
function getLength<T extends { length: number }>(arg: T): number {
  // length プロパティが必ずあるので安全に読める。
  return arg.length;
}

// OK: string は length を持つ → 5
// 文字列は length を持つ。
getLength("hello");
// OK: 配列は length を持つ → 3
// 配列も length を持つ。
getLength([1, 2, 3]);
// OK: length プロパティがある → 10
// 自前で { length: 10 } を渡しても OK。
getLength({ length: 10 });

// getLength(42);
// エラー: Argument of type 'number' is not assignable to
// parameter of type '{ length: number; }'
// number に length は無い → 制約違反。
// → number には length プロパティがない

// オブジェクトのキーに制限をかける
// K は「T のキーの中のいずれか」に制限。keyof T は T のキーのユニオン型を作る演算子。T[K] は「T 型オブジェクトの K プロパティの型」を取り出す書き方。
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

// ↑ user の型は { name: string; age: number; email: string; } と推論される。
const user = { name: "田中", age: 30, email: "tanaka@example.com" };

// string
// K="name" なので戻り値の型は user.name の型 = string。
const name = getProperty(user, "name");
// number
// K="age" なので number。
const age = getProperty(user, "age");

// "address" は user のキーに含まれない。
// getProperty(user, "address");
// エラー: Argument of type '"address"' is not assignable to
// parameter of type '"name" | "age" | "email"'
```

例4: ジェネリックなクラス

▼ **このコードがやること（先に日本語で）**：ジェネリクスを**クラス**に使う例として、「最後に入れたものが最初に出てくる」スタックというデータ構造を `Stack<T>` として作ります。`new Stack<number>()` なら数値専用、`new Stack<string>()` なら文字列専用、というふうに、**同じクラスを違う型で何度でも再利用**できるのがポイントです。

```ts
// スタック（Last In, First Out）のデータ構造
// class はオブジェクトの設計図を定義するキーワード。<T> でジェネリッククラスにする。「T 型の要素を入れるスタック」。
class Stack<T> {
  // private = クラスの外からはアクセス禁止。T[] 型の空配列で初期化。
  private items: T[] = [];

  // メソッドの宣言。スタックに要素を追加。
  push(item: T): void {
    // this は「このインスタンス自身」を指す。items 配列に追加。
    this.items.push(item);
  }

  // スタックの末尾を取り出して返す。空なら undefined。
  pop(): T | undefined {
    // 配列の pop メソッドは末尾を削除して返す（破壊的）。
    return this.items.pop();
  }

  // 末尾を「見るだけ」（削除しない）。
  peek(): T | undefined {
    // length-1 が最後の要素の添え字。
    return this.items[this.items.length - 1];
  }

  // 空かどうか。
  isEmpty(): boolean {
    return this.items.length === 0;
  }

  // 要素数を返す。
  size(): number {
    return this.items.length;
  }
}

// 数値スタック
// T = number で Stack を作る。new はクラスからインスタンスを作るキーワード。
const numberStack = new Stack<number>();
numberStack.push(10);
numberStack.push(20);
numberStack.push(30);
// 30
// 最後に push した 30 が取り出される（LIFO の性質）。
console.log(numberStack.pop());
// 20
// 次に末尾にあるのは 20。
console.log(numberStack.peek());
```

```
// 文字列スタック
// T = string で別のスタックを作る。同じ Stack クラスを違う型で再利用できる。
const stringStack = new Stack<string>();
stringStack.push("TypeScript");
stringStack.push("React");
// "React"
console.log(stringStack.pop());
```

例5: 書籍アプリで使える findById（制約付きジェネリクスの実践）

> ▼ **このコードがやること（先に日本語で）**:「 `id` を持つものの一覧」から、指定した `id` の1件を探す関数 `findById` を作ります。本の一覧でも著者の一覧でも、**同じ1つの関数**で探せて、しかも戻り値の型は「本なら本」「著者なら著者」と正しく決まります。 `id` を持たない配列を渡すと、型エラーで止まることも確かめます。ファイル名は `find-by-id.ts` です。

```ts
// T は「id: string を持っている何かの型」。それ以外の型は受け付けない
function findById<T extends { id: string }>(
  items: T[],
  id: string,
): T | undefined {
  // find は「条件に合う最初の要素」を返す。見つからなければ undefined
  return items.find((item) => item.id === id);
}

interface Book {
  id: string;
  title: string;
  price: number;
}

interface Author {
  id: string;
  name: string;
}

const books: Book[] = [
  { id: "b1", title: "TypeScript入門", price: 2980 },
  { id: "b2", title: "React実践ガイド", price: 3300 },
];

const authors: Author[] = [{ id: "a1", name: "山田太郎" }];

// T = Book と自動で決まる → 戻り値は Book | undefined
const book = findById(books, "b2");
console.log(book?.title);

// T = Author と自動で決まる → 戻り値は Author | undefined
const author = findById(authors, "a1");
console.log(author?.name);

// 見つからないときは undefined
const missing = findById(books, "b999");
console.log(missing);
```

▼ 実行結果（`node find-by-id.ts`。`tsc` はエラーなし）

```
React実践ガイド
山田太郎
undefined
```

▼ コードを1つずつ分解して解説

解説1: `<T extends { id: string }>` で「受け付ける型」に条件を付ける

```
function findById<T extends { id: string }>(
  items: T[],
  id: string,
): T | undefined {
```

- `<T extends { id: string }>` は「`T` はどんな型でもよいが、**`id: string` を持っていること**」という条件付きの型の引数です（例3で学んだ制約付きジェネリクス）。
- この条件があるので、関数の中で `item.id` と書いても TypeScript は「`T` なら必ず `id` を持っている」と分かり、エラーになりません。
- 引数や戻り値の型が長くなったので、`(` のあとで改行して1行に1つずつ書いています（Prettier という整形ツールもこの形にします）。

解説2: 戻り値の型は「渡した配列」から自動で決まる

```
const book = findById(books, "b2");
console.log(book?.title);
```

- `books` は `Book[]` なので、`T = Book` と自動で決まり、`book` の型は `Book | undefined` になります。だから `book?.title` と書くと、VS Code が `title` や `price` を補完してくれます。
- 同じ関数に `authors` を渡せば `T = Author` になり、今度は `author?.name` が使えます。`any` を使った関数では、この「型の連動」は得られません。
- `?.`（オプショナルチェイニング）は「左が `undefined` ならそこで止めて `undefined` を返す」書き方です。見つからない可能性（`| undefined`）があるので付けています。

解説3: 条件を満たさない配列はエラーになる

`id` を持たない数値の配列を渡すと、どうなるでしょうか（`find-by-id-ng.ts`）。

```
function findById<T extends { id: string }>(
  items: T[],
  id: string,
): T | undefined {
  return items.find((item) => item.id === id);
}

// 数値の配列には id が無いので、T の条件を満たさない
findById([1, 2, 3], "1");
```

▼ 型チェックの結果（`npx tsc --noEmit --strict --target esnext find-by-id-ng.ts`）

```
find-by-id-ng.ts:9:11 - error TS2322: Type 'number' is not assignable to type '{ id:
string; }'.

9 findById([1, 2, 3], "1");
            ~


find-by-id-ng.ts:9:14 - error TS2322: Type 'number' is not assignable to type '{ id:
string; }'.

9 findById([1, 2, 3], "1");
               ~


find-by-id-ng.ts:9:17 - error TS2322: Type 'number' is not assignable to type '{ id:
string; }'.

9 findById([1, 2, 3], "1");
                  ~


Found 3 errors in the same file, starting at: find-by-id-ng.ts:9
```

- 配列の要素 `1`・`2`・`3` のそれぞれについて「`number` は `{ id: string; }`（id を持つオブジェクト）に代入できません」と言われています。`extends` で付けた条件が、ちゃんと門番の役目を果たしているのが分かります。

---

6. ユニオン型とリテラル型

6.1 ユニオン型

ユニオン型（Union Type） は、複数の型のいずれかを受け入れる型です。`|`（パイプ）で型を区切ります。

> **▼ このコードがやること（先に日本語で）:**「`string` または `number` 」のように、複数の型のどれかを受け入れる**ユニオン型**（`|` で区切る）を学びます。大事なのは「使うときは中身が今どっちの型かを `typeof` で確かめてから使う」こと——この確認を**型ガード**と呼びます。配列に対して使うときは `( )` の付け方で意味が変わる点にも注意します。

```ts
// ============================================================================
// ユニオン型の基本 - 「A型 または B型」を受け入れる型
// ============================================================================

// (1) string 型 または number 型を入れられる変数を宣言する
//     let は const と違って後から再代入できる。
//     | は「または」の意味（AND ではなく OR）。
let value: string | number;

// OK（string なので許可）
value = "hello";
// OK（number なので許可）
value = 42;
// value = true;
// ▼ エラー
// Type 'boolean' is not assignable to type 'string | number'.
//   （boolean は string|number に代入不可）


// ============================================================================
// 関数の引数にユニオン型を使う場合の「型の絞り込み」
// ============================================================================
// ユニオン型のままでは「両方の型に共通するメソッドしか」呼べない。
// typeof 演算子などで「いま中身が何の型か」を確かめてから使う。
// この技法を「型ガード（Type Guard）」と呼ぶ。

function formatId(id: string | number): string {
  // (1) typeof 演算子は「値の型を文字列で返す」演算子
  //     文字列なら "string"、数値なら "number" を返す。
  if (typeof id === "string") {
    // (2) この if ブロックの中だけ、TS は id を string として扱ってくれる。
    //     なので string 専用メソッド .toUpperCase() が使える。
    // 例: "abc" → "ABC"
    return id.toUpperCase();
  } else {
    // (3) else 側では「string ではない」=「number しか残らない」と推論される。
    //     .toString() は数値を文字列に変換するメソッド。
    //     .padStart(5, "0") は「5文字に満たないなら左を 0 で埋める」メソッド。
    // 例: 42 → "00042"
    return id.toString().padStart(5, "0");
  }
}

console.log(formatId("abc"));
// ▼ 実行結果
// ABC

console.log(formatId(42));
// ▼ 実行結果
```

```
// 00042


// ============================================================================
// 配列のユニオン型 – 配列の中に複数の型が混ざってもOKにする
// ============================================================================
// 注意: () で囲まないと意味が変わる！
//   string | number[]   → 「string 1個」または「number の配列」
//   (string | number)[] → 「string と number が混在した配列」（こちらが正しい）
const mixed: (string | number)[] = [1, "two", 3, "four"];
console.log(mixed);
// ▼ 実行結果
// [ 1, 'two', 3, 'four' ]
```

▼ コードを1つずつ分解して解説

ユニオン型は「型ガード」とセットで理解するのが大事です。順番に、**1つずつ**ていねいに見ていきましょう。

解説1: ユニオン型で「A または B」を表す

```
let value: string | number;

// OK（string なので許可）
value = "hello";
// OK（number なので許可）
value = 42;
// ▼ エラー
// value = true;
```

- `: string | number` が**ユニオン型**の型注釈です。`|`（パイプ）は「**または**」を意味し、「`string` **または** `number` のどちらかが入る」という型になります。
- そのため `value = "hello"`（文字列）も `value = 42`（数値）もOKですが、`value = true`（真偽値）は許可されていない型なのでエラーになります。
- ここでは `let` を使っている点に注目してください。`let` は再代入できるので、文字列を入れたあとに数値を入れ直せます。

> **用語: ユニオン型** … `|` で複数の型をつないだ「どれか1つ」を表す型。許したい型の候補が複数あるときに使います。

解説2: 型ガードで「いま中身が何型か」を確かめる

```
function formatId(id: string | number): string {
  if (typeof id === "string") {
    // 例: "abc" → "ABC"
    return id.toUpperCase();
  } else {
    // 例: 42 → "00042"
    return id.toString().padStart(5, "0");
  }
}
```

- 引数 `id` は `string | number` 型です。このままだと「**文字列と数値の両方に共通する操作**」しかできません（片方にしかないメソッドは呼べない）。
- そこで `if (typeof id === "string")` で「いま `id` は文字列か？」を確認しています。`typeof`（タイプオブ）は値の型を文字列で返す演算子です。
- この `if` の中では、TypeScript が「ここでは `id` は `string` で確定」と判断してくれるので、文字列専用の `.toUpperCase()`（大文字化）が安全に呼べます。
- `else` 側では「文字列ではない＝残るのは `number` だけ」と判断され、数値を文字列にする `.toString()` などが使えます。この「中身の型を絞り込んでから使う」技法を**型ガード**と呼びます。

> **用語: 型ガード（Type Guard）** … `typeof` などで「いま値が何型か」を確認し、その範囲内で型を絞り込むこと。ユニオン型を安全に使うための必須テクニックです。

解説3: 配列に使うときは `( )` の位置に注意

```
const mixed: (string | number)[] = [1, "two", 3, "four"];
```

- 「文字列と数値が混ざった配列」を作りたいときは、`(string | number)[]` と書きます。**`( )` でユニオン型を囲んでから `[]` を付ける**のがポイントです。
- もし `( )` を付けずに `string | number[]` と書くと、意味が変わってしまいます。これは「**`string` 1個**」または「**`number` の配列**」という別物になります。
- 「混在した配列」を表したいときは必ず `( )` で囲む、と覚えておきましょう。

6.1.1 絞り込み（narrowing）を図と動くコードで確かめる

ユニオン型の値は、`if` で1つ調べるたびに「残りの候補（型）」が減っていきます。これを**型の絞り込み（narrowing：ナローイング）**と呼びます。

たとえ話: 封筒の中身が「お金・手紙・何も入っていない」のどれか分からないとき、まず「空っぽ？」と振って確かめ、次に「紙？」と触って確かめていくと、最後に残る可能性は1つになります。TypeScript も、あなたが書いた `if` の条件を読んで、同じように候補を減らしてくれます。

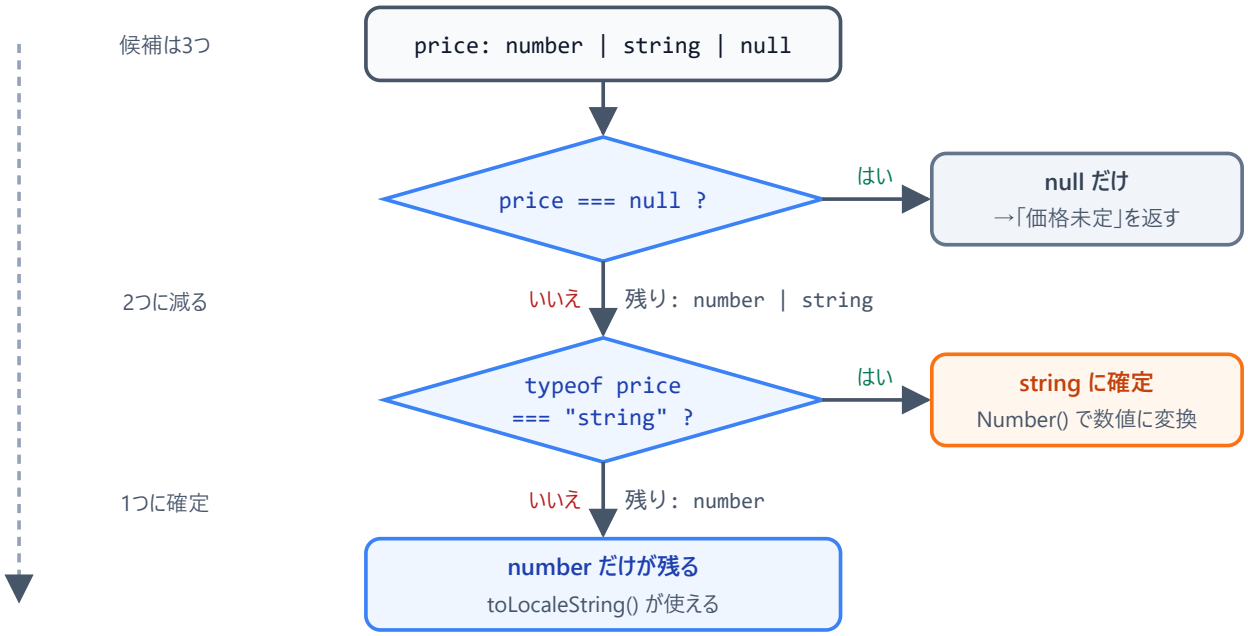


図2-12: 型の絞り込み。if で1つ調べるたびに、残りの候補（型）が減っていく

価格を表示用の文字に整える formatPrice

▼ このコードがやること（先に日本語で）: 書籍アプリの価格は「数値」「フォームに入力されたままの文字列」「まだ決まっていない（ `null` ）」のどれかになりうる、という場面を想定し、どれが来ても `2,980円` のような表示用の文字に整える関数を作ります。図2-12 の流れをそのままコードにしたものです。ファイル名は `format-price.ts` です。

```ts
// 価格は「数値」「文字列（フォームに入力されたまま）」「未定（null）」のどれか
function formatPrice(price: number | string | null): string {
  // ここでの price の型: number | string | null
  if (price === null) {
    // ここでの price の型: null
    return "価格未定";
  }

  // ここまで来たら null の可能性は消えた → number | string
  if (typeof price === "string") {
    // ここでの price の型: string
    // 文字列を数値に変換してから表示する
    const num = Number(price);
    return `${num.toLocaleString("ja-JP")}円`;
  }

  // 残りは number だけ → 数値専用の toLocaleString が使える
  return `${price.toLocaleString("ja-JP")}円`;
}

console.log(formatPrice(2980));
console.log(formatPrice("12800"));
console.log(formatPrice(null));
```

▼ 実行結果（`node format-price.ts`。`tsc` はエラーなし）

```
2,980円
12,800円
価格未定
```

▼ コードを1つずつ分解して解説

解説1: まず `null` を取り除く

```ts
if (price === null) {
  return "価格未定";
}
```

- `price === null` が成り立つ `{ }` の中では、`price` は `null` で確定します。ここで `return` して関数を抜けるのがポイントです。
- `return` したので、**この `if` より下に来た時点で `null` の可能性は消えています**。TypeScript はこれも理解していて、以降の `price` を `number | string` として扱います（このように「先に例外を片付けて抜ける」書き方を**早期リターン**と呼びます）。

解説2: `typeof` で文字列を見分けて数値に変換する

```
if (typeof price === "string") {
  const num = Number(price);
  return `${num.toLocaleString("ja-JP")}円`;
}
```

- `typeof price === "string"` の中では `price` は `string` です。`Number(price)` で数値に変換してから表示用に整えます。
- `toLocaleString("ja-JP")` は、数値を日本の書き方（3桁ごとのカンマ区切り）の文字列にするメソッドです。`12800` が `12,800` になります。

解説3: 最後に残るのは number だけ

```
return `${price.toLocaleString("ja-JP")}円`;
```

- `null` と `string` を取り除いたので、ここでの `price` は `number` だけです。数値のメソッド `toLocaleString` を、型ガードなしでそのまま呼べます。
- VS Code で各行の `price` にマウスを乗せると、場所ごとに型が `number | string | null` → `string` → `number` と変わっていくのが確認できます。

> **つまずきポイント: ユニオン型のまま、片方の型にしかないメソッドを呼ぶ** — `string | number` 型の値に、いきなり `toUpperCase()`（文字列専用）を呼ぶとエラーになります。TypeScript から見ると「もし数値だったら、そんなメソッドは無いよ」という状況だからです。

NG 例（`union-ng.ts`）

```
function shout(value: string | number): string {
  // string か number か分からないまま、string 専用のメソッドを呼んでいる
  return value.toUpperCase();
}
```

▼ 型チェックの結果

```
union-ng.ts:3:16 - error TS2339: Property 'toUpperCase' does not exist on type 'string
| number'.
  Property 'toUpperCase' does not exist on type 'number'.

3   return value.toUpperCase();
                 ~~~~~~~~~~~~


Found 1 error in union-ng.ts:3
```

- 2行目の `Property 'toUpperCase' does not exist on type 'number'.` が本当の理由です。「`string | number` のうち、**`number` の場合**に `toUpperCase` が無い」と教えてくれています。

OK 例（`union-ok.ts`）：`typeof` で絞り込んでから呼ぶ

```ts
function shout(value: string | number): string {
  // まず typeof で「今は string か？」を確かめる
  if (typeof value === "string") {
    return value.toUpperCase();
  }
  // ここでは number に絞り込まれている
  return value.toString();
}

console.log(shout("hello"));
console.log(shout(42));
```

▼ 実行結果

```
HELLO
42
```

> **つまずきポイント: `typeof null` は `"object"` になる** — JavaScript には「`typeof null` が `"object"` を返す」という昔からの有名な仕様があります。そのため「オブジェクトかどうか」を `typeof x === "object"` で調べても、**`null` は取り除けません**。

NG 例（`typeof-null-ng.ts`）

```ts
interface Book {
  title: string;
}

function getTitle(book: Book | null): string {
  // 「オブジェクトなら」のつもりで typeof を使った
  if (typeof book === "object") {
    return book.title;
  }
  return "（本がありません）";
}

console.log(typeof null);
console.log(getTitle(null));
```

▼ 型チェックの結果

```
typeof-null-ng.ts:8:12 - error TS18047: 'book' is possibly 'null'.

8     return book.title;
             ~~~~


Found 1 error in typeof-null-ng.ts:8
```

▼ 実行結果（`node typeof-null-ng.ts`。node 内部の行は省略）

```
object
C:\ts-practice\typeof-null-ng.ts:8
    return book.title;
               ^

TypeError: Cannot read properties of null (reading 'title')

Node.js v25.9.0
```

- TypeScript はこの仕様を知っているので、`typeof book === "object"` の中でも「`book` は `null` かもしれません（`'book' is possibly 'null'.`）」と教えてくれます。実行するとやはり `null` の `title` を読もうとして落ちます。

OK 例（`typeof-null-ok.ts`）：`null` かどうかは `!== null` で直接調べる

```ts
interface Book {
  title: string;
}

function getTitle(book: Book | null): string {
  // null かどうかは === null（または !== null）で直接調べる
  if (book !== null) {
    return book.title;
  }
  return "（本がありません）";
}

console.log(getTitle({ title: "TypeScript入門" }));
console.log(getTitle(null));
```

▼ 実行結果

```
TypeScript入門
（本がありません）
```

6.2 リテラル型

リテラル型（Literal Type） は、特定の値のみを許可する型です。

> **▼ このコードがやること（先に日本語で）:**「 `string` なら何でも」ではなく、「 `"north"` ・ `"south"` … のように**決められた値だけ**」を許す**リテラル型**を学びます。`type Direction = "north" | "south" | ...` のようにユニオン型と組み合わせて使い、typo（打ち間違い）を即座にエラーにできます。文字列・数値・真偽値それぞれの例を見ます。

```ts
// ============================================================================
// 文字列リテラル型 – 「決められた文字列のどれか」だけを許す型
// ============================================================================
// type で名前を付け、許可する値を | で並べる。Enum の代わりによく使う。

type Direction = "north" | "south" | "east" | "west";
//                 ↑ この4つの文字列以外は受け付けない

// OK（許可されている値）
let dir: Direction = "north";
// dir = "up";
// ▼ エラー
// Type '"up"' is not assignable to type 'Direction'.

console.log(dir);
// ▼ 実行結果
// north


// ============================================================================
// 数値リテラル型 – 「決められた数値のどれか」だけを許す型
// ============================================================================
// サイコロの目（1〜6）のように値が固定の場面で使う。
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;

// OK
let roll: DiceRoll = 3;
// roll = 7;
// ▼ エラー
// Type '7' is not assignable to type 'DiceRoll'.


// ============================================================================
// 真偽値リテラル型 – true だけ、または false だけを許す
// ============================================================================
// 滅多に使わないが、特定の値しか取らないフラグを表現したいとき便利。
type True = true;
// OK
let flag: True = true;
// ▼ エラー: Type 'false' is not assignable to type 'true'.
// flag = false;
```

6.2.1 型は「値の集まり」— 代入できる向き・できない向き

`Type 'string' is not assignable to type 'Direction'`（string は Direction に代入できない）のようなエラーは、**型を「入ってよい値の集まり（集合）」として見る**とすっきり理解できます。

たとえ話: 「日本の都市」という大きな袋の中に「東京23区」という小さな袋が入っていて、さらにその中に「新宿区」がある、という入れ子の関係を思い浮かべてください。「新宿区の住所」は必ず「日本の都市の住所」として通用しますが、「日本の都市の住所」が必ず「東京23区」とは限りません。型も同じで、**小さい（狭い）型の値は大きい（広い）型の変数に入れられますが、逆は確かめないと入れられません。**

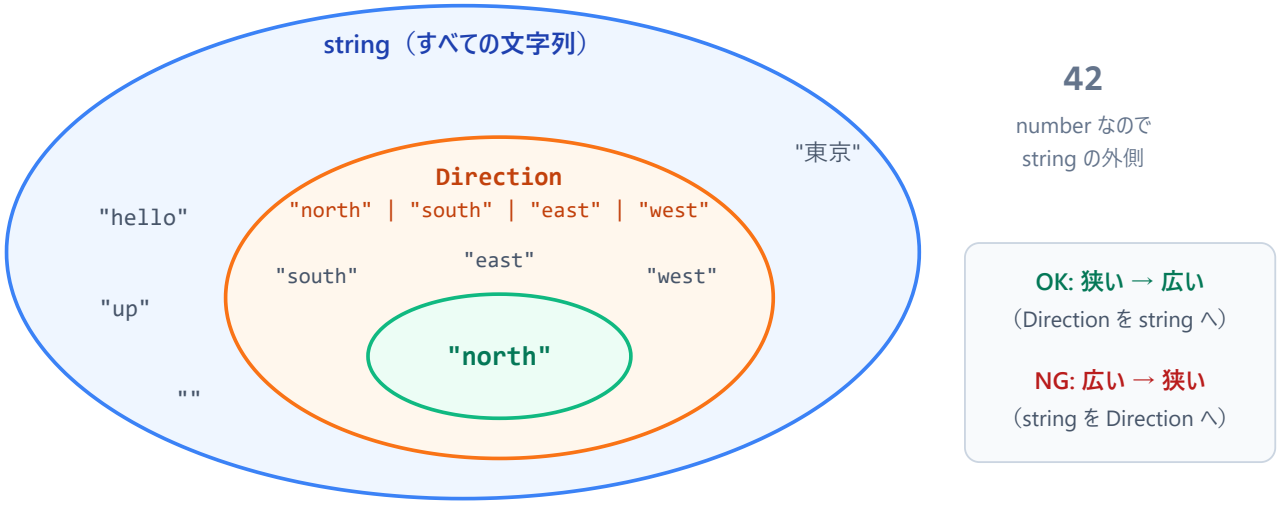


図2-13: 型は「入ってよい値の集まり」。"north" ⊂ Direction ⊂ string の入れ子になっている

代入できる向きを確かめる

▼ このコードがやること（先に日本語で）: 方角の型 `Direction` を使い、「狭い型 → 広い型」の代入は素直に通ること、「広い型 → 狭い型」の代入は**中身を確かめてから**でないとできないことを、実際に動かして確かめます。ファイル名は `direction.ts` です。

```ts
type Direction = "north" | "south" | "east" | "west";

// 狭い型（Direction）の値
const d: Direction = "north";

// 狭い型 → 広い型（string）への代入は OK
const s: string = d;
console.log(s);

// ユーザーが入力した文字列。どんな文字列が来るか分からないので広い型 string
const input: string = "up";

// 中身が本当に方角かどうかを調べる型ガード関数
function isDirection(value: string): value is Direction {
  return (
    value === "north" ||
    value === "south" ||
    value === "east" ||
    value === "west"
  );
}

if (isDirection(input)) {
  // ここでは input は Direction に絞り込まれているので代入できる
  const d2: Direction = input;
  console.log("方角です:", d2);
} else {
  console.log("方角ではありません:", input);
}
```

▼ **実行結果**（`node direction.ts`。`tsc` はエラーなし）

```
north
方角ではありません: up
```

▼ コードを1つずつ分解して解説

解説1: 狭い型 → 広い型 はそのまま代入できる

```ts
const d: Direction = "north";
const s: string = d;
```

- `Direction` に入る値（`"north"` など）は、どれも文字列です。つまり **`Direction` の値は必ず `string` としても正しい**ので、`string` 型の変数にそのまま入れられます（図2-13 の小さい円から大きい円への向き）。

解説2: 広い型 → 狭い型 は、確かめないと代入できない

次のように、確かめずに代入しようとするとエラーになります（`direction-ng.ts`）。

```
type Direction = "north" | "south" | "east" | "west";

const input: string = "up";

// 広い型（string）→ 狭い型（Direction）へ、確かめずに代入しようとした
const d2: Direction = input;
```

▼ 型チェックの結果

```
direction-ng.ts:6:7 - error TS2322: Type 'string' is not assignable to type
'Direction'.

6 const d2: Direction = input;
        ~~


Found 1 error in direction-ng.ts:6
```

- `string` 型の値は `"up"` かもしれないし `"東京"` かもしれません。`Direction` の円の外にある値かもしれないので、TypeScript は代入を止めます。これが `is not assignable to`（代入できない）の正体です。

解説3: 型ガード関数で「中身を確かめてから」代入する

```
function isDirection(value: string): value is Direction {
  return (
    value === "north" ||
    value === "south" ||
    value === "east" ||
    value === "west"
  );
}
```

- 戻り値の型 `value is Direction` は「この関数が `true` を返したら、`value` は `Direction` です」と TypeScript に教える書き方です（発展編の「型ガード」でもう一度出てきます）。
- `if (isDirection(input)) { ... }` の中では `input` が `Direction` に絞り込まれるので、`const d2: Direction = input;` が通ります。実際には `"up"` は方角ではないので、`else` 側が実行されました。

> **ヒント:** エラー文の `Type 'A' is not assignable to type 'B'` は、「A という集まりは、B という集まりの中に収まりきらない（はみ出す値がある）」と読み替えると、何を直せばよいかが見えてきます。「B を広げる（型定義を直す）」か「A を確かめて狭める（型ガード）」かのどちらかです。

6.3 書籍管理アプリでの実践例

▼ **このコードがやること（先に日本語で）**：リテラル型のユニオンを、実際の書籍管理アプリの「読書ステータス」に応用します。`ReadingStatus`（「読みたい・読書中・読了」の3択）を型として定義し、`Book` の `status` をその3つだけに限定します。`"finished"` のような決められていない値を入れると即エラーになる——これが型で「データの正しさ」を守る実例です。

```typescript
// ===== 読書ステータスの定義 =====

// 3つの文字列リテラルだけを許す型。
type ReadingStatus = "want_to_read" | "reading" | "completed";

// 書籍の基本型。
interface Book {
  id: string;
  title: string;
  author: string;
  // ステータスは ReadingStatus 型に限定。
  status: ReadingStatus;
}

// --- 正しい使い方 ---
const book1: Book = {
  id: "1",
  title: "TypeScript入門",
  author: "山田太郎",
  // 許可された値。
  status: "reading",
};

const book2: Book = {
  id: "2",
  title: "React実践ガイド",
  author: "佐藤花子",
  status: "completed",
};

const book3: Book = {
  id: "3",
  title: "Next.js入門",
  author: "鈴木一郎",
  status: "want_to_read",
};

// --- エラーになる例 ---
const invalidBook: Book = {
  id: "4",
  title: "間違った書籍",
  author: "テスト太郎",
  // エラー！
  // "finished" は ReadingStatus の3つに含まれていない。
  status: "finished",
  // Type '"finished"' is not assignable to type 'ReadingStatus'
  // → "want_to_read" | "reading" | "completed" のどれかにしてください
};
```

▼ **このコードがやること（先に日本語で）**：読書ステータスを使って、よくある実務処理——ラベルや色やアイコンへの変換、ステータスでの絞り込み（ `filter` ）、ステータスの変更——をまとめて作ります。 `switch` で全パターンを書けば、TypeScript が「漏れなく処理しているか」まで見てくれます。ステータス変更では「元を壊さず新しいオブジェクトを作る（ `{ ...book, status: ... }` ）」鉄則も登場します。

```ts
// ===== ステータスに応じた処理 =====

// ステータスを日本語ラベルに変換する関数。
function getStatusLabel(status: ReadingStatus): string {
  // switch 文で値ごとに分岐。
  switch (status) {
    // status が "want_to_read" のとき。
    case "want_to_read":
      // return すると関数が即終了。break 不要。
      return "読みたい";
    case "reading":
      return "読書中";
    case "completed":
      return "読了";
  }
  // すべてのケースを処理しているため、default 不要
  // ReadingStatus の3値すべてを書いたので、TypeScript は「漏れなし」と判断する。
  // TypeScript がすべてのケースが網羅されていることを保証
}

// ステータス → 色コード（#で始まる16進数）の変換。
function getStatusColor(status: ReadingStatus): string {
  switch (status) {
    case "want_to_read":
      // オレンジ
      return "#f39c12";
    case "reading":
      // 青
      return "#3498db";
    case "completed":
      // 緑
      return "#27ae60";
  }
}

// ステータス → アイコン名の変換（map 方式）。
function getStatusIcon(status: ReadingStatus): string {
  // Record<K, V> は「K 型のキーを全部持ち、値がすべて V 型」のオブジェクトを表す Utility Type。
  const icons: Record<ReadingStatus, string> = {
    // キー名が普通の識別子（英数字と _ だけ）なので、クォートは省略できる。
    // "want-to-read" のようにハイフンを含むキーなら "..." で囲む必要がある。
    want_to_read: "bookmark",
    reading: "book-open",
    completed: "check-circle",
  };
  // status をキーにして値を取り出す。Record で全キー網羅されているので必ず文字列が取れる。
  return icons[status];
}
```

```
// ===== フィルタリング =====

// 書籍配列を「指定ステータスのものだけ」に絞り込む関数。
function filterBooksByStatus(
  // 書籍の配列を受け取る。
  books: Book[],
  // 絞り込みたいステータス。
  status: ReadingStatus
// 戻り値は Book の配列。
): Book[] {
  // filter で「book.status と引数 status が一致する要素」だけ残す。
  return books.filter((book) => book.status === status);
}

// 3冊をまとめた配列。
const allBooks: Book[] = [book1, book2, book3];
// "reading" だけ残す → [book1]。
const readingBooks = filterBooksByStatus(allBooks, "reading");
// "completed" だけ残す → [book2]。
const completedBooks = filterBooksByStatus(allBooks, "completed");

// ===== ステータスの変更 =====

// book に新しい status を当てた新しい book を返す関数。
function updateBookStatus(
  book: Book,
  newStatus: ReadingStatus
): Book {
  // ... はスプレッド構文（オブジェクト版）。book の全プロパティをコピーして、後ろの status を上書き。元の book は壊さない（イミュータブル）。
  return { ...book, status: newStatus };
}

// book1 の status だけ "completed" にした新オブジェクト。
const updatedBook = updateBookStatus(book1, "completed");
// "completed"
console.log(updatedBook.status);

// 無効なステータスへの変更はコンパイルエラー
// ReadingStatus にない値なのでエラー。
// updateBookStatus(book1, "abandoned");
// エラー: Argument of type '"abandoned"' is not assignable to type 'ReadingStatus'
```

▼ **このコードがやること（先に日本語で）**：TypeScript の鉄板パターン「**判別可能なユニオン型**」を学びます。`status` という共通の目印（タグ）を使い、「読みたい本は理由を持つ」「読書中の本は現在ページを持つ」のように、状態ごとに**持つ情報を変える**型を作ります。`switch (book.status)` で分岐すると、その枝の中だけそのバリエーション固有のプロパティに安全にアクセスできます。

```ts
// ===== 判別可能なユニオン型（Discriminated Union）=====
// 判別可能なユニオン型 = 「共通のキー（タグ）」を使って、ユニオン型の中の
// どのバリエーションかを区別できる型。TypeScript で複雑な状態を扱う鉄板パターン。

// ステータスに応じて異なる追加情報を持つ型
type BookWithDetails =
  // ユニオンの1つ目: status="want_to_read" のとき。
  | {
      // ↑ 「判別タグ」。これがあるとTSがバリエーションを区別できる。
      status: "want_to_read";
      title: string;
      // 読みたい理由
      // この種類だけが reason を持つ。
      reason: string;
    }
  // 2つ目: status="reading" のとき。
  | {
      status: "reading";
      title: string;
      // 読書中だけ「現在ページ」を持つ。
      currentPage: number;
      totalPages: number;
    }
  // 3つ目: status="completed" のとき。
  | {
      status: "completed";
      title: string;
      // 数値リテラル型のユニオンで 1〜5 に限定。
      rating: 1 | 2 | 3 | 4 | 5;
      // review はオプショナル。
      review?: string;
    };

function displayBookInfo(book: BookWithDetails): string {
  // book.status を見て分岐。
  switch (book.status) {
    case "want_to_read":
      // ここでは book.reason にアクセス可能
      // この case 内では TS が「book は『1つ目』のバリエーション」と絞り込む。
      return `「${book.title}」を読みたい（理由: ${book.reason}）`;

    case "reading":
      // ここでは book.currentPage, book.totalPages にアクセス可能
      // この case では「2つ目」に絞り込み。currentPage が読める。
      // Math.round は四捨五入。
      const progress = Math.round(
        // 進捗率 = 現在ページ / 総ページ × 100。
        (book.currentPage / book.totalPages) * 100
      );
```

```
      return `「${book.title}」を読書中（進捗: ${progress}%）`;

    case "completed":
      // ここでは book.rating, book.review にアクセス可能
      // この case では「3つ目」に絞り込み。
      // .repeat(n) は文字列を n 回繰り返すメソッド。「★★★★☆」のような星評価を作る。
      const stars = "★".repeat(book.rating) + "☆".repeat(5 - book.rating);
      return `「${book.title}」読了 ${stars}`;
  }
}

// 使用例
console.log(
  displayBookInfo({
    status: "reading",
    title: "TypeScript入門",
    currentPage: 150,
    totalPages: 400,
  })
);
// => 「TypeScript入門」を読書中（進捗: 38%）

console.log(
  displayBookInfo({
    status: "completed",
    title: "React実践ガイド",
    rating: 4,
    review: "とても実践的でした",
  })
);
// => 「React実践ガイド」読了 ★★★★☆
```

7. TypeScript の設定（tsconfig.json）

7.1 tsconfig.json とは

`tsconfig.json` は TypeScript プロジェクトの設定ファイルです。コンパイラのオプション、対象ファイル、出力先などを指定します。

```jsonc
{
  // TypeScript コンパイラ（tsc）に渡す設定。
  "compilerOptions": {
    // コンパイル後の JavaScript のバージョン。新しいほどモダンな構文が使える。
    "target": "ES2022",
    // モジュールシステム。"ESNext" = ES Modules（import/export）形式で出力。
    "module": "ESNext",
    // 使える組み込み型の集合。"DOM" を入れるとブラウザの window や document 等の型が使える。
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    // 厳格モード一括有効化。これだけで厳しい型チェックが7〜8項目まとめてONになる。必須。
    "strict": true,
    // CommonJS と ES Modules の互換性を良くする。 import の書き心地が良くなる。
    "esModuleInterop": true,
    // node_modules 内の型定義のチェックを省略してビルドを高速化。
    "skipLibCheck": true,
    // ファイル名の大文字小文字を厳格にチェック。macOS/Linux の差を吸収。
    "forceConsistentCasingInFileNames": true,
    // import data from "./foo.json" のように JSON を読み込める。
    "resolveJsonModule": true,
    // 1ファイルだけ見て変換できる前提を強制。Babel や SWC との互換性のため。
    "isolatedModules": true,
    // tsc から .js ファイルを出力しない。Next.js 等のバンドラーに任せる場合に true。
    "noEmit": true,
    // JSX（React の <Component/> 構文）の変換方式。React 17+ の新形式。
    "jsx": "react-jsx",
    // パスエイリアスの基準ディレクトリ。"." はプロジェクトルート。
    "baseUrl": ".",
    // 短縮パスの定義。
    "paths": {
      // "@/foo" と書けば "./src/foo" を指す。深いネストでも import が短く書ける。
      "@/*": ["./src/*"]
    }
  },
  // コンパイル対象。** は「任意の階層」、* は「任意のファイル」。
  "include": ["src/**/*"],
  // 除外対象。ライブラリやビルド成果物はチェックしない。
  "exclude": ["node_modules", "dist"]
}
```

7.2 主要な設定項目

設定項目	説明	推奨値	備考
`target`	コンパイル先の JavaScript バージョン	`"ES2022"`	モダンブラウザ対象なら ES2022 以降
`module`	モジュールシステム	`"ESNext"`	ES Modules を使用
`lib`	使用する型定義ライブラリ	`["ES2022", "DOM"]`	ブラウザ用に DOM を含める
`strict`	すべての厳格チェックを有効化	`true`	**必ず true にする**
`noEmit`	JavaScript ファイルを出力しない	`true`	Next.js 等のバンドラーを使う場合
`jsx`	JSX の処理方法	`"react-jsx"`	React 17+ の新しい JSX Transform
`esModuleInterop`	CommonJS/ESModule の相互運用	`true`	import 構文の互換性向上
`skipLibCheck`	型定義ファイルのチェックを省略	`true`	コンパイル速度の向上
`forceConsistentCasingInFileNames`	ファイル名の大文字小文字を区別	`true`	OS 間の差異を防止
`resolveJsonModule`	JSON ファイルの import を許可	`true`	JSON をモジュールとして読み込める
`isolatedModules`	ファイル単位のトランスパイルを保証	`true`	Babel 等との互換性
`baseUrl`	モジュール解決のベースパス	`"."`	パスエイリアスの基準
`paths`	パスエイリアスの定義	`{"@/*": ["./src/*"]}`	`@/components` のような短縮パス

strict モードに含まれる個別オプション

`"strict": true` を設定すると、以下のオプションがすべて有効になります。

オプション	説明
`strictNullChecks`	`null` / `undefined` を他の型と区別する
`strictFunctionTypes`	関数の引数の型チェックを厳密にする
`strictBindCallApply`	`bind`, `call`, `apply` の引数を厳密にチェック
`strictPropertyInitialization`	クラスプロパティの初期化を必須にする
`noImplicitAny`	暗黙の `any` 型を禁止する
`noImplicitThis`	暗黙の `this` 型を禁止する
`alwaysStrict`	JavaScript の strict mode を有効にする
`useUnknownInCatchVariables`	catch の変数を `unknown` 型にする

7.3 Next.js での TypeScript 設定

Next.js プロジェクトでは、`next.config.ts` と `tsconfig.json` の両方で TypeScript の設定を行います。Next.js が自動的に最適な `tsconfig.json` を生成してくれます。

```jsonc
// Next.js プロジェクトの tsconfig.json（自動生成される）
{
  "compilerOptions": {
    // 古めのブラウザも考慮した出力バージョン。
    "target": "ES2017",
    // ブラウザの DOM 系 + 最新の JS 機能の型を有効化。
    "lib": ["dom", "dom.iterable", "esnext"],
    // .js ファイルも TypeScript と同居できるようにする。
    "allowJs": true,
    "skipLibCheck": true,
    // 厳格モード ON。
    "strict": true,
    // Next.js 側のバンドラ（SWC）が出力するので、tsc では出力しない。
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    // バンドラ向けの解決方式。最新の Next.js 推奨。
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    // JSX を変換せずそのまま残す。あとで Next.js（SWC）が変換する。
    "jsx": "preserve",
    // 前回の型情報を再利用してビルドを高速化。
    "incremental": true,
    "plugins": [
      {
        // Next.js 用 TypeScript プラグインを有効化（型補完が強化される）。
        "name": "next"
      }
    ],
    "paths": {
      // @/foo → ./src/foo のショートカット。
      "@/*": ["./src/*"]
    }
  },
  "include": [
    // Next.js が自動生成する型定義ファイル。
    "next-env.d.ts",
    // すべての .ts ファイル。
    "**/*.ts",
    // JSX を含む .tsx ファイル。
    "**/*.tsx",
    // .next ビルド出力中の型ファイル。
    ".next/types/**/*.ts"
  ],
  "exclude": ["node_modules"]
}
```

ポイント: Next.js では `"jsx": "preserve"` が使われます。これは JSX の変換を Next.js（SWC）に任せるためです。`"noEmit": true` なので TypeScript コンパイラ自身は JavaScript を出力しません。

▼ このコードがやること（先に日本語で）: これから使う Next.js で実際に登場する「型付き」コードを先取りで眺めます。ページの情報（`Metadata`）、サーバーでデータを取得する `async`（非同期）関数と `await`、props の型定義などです。今は細部を理解しなくてOK——「これまで学んだ型の知識が、こういう実際の画面コードで使われるんだ」と雰囲気をつかむのが目的です。

```tsx
// Next.js 特有の型の例

// ページコンポーネントの型
// import type は「型情報だけをインポートする」構文。コンパイル後には何も残らない。
import type { Metadata } from "next";

// ページのメタ情報（ブラウザのタブ表示やSEOで使われる）。
export const metadata: Metadata = {
  title: "書籍管理アプリ",
  description: "あなたの読書を管理するアプリです",
};

// export default = このファイルの「主役」をエクスポート。
export default function HomePage() {
  // JSX。HTML のように見えるが実は関数呼び出しの構文糖衣。
  return <h1>書籍管理アプリ</h1>;
}

// Server Component のデータ取得
// Next.js のページコンポーネントに渡される props の型。
interface PageProps {
  // URL のパスパラメータ。Next.js 15+ では Promise に包まれている。
  params: Promise<{ id: string }>;
  // URL のクエリ文字列。[key: string] は「任意の文字列キー」を表すインデックスシグネチャ。
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>;
}

// async = 非同期関数。{ params } は分割代入で props から params だけ取り出す。
export default async function BookPage({ params }: PageProps) {
  // await は Promise の中身を取り出す演算子。params から id を分割代入。
  const { id } = await params;
  // サーバーサイドでデータを取得
  // fetchBook(id) も Promise を返すので await。
  const book = await fetchBook(id);
  // JSX の中で { 式 } と書くと、その値を埋め込める。
  return <div>{book.title}</div>;
}
```

▼ コードを1つずつ分解して解説

ここには次章以降で頻出する `import type` や `async` / `await` が登場します。初見でこわく見える部分を、**1つずつ**ていねいに見ていきましょう。

解説1: 型だけをインポートする（`import type`）

```
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "書籍管理アプリ",
  description: "あなたの読書を管理するアプリです",
};
```

- `import type { Metadata } from "next"` … `import type` は「**型情報だけを取り込む**」専用の構文です。`Metadata` という型を `next` というライブラリから借りてきています。
- `type` を付けると「これは型のためだけのインポートだ」と TypeScript に伝わり、コンパイル後の JavaScript には**何も残りません**（型は実行時に消えるため）。
- `export const metadata: Metadata = { ... }` … `: Metadata` の型注釈で「この `metadata` は `Metadata` 型の形をしている」と宣言しています。`title` や `description` のスペルを間違えたり型を間違えたりすると、すぐエラーで気づけます。
- `export` は「このファイルの外から使えるように公開する」キーワードです。

> **用語: import type** … 「型だけ」を取り込むインポート。実行時のコードには影響しないため、ビルド後のファイルが軽くなる利点があります。

解説2: props の型を定義する（`Promise` 入り）

```
interface PageProps {
  params: Promise<{ id: string }>;
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>;
}
```

- `PageProps` は、Next.js のページ部品に渡される入力データ（props）の「形」を定義した型です。
- `params: Promise<{ id: string }>` … `Promise<...>` は「**いますぐではなく、少し後で値が用意される**」ことを表す型です。`{ id: string }` が将来手に入る中身で、ここでは URL の `id` です。
- `searchParams` の `{ [key: string]: ... }` は**インデックスシグネチャ**といい、「**任意の文字列をキーに持てる**オブジェクト」を表します。URL のクエリ文字列はどんなキーが来るか事前に決められないため、この書き方を使います。値が `string | string[] | undefined` のユニオン型なのは「1個・複数個・無し」のどれもあり得るからです。

- 今は細部を覚えなくてOK。「URL の情報が `Promise` に包まれて渡ってくるんだな」とだけつかめれば十分です。

> **用語: Promise（プロミス）** ...「いまはまだ無いが、あとで用意される値」を表す入れ物。サーバーへの問い合わせなど「時間のかかる処理」の結果を表すのに使います。

解説3: 非同期関数でデータを取得する（`async` / `await`）

```
export default async function BookPage({ params }: PageProps) {
  const { id } = await params;
  const book = await fetchBook(id);
  return <div>{book.title}</div>;
}
```

- `async function` ... 関数の前に付く `async`（エイシンク）は「**この関数は非同期処理を含む**」という印です。`async` を付けると、その中で `await` が使えるようになります。
- `({ params }: PageProps)` ... 引数で props を受け取り、その場で**分割代入**して `params` だけを取り出しています。`: PageProps` は引数の型注釈です。
- `const { id } = await params` ... `await`（アウェイト）は「`Promise` **の中身が用意されるまで待って、その値を取り出す**」演算子です。`params` は `Promise` なので `await` で中身を待ち、そこから `id` を分割代入で取り出しています。
- `const book = await fetchBook(id)` ... `fetchBook(id)` も `Promise` を返す（データ取得に時間がかかる）ので、`await` で結果を待ちます。
- `return <div>{book.title}</div>` ... 取得した `book` のタイトルを JSX に埋め込んで画面に返しています。`{ }` の中には JavaScript の式を書けます。

> **用語: async / await** ...「時間のかかる処理（サーバー通信など）」を、まるで順番に上から実行するかのように書ける仕組み。`async` を付けた関数の中で `await` を使い、結果が出るまで待ってから次に進みます。

7.3.1 Promise と async / await を動かして理解する

上の Next.js のコードでは `async` / `await` / `Promise` がいきなり出てきました。書籍管理アプリでは Supabase（データベース）から本を取ってくるたびにこの3つを使うので、ここで小さく動かして「**どの順番で何が起きるのか**」を体で覚えておきましょう。

> **たとえ話:** フードコートで料理を注文すると、料理の代わりに**呼び出しベル**を渡されます。これが `Promise`（「あとで料理が届く」という約束）です。ベルを持って席で待っている間、あなたは水を取りに行くなど別のことができます。ベルが鳴ったら料理を受け取る——この「鳴るまで待って受け取る」のが `await` です。

「待っている間に、ほかの処理が先に進む」を確かめる

> **▼ このコードがやること（先に日本語で）**：サーバーから本を取ってくる「ふり」をする関数（300ミリ秒かかる）を作り、`await` で待つ前後と、関数を呼んだ直後に、それぞれログを出します。ログが 1 → 2 → 3 の順に出るのはなぜか、を図と合わせて確かめます。ファイル名は `async-basic.ts` です。

```ts
// 指定したミリ秒だけ待ってから「完了」になる Promise を返す関数
function wait(ms: number): Promise<void> {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

interface Book {
  id: string;
  title: string;
}

// サーバーから本を取ってくる「ふり」をする関数（300ミリ秒かかる）
async function fetchBook(id: string): Promise<Book> {
  await wait(300);
  return { id, title: `本その${id}` };
}

async function main(): Promise<void> {
  console.log("1. 本の取得を開始します");
  // fetchBook が終わるまで、main の中だけがここで一時停止する
  const book = await fetchBook("1");
  console.log("3. 取得できました:", book.title);
}

main();
// main() は await で止まっている間に呼び出し元へ戻ってくるので、先にこちらが表示される
console.log("2. main() を呼んだ直後の行です");
```

▼ 実行結果（`node async-basic.ts`。`tsc` はエラーなし。3行目は約0.3秒後に表示される）

```
1. 本の取得を開始します
2. main() を呼んだ直後の行です
3. 取得できました: 本その1
```

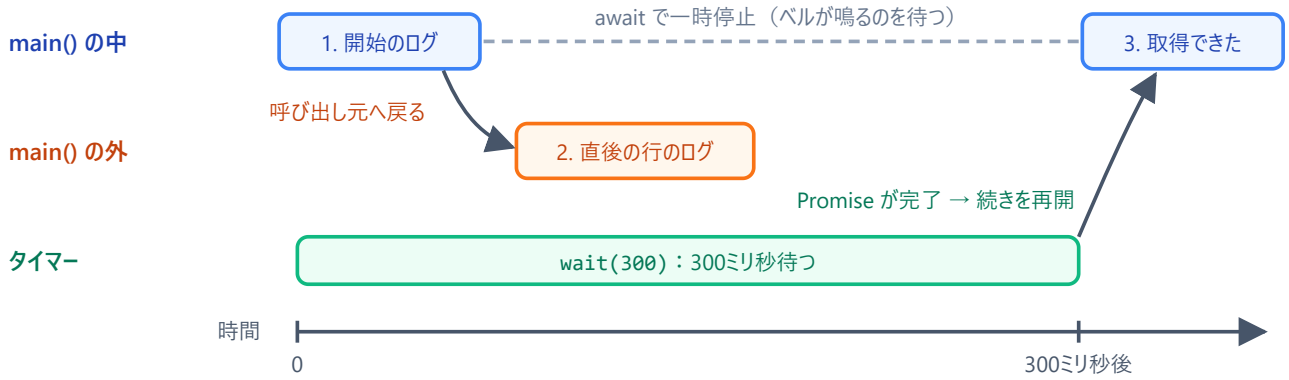


図2-14: async-basic.ts の時系列。await で main() が一時停止している間に、外側の「2.」が先に実行される

▼ コードを1つずつ分解して解説

解説1: `Promise<void>` を返す wait 関数

```
function wait(ms: number): Promise<void> {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}
```

- `new Promise((resolve) => { ... })` は「呼び出しベル」を作る書き方です。中で `resolve`（リゾルブ：解決する）を呼んだ瞬間に、ベルが鳴ります（Promise が**完了**します）。
- `setTimeout(resolve, ms)` は「`ms` ミリ秒たったら `resolve` を呼んでね」とタイマーに頼む関数です。つまり「`ms` ミリ秒後に鳴るベル」ができます。
- 戻り値の型 `Promise<void>` は「あとで完了するが、中身の値は無い（`void`）約束」という意味です。`< >` の中は**届く中身の型**で、5節のジェネリクスと同じ書き方です。

解説2: async 関数の戻り値は、必ず Promise になる

```
async function fetchBook(id: string): Promise<Book> {
  await wait(300);
  return { id, title: `本その${id}` };
}
```

- `async` を付けた関数は、`return` した値（ここでは `Book`）を**自動で Promise に包んで**返します。だから戻り値の型は `Book` ではなく `Promise<Book>`（あとで `Book` が届く約束）と書きます。
- `{ id, title: ... }` の `id` は `id: id` の省略形です（変数名とキー名が同じときに使える書き方）。

解説3: await で止まるのは「その async 関数の中だけ」

```
const book = await fetchBook("1");
```

- `await` は、Promise が完了するまで**その `async` 関数（`main`）の続きを一時停止**し、届いた中身（`Book`）を取り出します。だから `book.title` がそのまま使えます。
- ただし止まるのは `main` の中だけです。`main()` を呼んだ側は待たずに次の行へ進むので、`2.` が `3.` より先に表示されました（図2-14）。
- この「待っている間にほかの処理を進められる」性質のおかげで、サーバーからの返事を待つ間も画面が固まらずに済むのです。

2冊を「順番に待つ」か「同時に待つ」か

> **▼ このコードがやること（先に日本語で）**：300ミリ秒かかる取得を2回行うとき、①1冊ずつ `await` する場合と、② `Promise.all` で2冊同時に待つ場合とで、かかる時間を比べます。ファイル名は `async-parallel.ts` です。

```ts
function wait(ms: number): Promise<void> {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

interface Book {
  id: string;
  title: string;
}

async function fetchBook(id: string): Promise<Book> {
  await wait(300);
  return { id, title: `本その${id}` };
}

// 経過ミリ秒を 100 単位に切り捨てて返す（実行のたびに数ミリ秒ずれるため）
function elapsed(start: number): number {
  return Math.floor((Date.now() - start) / 100) * 100;
}

async function main(): Promise<void> {
  // ① 1冊ずつ順番に待つ
  let start = Date.now();
  const a = await fetchBook("1");
  const b = await fetchBook("2");
  console.log(`順番に待つ: ${a.title}, ${b.title}（約${elapsed(start)}ミリ秒）`);

  // ② Promise.all で同時に待つ
  start = Date.now();
  const [c, d] = await Promise.all([fetchBook("3"), fetchBook("4")]);
  console.log(`同時に待つ: ${c.title}, ${d.title}（約${elapsed(start)}ミリ秒）`);
}

main();
```

▼ 実行結果（`node async-parallel.ts`。`tsc` はエラーなし）

```
順番に待つ: 本その1, 本その2（約600ミリ秒）
同時に待つ: 本その3, 本その4（約300ミリ秒）
```

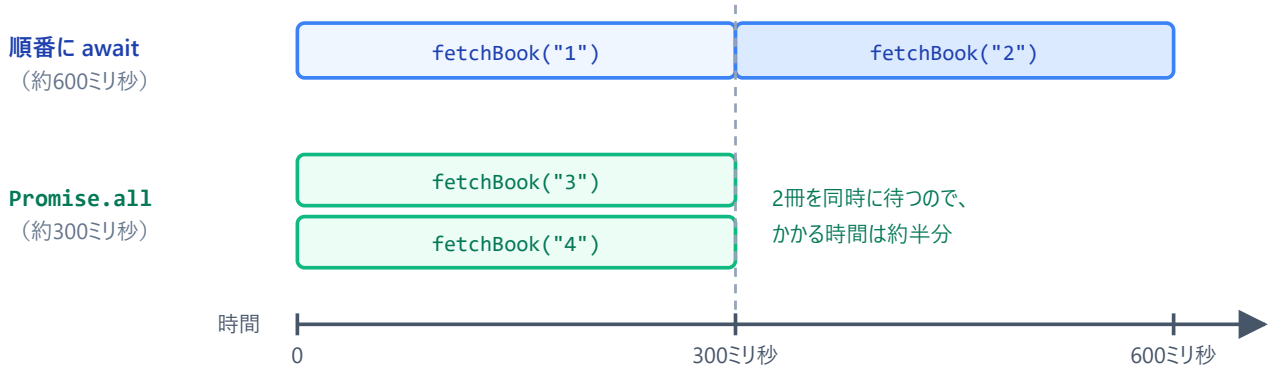


図2-15: 順番に await すると時間が足し算になる。Promise.all なら同時に待てる

- ①では `fetchBook("1")` が終わるまで `fetchBook("2")` を**始めない**ので、300 + 300 = 約600ミリ秒かかります。
- ②の `Promise.all([...])` は「配列で渡した Promise を**全部同時に**待ち、全部そろったら結果を配列で返す」関数です。2つのタイマーが同時に進むので約300ミリ秒で済みます。
- `const [c, d] = ...` は配列の分割代入です。結果は**渡した順番**（`"3"`、`"4"`）どおりに並びます。
- 「1冊目の結果を使って2冊目を取る」ような**順番に意味がある**場合は①、互いに関係ないデータをまとめて取る場合は②、と使い分けます。

> **つまずきポイント: `await` を書き忘れると「Promise そのもの」を受け取ってしまう** — `async` 関数を呼ぶと、返ってくるのは中身ではなく**呼び出しベル（Promise）**です。`await` を付け忘れると、ベルに向かって「タイトルを見せて」と言っているような状態になります。

NG 例（`await-ng.ts`。`wait`・`Book`・`fetchBook` は上と同じ）

```ts
function wait(ms: number): Promise<void> {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

interface Book {
  id: string;
  title: string;
}

async function fetchBook(id: string): Promise<Book> {
  await wait(300);
  return { id, title: `本その${id}` };
}

async function showTitle(): Promise<void> {
  // await を書き忘れた
  const book = fetchBook("1");
  console.log(book);
  console.log(book.title);
}

showTitle();
```

▼ 型チェックの結果（`npx tsc --noEmit --strict --target esnext await-ng.ts`）

```
await-ng.ts:21:20 - error TS2339: Property 'title' does not exist on type
'Promise<Book>'.

21   console.log(book.title);
                     ~~~~~

  await-ng.ts:21:20 - Did you forget to use 'await'?
    21   console.log(book.title);
                          ~~~~~


Found 1 error in await-ng.ts:21
```

▼ 実行結果（`node await-ng.ts`）

```
Promise { <pending> }
undefined
```

- `book` の型が `Book` ではなく `Promise<Book>` になっているので、「`Promise<Book>` に `title` はありません」と言われています。補足に `Did you forget to use 'await'?`（`await` を書き忘れていませんか？） とヒントまで出ています。
- 実行すると、`Promise { <pending> }`（まだ完了していない約束）と `undefined` が表示されるだけで、エラーにならずに**黙って間違った結果**になります。Next.js の画面では「タイトルが表示されない」という形で現れるので、型エラーで気づけるのはとても大きな助けです。

OK 例（`await-ok.ts`）：`showTitle` の中だけを直す

```
async function showTitle(): Promise<void> {
  // await で「中身が届くまで待って」から取り出す
  const book = await fetchBook("1");
  console.log(book);
  console.log(book.title);
}
```

▼ 実行結果（ファイル全体は NG 例の該当行を差し替えたもの）

```
{ id: '1', title: '本その1' }
本その1
```

8. よくあるエラーと対処法

TypeScript を使い始めると、必ず遭遇するエラーがあります。ここでは代表的なエラーとその解決方法を詳しく解説します。

エラーメッセージの読み方（まずここから）

個別のエラーを見る前に、`tsc` が出すエラーメッセージの**読み方の型**を身に付けましょう。英語がずらっと並ぶと身構えてしまいますが、どのエラーも決まった形をしているので、**場所** → **番号** → **本文** → **補足**の順に読めば怖くありません。

> **▼ このコードがやること（先に日本語で）：** `Book` 型の本を作るときに、①価格を文字列で書く ② `title` を `titel` と打ち間違える、という2つの間違いをわざと入れて、`tsc` の出力を読み解きます。ファイル名は `errors.ts` です。

```ts
interface Book {
  title: string;
  price: number;
}

const book: Book = {
  title: "TypeScript入門",
  price: "2980",
};

console.log(book.titel);
```

▼ 型チェックの結果（`npx tsc --noEmit --strict --target esnext errors.ts`）

```
errors.ts:8:3 - error TS2322: Type 'string' is not assignable to type 'number'.

8   price: "2980",
    ~~~~~

  errors.ts:3:3 - The expected type comes from property 'price' which is declared here
on type 'Book'
    3   price: number;
        ~~~~~

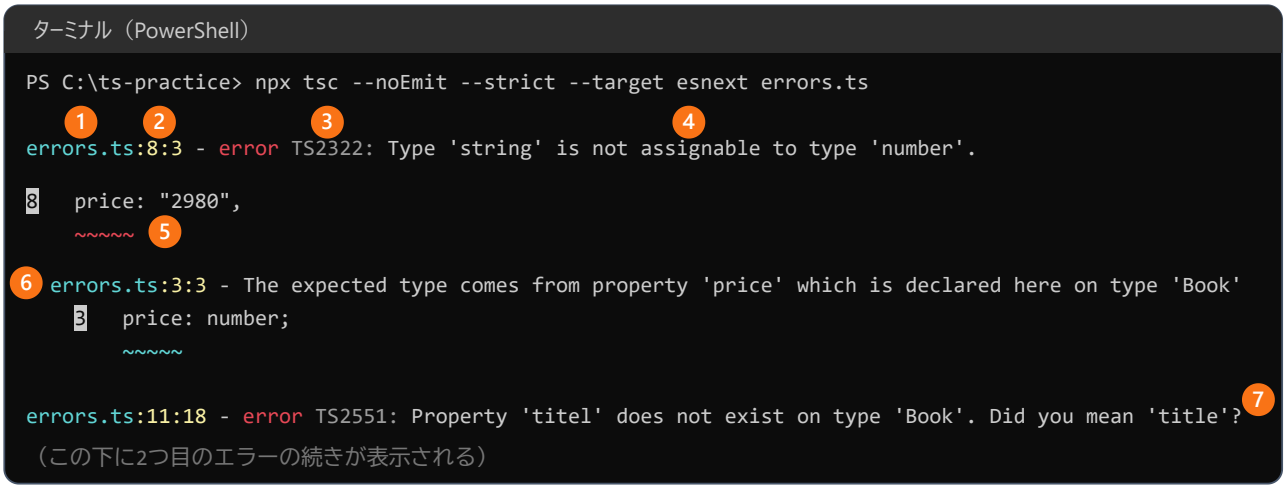
errors.ts:11:18 - error TS2551: Property 'titel' does not exist on type 'Book'. Did you
mean 'title'?

11 console.log(book.titel);
                    ~~~~~

  errors.ts:2:3 - 'title' is declared here.
    2   title: string;
        ~~~~~


Found 2 errors in the same file, starting at: errors.ts:8
```

ターミナルでは色付きで表示されます。上の部分を拡大して、各部分に番号を振ったのが図2-16です。



(1) ファイル名　(2) 行:列（8行目の3文字目）　(3) エラー番号（検索に使える）
(4) エラーの本文（何が起きたか）　(5) 問題の場所（~~~ の印）
(6) 補足：期待される型が決められた場所　(7) 次のエラー（Did you mean で正解を提案）

図2-16: tsc のエラー出力（画面イメージ）。場所 → 番号 → 本文 → 補足の順に読む

番号ごとの読み方をまとめると、次のとおりです。

No.	部分	例	読み方
(1) (2)	場所	`errors.ts:8:3`	`errors.ts` の **8行目・3文字目**。VS Code なら `Ctrl + G` で行番号へジャンプできる
(3)	エラー番号	`TS2322`	エラーの種類の番号。`TS2322` で検索すると解説が見つかる
(4)	本文	`Type 'string' is not assignable to type 'number'.`	「**string 型の値を、number 型の場所に入れようとした**」
(5)	印	`~~~~~`	問題がある部分の真下に引かれる（VS Code の赤い波線と同じ場所）
(6)	補足	`The expected type comes from property 'price' ...`	「number という期待は、3行目の `price: number` から来ています」
(7)	提案	`Did you mean 'title'?`	「もしかして `title` のことですか？」。打ち間違いのときに出る

よく出る本文のパターンは、次の4つの「型」で読めます。

英語の本文	日本語での読み方
`Type 'A' is not assignable to type 'B'.`	A 型の値は、B 型の場所に入れられない
`Argument of type 'A' is not assignable to parameter of type 'B'.`	A 型の引数は、B 型を受け取る引数（の席）に渡せない
`Property 'x' does not exist on type 'T'.`	T 型には x というプロパティが無い
`'x' is possibly 'undefined'.`（または `'null'`）	x は undefined（null）かもしれないので、そのまま使えない

> **ヒント:** エラーがたくさん出ても慌てないでください。**一番上のエラーから1つずつ**直し、直すたびにもう一度 `tsc` を実行するのがコツです。1つ目の間違いが原因で2つ目・3つ目が連鎖的に出ていることもよくあり、上を直すと下がまとめて消えることがあります。

8.1 Type 'X' is not assignable to type 'Y'

最も頻出するエラー です。ある型の値を、互換性のない別の型に代入しようとした際に発生します。

> **▼ このコードがやること（先に日本語で）:** 最頻出エラー「`Type 'X' is not assignable to type 'Y'`（X型はY型に代入できません）」の典型パターンと**直し方**を、悪い例→良い例の順で見ます。要は「箱の型と中身の型が合っていない」状態。文字列を数値に変換する、型注釈を直す、`as const` を使う、といった具体的な対処法を覚えるのが目的です。

```
// ===== エラーパターン1: プリミティブ型の不一致 =====

// 悪い例
// number 型に "25"（文字列）を入れている → 型違反。
const age: number = "25";
// エラー: Type 'string' is not assignable to type 'number'

// 良い例
// 数値リテラルを直接入れる。
const age: number = 25;
// または、文字列から変換する場合
// parseInt は文字列を整数に変換する関数。第2引数 10 は「10進数として解釈」の意味。
const age: number = parseInt("25", 10);
// Number() は文字列・boolean などを number に変換する関数。
const age: number = Number("25");
```

```ts
// ===== エラーパターン2: オブジェクト型の不一致 =====

// 型を定義。
interface User {
  name: string;
  age: number;
}

// 悪い例
const user: User = {
  name: "田中",
  // string を number に代入
  // age は number でなければならないのに文字列。
  age: "30",
};
// エラー: Type 'string' is not assignable to type 'number'

// 良い例
const user: User = {
  name: "田中",
  // 数値に修正。
  age: 30,
};
```

```ts
// ===== エラーパターン3: ユニオン型のリテラルが一致しない =====

// 許可される値は "active" か "inactive" のみ。
type Status = "active" | "inactive";

// 悪い例
// "enabled" は含まれていない。
const status: Status = "enabled";
// エラー: Type '"enabled"' is not assignable to type 'Status'

// 良い例
const status: Status = "active";
```

```ts
// ===== エラーパターン4: 変数の型が広すぎる =====

type Color = "red" | "blue" | "green";

// 悪い例
// string と推論される
// let は再代入されうるので「広い型」（string）に推論されてしまう。
let colorName = "red";
// string は Color（3リテラルだけ）に代入できない。
const color: Color = colorName;
// エラー: Type 'string' is not assignable to type 'Color'

// 良い例（方法1: as const を使う）
// "red" リテラル型と推論
// as const は「値をできる限り狭いリテラル型に固定する」キーワード。
const colorName = "red" as const;
// OK
const color: Color = colorName;

// 良い例（方法2: 型注釈を使う）
// 最初から Color 型として宣言。
const colorName: Color = "red";

// 良い例（方法3: satisfies を使う）
// "red" リテラル型かつ Color として検証
// satisfies は「この値は X 型を満たしますよね？と確認しつつ、値そのものの狭い型は維持する」演算子。as より安全。
const colorName = "red" satisfies Color;
```

8.2 Object is possibly 'undefined'

`strictNullChecks` が有効な場合に発生する、**null や undefined の可能性がある値にアクセスしようとしたとき**のエラーです。

> **注意（メッセージの表示が変わっています）**：古い TypeScript では、このエラーは見出しのとおり `Object is possibly 'undefined'.` と表示されていました。現在の TypeScript では、**どの変数が**怪しいのかまで書いてくれる形に変わっています。undefined の場合は `error TS18048: 'user.email' is possibly 'undefined'.`、null の場合は `error TS18047: 'button' is possibly 'null'.` のように表示されます（この節のコード中のエラー文も、現在の表示に合わせています）。意味はどちらも「その値は undefined（null）かもしれないので、そのまま使えません」です。

▼ このコードがやること（先に日本語で）:「 `Object is possibly 'undefined'` （値が無いかもしれない）」エラーの対処法を、配列アクセスやオプショナルなプロパティなど色々な場面で見ます。直し方は3つ——① `if` で「無いか」を先に確認、② `?.` （無ければ undefined を返す）、③ `??` （無ければ代替値を使う）。クラッシュを防ぐ実践的なテクニック集です。

```
// ===== エラーパターン1: 配列の要素アクセス =====

const fruits = ["りんご", "みかん", "バナナ"];

// 悪い例
// noUncheckedIndexedAccess が有効だと、配列[i] は string | undefined と推論される。
const first: string = fruits[0];
// エラー（strict 設定次第）: Type 'string | undefined' is not assignable to type 'string'
// 配列のインデックスアクセスは undefined を返す可能性がある

// 良い例（方法1: undefined チェック）
// 型注釈を省くと string | undefined。
const first = fruits[0];
// undefined でないか確認する型ガード。
if (first !== undefined) {
  // OK
  // ガード内では string に絞り込まれている。
  console.log(first.toUpperCase());
}

// 良い例（方法2: デフォルト値）
// ?? で undefined の場合のデフォルト値を用意。これで first は必ず string。
const first = fruits[0] ?? "デフォルト";
// OK
console.log(first.toUpperCase());
```

```ts
// ===== エラーパターン2: オプショナルプロパティ =====

interface User {
  name: string;
  // オプショナル（string | undefined）
  // ? を付けると省略可能なプロパティになる。
  email?: string;
}

// email を省略。値は undefined。
const user: User = { name: "田中" };

// 悪い例
// user.email は string|undefined。undefined に .toUpperCase は呼べない。
console.log(user.email.toUpperCase());
// エラー: 'user.email' is possibly 'undefined'.
// email は設定されていないかもしれない

// 良い例（方法1: if チェック）
// truthy チェック（undefined や "" でない）。
if (user.email) {
  // OK
  // ガード内では string に絞り込まれる。
  console.log(user.email.toUpperCase());
}

// 良い例（方法2: オプショナルチェイニング）
// undefined なら undefined を返す
// ?. を付けると「左が null/undefined なら、その場で undefined を返す」。クラッシュしない。
console.log(user.email?.toUpperCase());

// 良い例（方法3: null 合体演算子と組み合わせ）
// 結果が undefined なら ?? の右側 "メール未設定" を使う。
console.log(user.email?.toUpperCase() ?? "メール未設定");
```

```ts
// ===== エラーパターン3: Map.get() の戻り値 =====

// Map は「キーと値のペアを保持するデータ構造」。<キーの型, 値の型> でジェネリック指定。
const userMap = new Map<string, string>();
// .set(キー, 値) で追加。
userMap.set("user1", "田中");

// 悪い例
// .get(キー) は「キーが存在しないこともある」のを考慮して string | undefined を返す。
const name: string = userMap.get("user1");
// エラー: Type 'string | undefined' is not assignable to type 'string'
// Map.get() は undefined を返す可能性がある

// 良い例
// 型は string | undefined。
const name = userMap.get("user1");
if (name !== undefined) {
  // OK
  console.log(name);
}

// または
// 未設定キーに対するデフォルト値で string に確定。
const name = userMap.get("user1") ?? "不明";
```

```
// ===== エラーパターン4: document.querySelector =====

// 悪い例
// querySelector は CSS セレクタで要素を探す DOM API。見つからないと null を返す。戻り値は Element | null。
const button = document.querySelector("#submit-btn");
// null かもしれないものに addEventListener は呼べない。
button.addEventListener("click", handleClick);
// エラー: 'button' is possibly 'null'.
// querySelector は要素が見つからない場合 null を返す

// 良い例（方法1: null チェック）
const button = document.querySelector("#submit-btn");
// null/undefined でない=要素ありを確認。
if (button) {
  // ガード内では非 null に絞り込み済み。
  button.addEventListener("click", handleClick);
}

// 良い例（方法2: 存在が確実な場合は Non-null assertion）
// 末尾の ! は「非nullアサーション」。「絶対 null じゃないと俺が保証する」と TypeScript に伝える演算子。型からのみ null を取り除く（実行時のチェックはしない）。
const button = document.querySelector("#submit-btn")!;
// ただし、要素が存在しない場合は実行時エラーになるので注意
button.addEventListener("click", handleClick);

// 良い例（方法3: 型を絞り込む）
// <HTMLButtonElement> で「ボタン要素を期待している」と型引数を渡す。戻り値は HTMLButtonElement | null。
const button = document.querySelector<HTMLButtonElement>("#submit-btn");
// instanceof は「特定のクラスのインスタンスか」を判定する演算子。
if (button instanceof HTMLButtonElement) {
  // HTMLButtonElement のプロパティにアクセス可能
  // ガード内では HTMLButtonElement に絞り込まれ、.disabled が使える。
  button.disabled = true;
}
```

8.3 Property does not exist on type

オブジェクトに存在しないプロパティにアクセスしようとした際 に発生するエラーです。

> **▼ このコードがやること（先に日本語で）:**「 `Property does not exist on type` （そのプロパティは存在しません）」エラーの原因と直し方を見ます。多くは①プロパティ名の**打ち間違い**（TypeScript が正しい名前を提案してくれる）、②型定義に項目が足りない、③ユニオン型で「今どっちの型か」を絞り込めていない、のいずれか。順に対処法を確認します。

```ts
// ===== エラーパターン1: タイプミス =====

interface User {
  name: string;
  email: string;
}

const user: User = { name: "田中", email: "tanaka@example.com" };

// 悪い例
// emial（タイプミス）。User 型に emial プロパティは無い。
console.log(user.emial);
// エラー: Property 'emial' does not exist on type 'User'.
// TypeScript が「もしかして email では？」とヒントもくれる。
// Did you mean 'email'?

// 良い例
console.log(user.email);
```

```ts
// ===== エラーパターン2: 型定義にプロパティが不足 =====

interface Product {
  name: string;
  price: number;
}

const product: Product = { name: "ペン", price: 150 };

// 悪い例
// Product 型に description は存在しない。
console.log(product.description);
// エラー: Property 'description' does not exist on type 'Product'

// 良い例（方法1: 型定義を修正）
interface Product {
  name: string;
  price: number;
  // プロパティを追加
  // オプショナル（?）で追加すれば既存データを壊さない。
  description?: string;
}

// 良い例（方法2: オブジェクトリテラルにプロパティを追加する場合）
// スプレッドで既存プロパティを展開し、後ろに description を追加した新オブジェクトを作る。型は自動推論される。
const productWithDesc = { ...product, description: "赤いボールペン" };
```

```ts
// ===== エラーパターン3: ユニオン型での共通でないプロパティ =====

interface Dog {
  // 判別タグ。"dog" リテラル型。
  kind: "dog";
  // bark メソッド（戻り値なし）。
  bark(): void;
}

interface Cat {
  kind: "cat";
  meow(): void;
}

// Dog または Cat。
type Animal = Dog | Cat;

// 悪い例
function makeSound(animal: Animal) {
  // Cat に bark は無い。ユニオン型では「全バリエーションに共通する性質」しか使えない。
  animal.bark();
  // エラー: Property 'bark' does not exist on type 'Animal'
  // Property 'bark' does not exist on type 'Cat'
  // → Animal が Cat の場合、bark() は存在しない
}

// 良い例（型の絞り込み）
function makeSound(animal: Animal) {
  // kind を使ってどちらか判別。
  if (animal.kind === "dog") {
    // OK: Dog 型として認識
    // ガード内では Dog に絞り込まれる。
    animal.bark();
  } else {
    // OK: Cat 型として認識
    // else 側では Cat に絞り込まれる。
    animal.meow();
  }
}
```

```
// ===== エラーパターン4: API レスポンスの型が不足 =====

// 悪い例: JSON.parse の結果は any ではなく unknown として扱うべき
// async/await を使った非同期関数。
async function fetchData() {
  // fetch でサーバーに HTTP リクエスト。await で結果待ち。
  const response = await fetch("/api/data");
  // any 型
  // .json() の戻り値の型はデフォルトで any。型情報が欠落。
  const data = await response.json();

  // この時点では data の構造が不明
  // 何のチェックも無しに data.items.length にアクセス。
  console.log(data.items.length);
  // ← 実行時エラーの可能性あり
}

// 良い例: 型を明示的に定義
// 返ってくる JSON の構造を型として定義。
interface ApiData {
  items: string[];
  total: number;
}

// 戻り値の型を Promise<ApiData> と明示。
async function fetchData(): Promise<ApiData> {
  const response = await fetch("/api/data");
  // 取得結果を ApiData 型として扱う（型注釈）。
  // response.json() の結果は any なので、中身が本当にこの形かは実はチェックされない（下のつまずきポイント参照）。
  const data: ApiData = await response.json();

  // 型安全にアクセスできる
  // items は string[] と分かるので length が読める。
  console.log(data.items.length);
  console.log(data.total);

  return data;
}
```

つまずきポイント: `as` や `!` でエラーを「黙らせる」のは最後の手段 — `値 as 型`（型アサーション）は「この値はこの型です。私が保証します」と TypeScript に**言い切る**書き方です。TypeScript はそれを信じてチェックをやめるので、エラーは消えます。しかし中身が本当にその形かは**誰も確かめていない**ため、間違っていれば実行時にクラッシュします。上の良い例の `const data: ApiData = await response.json();` も、`json()` の結果が `any` なので、実質的には同じく「信じているだけ」です。

NG 例（`as-ng.ts`）：サーバーから届いた JSON を `as Book` で言い切ってしまう

```ts
interface Book {
  title: string;
  price: number;
}

// サーバーから届いた JSON 文字列（price が抜けている！）
const json = '{"title":"TypeScript入門"}';

// as Book で「これは Book です」と言い切ってしまう
const book = JSON.parse(json) as Book;
console.log(book.price.toLocaleString());
```

`npx tsc --noEmit --strict --target esnext as-ng.ts` は**エラーなし**。ところが実行すると落ちます。

▼ **実行結果**（`node as-ng.ts`。node 内部の行は省略）

```
C:\ts-practice\as-ng.ts:11
console.log(book.price.toLocaleString());
                       ^

TypeError: Cannot read properties of undefined (reading 'toLocaleString')

Node.js v25.9.0
```

OK 例（`as-ok.ts`）：いったん `unknown` で受け取り、型ガード関数で中身を確かめてから使う

```ts
interface Book {
  title: string;
  price: number;
}

// 「本当に Book の形をしているか」を実行時に調べる型ガード関数
function isBook(value: unknown): value is Book {
  if (typeof value !== "object" || value === null) {
    return false;
  }
  // in で「そのプロパティを持っているか」を確かめてから、型を調べる
  return (
    "title" in value &&
    typeof value.title === "string" &&
    "price" in value &&
    typeof value.price === "number"
  );
}

const json = '{"title":"TypeScript入門"}';

// JSON.parse の結果は「まだ何か分からない」ので unknown として受け取る
const data: unknown = JSON.parse(json);

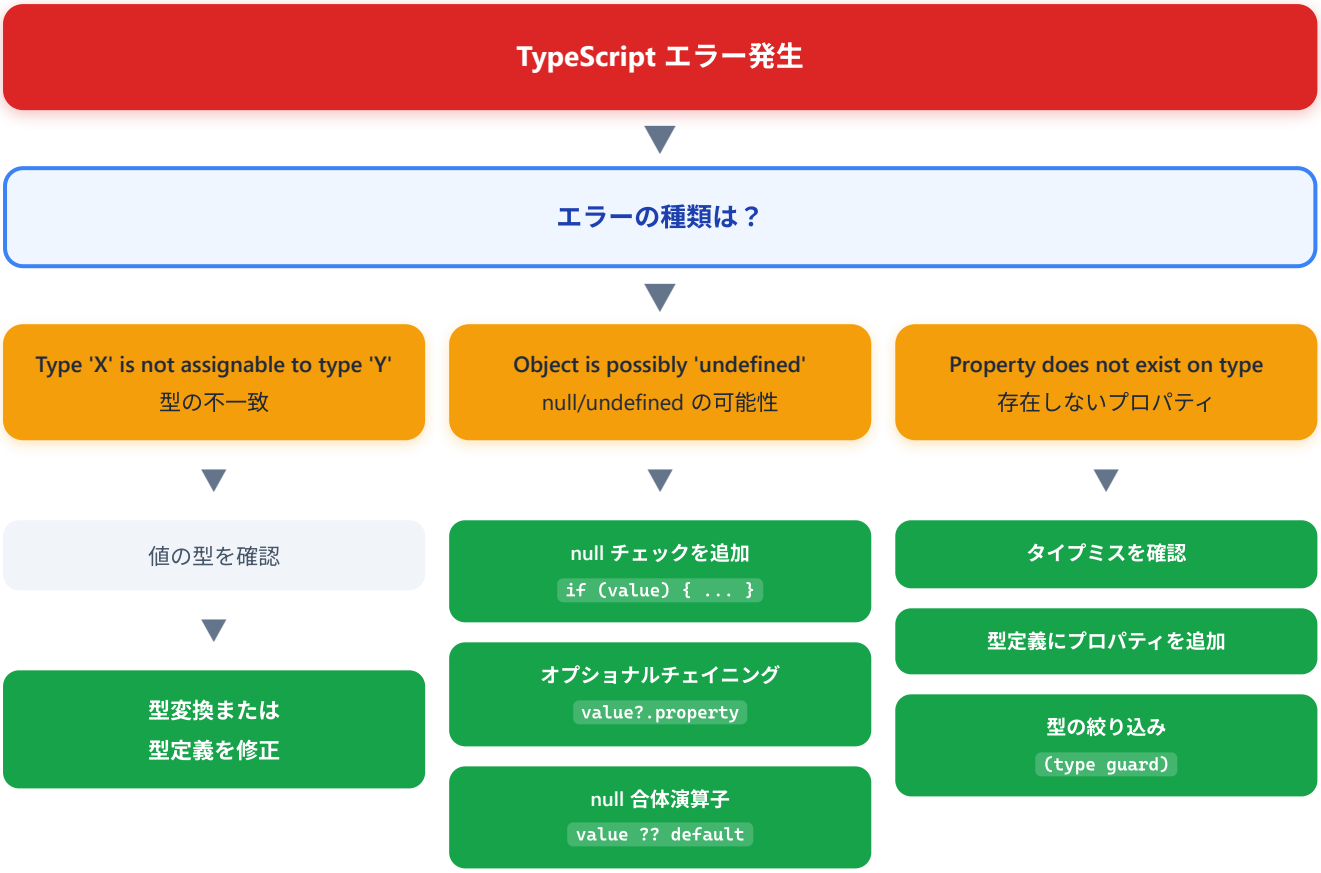
if (isBook(data)) {
  console.log(data.price.toLocaleString());
} else {
  console.log("本のデータの形が正しくありません:", data);
}
```

▼ **実行結果**（`tsc` はエラーなし）

```
本のデータの形が正しくありません: { title: 'TypeScript入門' }
```

- `unknown` で受け取ると、確かめるまで中身に触れなくなります（2.8 参照）。`isBook` が `true` を返したときだけ `Book` として扱えるので、`price` が無いデータで落ちることがありません。
- 末尾の `!`（非 null アサーション。8.2 のエラーパターン4）も同じ仲間で、「絶対に null ではない」と言い切るだけです。`as` や `!` を書きたくなったら、まず `if` や型ガードで確かめられないかを考えましょう。
- 書籍管理アプリでは Supabase のライブラリが「データベースの列に合わせた型」を付けて返してくれるので、自分で `as` を書く場面はほとんどありません。

8.4 エラー対処のまとめフローチャート



発展: アプリでは使っていない重要なTypeScript機能

> この章の本編とアプリ作りでは登場しなかったけれど、**実務やほかの人のコードを読むとき必ず出てくる**TypeScriptの機能をまとめて紹介します。どれも「これ単体で動く小さな例」なので、気になったものから順に読んでもらって大丈夫です。難しく感じたら「こういう機能があるんだな」と眺めるだけでもOKです。
>
> **補足:** 「ユニオン型」「リテラル型」は本編の第6章ですでに扱ったので、ここでは省略します。

Utility Types（型を組み立て直す便利な道具箱）

> **▼ このコードがやること（先に日本語で）**：すでにある型をもとに「一部だけ省略可能にした型」「特定の項目だけ抜き出した型」などを**自動で作り出す**便利機能（Utility Types）を一通り使います。同じ型を何度も手書きしなくて済むのが利点で、ここでは `Partial` ・ `Pick` ・ `Omit` ・ `Required` ・ `Record` の5つをまとめて見ます。

```ts
// ============================================================================
// Utility Types のサンプル — 既存の型から新しい型を「自動生成」する道具
// ============================================================================

// もとになる型を1つ用意する。これを使い回していく。
interface User {
  // ユーザーID（必須）
  id: number;
  // 名前（必須）
  name: string;
  // メールアドレス（必須）
  email: string;
}

// (1) Partial<T> … すべてのプロパティを「省略してもよい（?付き）」にした型を作る
//     → { id?: number; name?: string; email?: string } と同じ意味になる
// id も email も省略できる
const draft: Partial<User> = { name: "田中" };

// (2) Pick<T, キー> … T から指定したキーだけ「抜き出した」型を作る
//     → { id: number; name: string } と同じ意味
const summary: Pick<User, "id" | "name"> = { id: 1, name: "田中" };

// (3) Omit<T, キー> … T から指定したキーを「取り除いた」型を作る（Pick の逆）
//     → { id: number; name: string }（email を除いた形）
const noEmail: Omit<User, "email"> = { id: 1, name: "田中" };

// (4) Required<T> … すべてのプロパティを「必須」にした型を作る（Partial の逆）
interface Config {
  // ? が付いていて省略可能
  host?: string;
  port?: number;
}
// 両方とも必須になる
const fullConfig: Required<Config> = { host: "localhost", port: 8080 };

// (5) Record<キーの型, 値の型> … 「キーと値の組」をまとめて表す型を作る
//     ここでは「文字列キー → number 値」の辞書を表している
const scores: Record<string, number> = {
  math: 80,
  english: 95,
};

console.log(draft, summary, noEmail, fullConfig, scores);
// ▼ 実行結果
// { name: '田中' } { id: 1, name: '田中' } { id: 1, name: '田中' } { host: 'localhost',
port: 8080 } { math: 80, english: 95 }
```

▼ コードを1つずつ分解して解説

Utility Types は名前と役割さえ覚えれば、あとは組み合わせるだけです。**1つずつ**見ていきましょう。

解説1: Partial（全部を「省略可能」にする）

```
const draft: Partial<User> = { name: "田中" };
```

- `Partial<User>` は、`User` 型のすべてのプロパティに `?`（省略可能マーク）を付けた型を**自動で**作ります。
- もとの `User` は `id`・`name`・`email` がすべて必須ですが、`Partial<User>` ではどれを書いても・書かなくてもOKになります。
- 「入力途中の下書きデータ」や「一部だけ更新したいとき」によく使います。

> **用語: Utility Types（ユーティリティ型）** … TypeScript に最初から用意されている「型を加工する道具」のこと。
> 型の名前<もとの型> の形で使います。

解説2: Pick と Omit（必要な項目を選ぶ／いらない項目を捨てる）

```
const summary: Pick<User, "id" | "name"> = { id: 1, name: "田中" };
const noEmail: Omit<User, "email"> = { id: 1, name: "田中" };
```

- `Pick<User, "id" | "name">` は、`User` から `id` と `name` **だけを抜き出した**型です。`"id" | "name"` のように `|` で複数のキーを並べられます。
- `Omit<User, "email">` は、`User` から `email` **を取り除いた**型です。`Pick` のちょうど逆です。
- 「この場面では一部のプロパティしか使わない」というとき、もとの型を壊さずに新しい型を作れます。

解説3: Required（全部を「必須」にする）

```
const fullConfig: Required<Config> = { host: "localhost", port: 8080 };
```

- `Required<Config>` は、`Config` の `?`（省略可能マーク）を**すべて外して必須**にした型です。`Partial` の逆の働きをします。
- もとの `Config` は `host?` と `port?` がどちらも省略可能ですが、`Required<Config>` では両方とも必ず書かないとエラーになります。

解説4: Record（キーと値の組をまとめて表す）

```
const scores: Record<string, number> = {
  math: 80,
  english: 95,
};
```

- `Record<string, number>` は「**キーが `string`、値が `number` の組がいくつでも入る**」型です。
- `< >` の中は左がキーの型、右が値の型です。今回は「科目名（文字列）→ 点数（数値）」の辞書を表しています。
- 「どんなキー名が来るか事前に決まっていない辞書」を型で表したいときに便利です。

---

enum（列挙型 — 決まった選択肢に名前を付ける）

> **▼ このコードがやること（先に日本語で）**：「赤・黄・青」のような**決まった数の選択肢**に、それぞれ分かりやすい名前を付けてまとめる仕組み（enum）を使います。`Signal.Red` のように名前で書けるので、コードを読んだときに何を表しているか一目で分かるのが利点です。

```ts
// ============================================================================
// enum のサンプル – 「決まった選択肢の集まり」に名前を付ける
// ============================================================================

// enum（イーナム）= 「列挙型」。関連する定数（変わらない値）に名前を付けてまとめる。
enum Signal {
  // 名前を書くだけだと、自動で 0 が割り当てられる
  Red,
  // 次は 1
  Yellow,
  // 次は 2
  Green,
}

// enum の値は「列挙名.メンバー名」で取り出す
// current には 0 が入る（が、意味は「赤信号」）
const current: Signal = Signal.Red;

// 値そのものではなく「名前」で比較できるので読みやすい
if (current === Signal.Red) {
  console.log("止まれ");
}

// 数値ではなく文字列を割り当てることもできる（こちらの方がデバッグ時に分かりやすい）
enum Direction {
  Up = "UP",
  Down = "DOWN",
  Left = "LEFT",
  Right = "RIGHT",
}

// move には "UP" が入る
const move: Direction = Direction.Up;
console.log(move);

// ▼ 実行結果
// 止まれ
// UP
```

▼ コードを1つずつ分解して解説

enum は「選択肢に名前を付ける」だけのシンプルな機能です。**1つずつ**見ていきましょう。

解説1: 数値が自動で割り当たる enum

```
enum Signal {
  Red,
  Yellow,
  Green,
}
```

- `enum`（イーナム）は「**関連する選択肢をひとまとめにして名前を付ける**」仕組みです。
- 値を書かずに名前だけ並べると、上から順に `0, 1, 2, ...` という数値が自動で割り当てられます。`Red` は `0`、`Yellow` は `1`、`Green` は `2` です。
- コードの中では `Signal.Red` のように「名前」で書けるので、`0` と直接書くより**何を表しているか分かりやすく**なります。

> **用語: enum（列挙型）** … 「とりうる値があらかじめ決まっている」ものに名前を付けてまとめた型。曜日・信号・状態など「種類が固定の選択肢」を表すのに向いています。

解説2: 文字列を割り当てる enum

```
enum Direction {
  Up = "UP",
  Down = "DOWN",
  Left = "LEFT",
  Right = "RIGHT",
}

const move: Direction = Direction.Up;
```

- `= "UP"` のように、各メンバーに好きな値（ここでは文字列）を指定することもできます。
- 数値の enum だと表示したとき `0` などになって意味が分かりにくいですが、文字列を割り当てておくと `console.log(move)` で `"UP"` と表示され、**動作確認のとき分かりやすい**という利点があります。

> **つまずきポイント: enum は `node ファイル名.ts` では動かない** — この章の練習で使っている `node ファイル名.ts` は、型の部分を「消すだけ」で実行する仕組みです。ところが `enum` は型ではなく、**JavaScript のコード（オブジェクト）を新しく作り出す**特別な構文なので、消すだけでは済みません。そのため Node.js は実行を拒否します（Next.js では変換ツールが enum も処理するので、アプリの中では動きます）。

NG 例（`enum-ng.ts`）

```ts
enum Signal {
  Red = "RED",
  Yellow = "YELLOW",
  Green = "GREEN",
}

console.log(Signal.Red);
```

▼ **実行結果**（`node enum-ng.ts`。node 内部の行は省略）

```
C:\ts-practice\enum-ng.ts:1
  > enum Signal {
      Red = "RED",
      Yellow = "YELLOW",
      Green = "GREEN",
  > }

SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]: TypeScript enum is not supported in
strip-only mode
  code: 'ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX'
}

Node.js v25.9.0
```

- `TypeScript enum is not supported in strip-only mode` は「型を消すだけのモード（strip-only mode）では、TypeScript の enum は使えません」という意味です。なお `npx tsc` の型チェックでは、このファイルにエラーは出ません。

OK 例（`enum-ok.ts`）：`as const` を付けたオブジェクトで、enum と同じことをする

```ts
// enum の代わりに「as const を付けたオブジェクト」で選択肢をまとめる
const Signal = {
  Red: "RED",
  Yellow: "YELLOW",
  Green: "GREEN",
} as const;

// 値の一覧から型を作る → "RED" | "YELLOW" | "GREEN"
type Signal = (typeof Signal)[keyof typeof Signal];

function canGo(signal: Signal): boolean {
  return signal === Signal.Green;
}

console.log(Signal.Red);
console.log(canGo(Signal.Green));
console.log(canGo("RED"));
```

▼ **実行結果**（`tsc` はエラーなし）

```
RED
true
false
```

- `as const`（このあと発展編で詳しく扱います）を付けると、`Signal.Red` の型が `string` ではなく `"RED"` という決め打ちの値になります。
- `type Signal = (typeof Signal)[keyof typeof Signal];` は「`Signal` オブジェクトの値を全部集めたユニオン型」を作る決まり文句です。`keyof typeof Signal` で `"Red" | "Yellow" | "Green"`（キーの一覧）を作り、そのキーで値の型を取り出しています。最初は丸ごと覚えてしまって構いません。
- この形なら型を消しても普通のオブジェクトが残るだけなので、`node` でもそのまま動きます。最近の TypeScript のコードでは、enum の代わりにこの書き方や、単純な文字列リテラルのユニオン型（6章）がよく使われます。

型ガード（今この値はどの型か、を絞り込む）

> ▼ **このコードがやること（先に日本語で）**：「文字列かもしれないし数値かもしれない」という値を扱うとき、**今この瞬間はどちらの型なのかを調べて安全に処理する**テクニック（型ガード）を学びます。`typeof`・`in`・自作の判定関数の3通りを使い、それぞれ「ここから先はこの型だ」とTypeScriptに伝える方法を見ます。

```ts
// ============================================================================
// 型ガードのサンプル – 「今この値はどの型か」を調べて安全に絞り込む
// ============================================================================

// (1) typeof による型ガード … 文字列・数値などの「基本の型」を調べる
// value は string か number のどちらか
function describe(value: string | number): string {
  if (typeof value === "string") {
    // この { } の中では value は string だと TypeScript が分かっている
    // string 専用の .toUpperCase() が使える
    return `文字列です: ${value.toUpperCase()}`;
  } else {
    // ここでは value は number に絞り込まれている
    // number 専用の .toFixed() が使える
    return `数値です: ${value.toFixed(1)}`;
  }
}

// (2) in による型ガード … 「そのプロパティを持っているか」で型を見分ける
interface Dog {
  // 犬は bark を持つ
  bark: () => void;
}
interface Cat {
  // 猫は meow を持つ
  meow: () => void;
}

function speak(animal: Dog | Cat): void {
  // animal が bark プロパティを持っているか調べる
  if ("bark" in animal) {
    // ここでは Dog に絞り込まれている
    animal.bark();
  } else {
    // ここでは Cat に絞り込まれている
    animal.meow();
  }
}

// (3) 型述語（x is T）による自作の型ガード関数
//     戻り値の型に「value is string」と書くと、true のとき value は string だと TS に伝わる
function isString(value: unknown): value is string {
  return typeof value === "string";
}

const data: unknown = "hello";
if (isString(data)) {
  // isString が true なので、ここでは data は string 扱い
  // string の .length が使える
```

```
  console.log(data.length);
}

console.log(describe("hello"));
console.log(describe(3.14159));
// ▼ 実行結果
// 文字列です: HELLO
// 数値です: 3.1
```

▼ コードを1つずつ分解して解説

型ガードは「if文で調べると、その中では型が確定する」のがポイントです。**1つずつ**見ていきましょう。

解説1: typeof で基本の型を見分ける

```
if (typeof value === "string") {
  return `文字列です: ${value.toUpperCase()}`;
} else {
  return `数値です: ${value.toFixed(1)}`;
}
```

- `typeof value` は「その値が何の型か」を文字列で返す演算子です（`"string"`, `"number"`, `"boolean"` など）。
- `if (typeof value === "string")` が成り立った `{ }` の中では、TypeScript が「ここでは `value` は `string` だ」と理解してくれます。だから `string` 専用の `.toUpperCase()` を安心して呼べます。
- `else` 側では残った可能性（`number`）に絞り込まれるので、`.toFixed()` が使えます。

> **用語: 型ガード** ... 「今この値はどの型か」を調べて、コードの一部分だけ型を確定させる仕組みのこと。`if` と組み合わせて使います。

解説2: in でプロパティの有無を見分ける

```
if ("bark" in animal) {
  animal.bark();
} else {
  animal.meow();
}
```

- `"プロパティ名" in オブジェクト` は「そのオブジェクトがそのプロパティを持っているか」を調べる書き方です。
- `Dog` だけが `bark` を持つので、`"bark" in animal` が成り立てば `animal` は `Dog` だと絞り込めます。

- 基本の型（string/number など）ではなく、**オブジェクトの種類を見分けたいとき**に便利です。

解説3: 型述語（`x is T`）で自作の型ガードを作る

```
function isString(value: unknown): value is string {
  return typeof value === "string";
}
```

- 関数の戻り値の型に `value is string` と書くと、その関数が `true` を返したとき「`value` は `string` だ」とTypeScriptに教えられます。これを**型述語**と呼びます。
- 中身は `true` / `false` を返すだけの普通の関数ですが、`: boolean` の代わりに `: value is string` と書くのがポイントです。
- これを使うと `if (isString(data)) { ... }` の中で `data` を `string` として扱えるようになります。複雑な判定を関数にまとめて使い回せて便利です。

> **用語: `unknown`** … 「何が入っているか分からない値」を表す型。安全のため、型ガードで絞り込まないとほとんどの操作ができません。

---

as const（値を「変わらない決め打ちの値」として固定する）

> **▼ このコードがやること（先に日本語で）**：普通に書くと「ただの文字列」と扱われる値を、`as const` を付けることで「**まさにその値だけ**」という厳密な型に固定します。設定値や選択肢の一覧など「あとから変えたくない・特定の値だけ許したい」ものを安全に扱えるようになります。

```ts
// ============================================================================
// as const のサンプル – 値を「その値そのもの」に固定する
// ============================================================================

// (1) as const を付けない場合
// 型は string（"admin" 以外の文字列も入りうる、と判断される）
let role1 = "admin";

// (2) as const を付けた場合
// 型は "admin"（まさに "admin" という値だけ、に固定される）
const role2 = "admin" as const;

// (3) オブジェクトに付けると、中身すべてが「変更不可＆値固定」になる
const config = {
  host: "localhost",
  port: 8080,
} as const;
// ❌ as const のため変更不可（readonly になる）
// config.port = 3000;

// (4) 配列に付けると「読み取り専用＆中身の値が固定されたタプル」になる
//     よく「決まった選択肢の一覧」を作るのに使う
const colors = ["red", "green", "blue"] as const;
// colors の型は readonly ["red", "green", "blue"]

// この配列から「"red" | "green" | "blue"」というユニオン型を取り出せる
// "red" | "green" | "blue"
type Color = typeof colors[number];

// OK
const myColor: Color = "red";
// ❌ 一覧に無い値はエラー
// const ng: Color = "purple";

console.log(role2, config, colors, myColor);
// ▼ 実行結果
// admin { host: 'localhost', port: 8080 } [ 'red', 'green', 'blue' ] red
```

▼ コードを1つずつ分解して解説

`as const` は「値をガチガチに固定する」だけですが、便利な使い道があります。**1つずつ**見ていきましょう。

解説1: 付けるかどうかで型が変わる

```
// 型は string
let role1 = "admin";
// 型は "admin"
const role2 = "admin" as const;
```

- `let role1 = "admin"` の型はただの `string` です。「文字列なら何でも入る箱」と判断されます。
- `"admin" as const` と書くと、型が **`"admin"` という値そのもの**になります。つまり「`"admin"` 以外は入れられない」という厳密な型です。
- 「決まった値だけを許したい」ときに役立ちます。

> **用語: `as const`** … 値のうしろに付けると「この値は変わらない決め打ちの値だ」とTypeScriptに伝える書き方。型がぐっと厳密になります。

解説2: オブジェクト・配列に付けると全体が固定される

```
const config = {
  host: "localhost",
  port: 8080,
} as const;

const colors = ["red", "green", "blue"] as const;
```

- オブジェクトに `as const` を付けると、中のすべてのプロパティが `readonly`（変更不可）になり、値も固定されます。`config.port = 3000` のような書き換えはエラーになります。
- 配列に付けると「読み取り専用で、中身の値も順番も固定された配列（タプル）」になります。
- 「設定値」や「選択肢の一覧」のように、**あとから変えてほしくないデータ**にぴったりです。

解説3: 一覧からユニオン型を取り出す

```
const colors = ["red", "green", "blue"] as const;
// "red" | "green" | "blue"
type Color = typeof colors[number];
```

- `typeof colors` で「`colors` 変数の型」を取り出し、さらに `[number]`（配列の中身の型を取り出す書き方）を付けると、`"red" | "green" | "blue"` というユニオン型になります。
- こうすると「選択肢の一覧」と「許される値の型」を**1か所で管理**でき、片方を直せばもう片方も自動で揃います。選択肢が増えても書き換えるのは配列だけで済みます。

オプショナルチェイニング `?.` と Null合体演算子 `??`

> **▼ このコードがやること（先に日本語で）**：「値がないかもしれない」場面で安全に値を取り出す2つの便利演算子を学びます。`?.` は「途中が空っぽならそこで止めてエラーを防ぐ」、`??` は「値が空っぽだったらこっちの値を使う」という働きです。データが欠けていてもプログラムを止めずに済みます。

```
// ============================================================================
// ?. と ?? のサンプル – 「値がないかも」を安全に扱う
// ============================================================================

interface User {
  name: string;
  // address は省略可能（無いかもしれない）
  address?: {
    // city も省略可能
    city?: string;
  };
}

// (1) オプショナルチェイニング ?. … 途中が null / undefined なら、そこで止めて undefined を返す
// address が無い
const user1: User = { name: "田中" };
// address が無いので undefined（エラーにならない！）
console.log(user1.address?.city);

const user2: User = { name: "鈴木", address: { city: "東京" } };
// → "東京"
console.log(user2.address?.city);

// もし ?. を使わずに user1.address.city と書くと、実行時にクラッシュしてしまう

// (2) Null合体演算子 ?? … 左が null / undefined のときだけ、右の値を使う
// 左が undefined なので "未設定"
const city1 = user1.address?.city ?? "未設定";
// 左が "東京" なのでそのまま "東京"
const city2 = user2.address?.city ?? "未設定";

console.log(city1, city2);
// ▼ 実行結果
// undefined
// 東京
// 未設定 東京
```

▼ コードを1つずつ分解して解説

この2つはセットで使うことが多い演算子です。**1つずつ**見ていきましょう。

解説1: オプショナルチェイニング `?.`

```
const user1: User = { name: "田中" };
// undefined（エラーにならない）
console.log(user1.address?.city);
```

- `?.`（オプショナルチェイニング）は「**`?.` の左側が `null` か `undefined` なら、その場で止めて `undefined` を返す**」演算子です。
- `user1.address` が無い（`undefined`）場合、`user1.address.city` と普通に書くと「`undefined` の中の `city` 」を読もうとして実行時にクラッシュします。
- `user1.address?.city` と書けば、途中で安全に止まって `undefined` が返るだけなので、プログラムが落ちません。

> **用語: オプショナルチェイニング** … `?.` を使って「途中が空っぽでも安全につなげて値を取り出す」書き方。

解説2: Null合体演算子 `??`

```
const city1 = user1.address?.city ?? "未設定";
```

- `??`（ヌル合体演算子）は「**左側が `null` か `undefined` のときだけ、右側の値を使う**」演算子です。
- `user1.address?.city` は `undefined` なので、`??` の右側の `"未設定"` が `city1` に入ります。
- 似た `||`（OR）と違い、`??` は `0` **や空文字** `""` **を「値あり」として扱う**点が重要です。「`0` も正しい値」という場面では `??` を使うと安全です。

> **用語: Null合体演算子** … `??` を使って「値が無いときだけ別の値（デフォルト値）を使う」書き方。

`||` との違いは、値が `0` のときにはっきり表れます。

> **つまずきポイント: 「0 冊」が「情報なし」に化ける（`||` の落とし穴）** — `||` は左側が `0`・`""`（空文字）・`false` などの「偽っぽい値」のときにも右側を使います。そのため、`0` や空文字が**正しい値**である場面で `||` を使うと、データが勝手に置き換わってしまいます。

NG 例と OK 例（`nullish.ts`）

```
// 在庫数。0 は「在庫が 0 冊」という正しい値
const stock: number | null = 0;

// || は 0 や "" も「値がない」扱いにしてしまう
console.log(stock || "在庫情報なし");

// ?? は null と undefined のときだけ右側を使う
console.log(stock ?? "在庫情報なし");
```

▼ **実行結果**（`tsc` はエラーなし）

```
在庫情報なし
0
```

- 1行目（`||`、NG）は「在庫 0 冊」なのに「在庫情報なし」と表示してしまいました。2行目（`??`、OK）は `0` をそのまま表示しています。
- 「`null` や `undefined` のときだけ代わりの値を使いたい」なら `??`、と覚えておけば安全です。ページ数・評価・価格のように `0` があり得る数値では特に注意しましょう。

ジェネリック関数（型を後から決められる関数）

> ▼ **このコードがやること（先に日本語で）**：「どんな型の値が来ても対応できる」関数の書き方（ジェネリック関数）を学びます。`<T>` という印を付けると「型はあとで呼び出すときに決める」という意味になり、文字列でも数値でも同じ1つの関数で扱えます。しかも型の情報は失われないので安全です。

```ts
// ============================================================================
// ジェネリック関数のサンプル – 「型をあとから決める」関数
// ============================================================================

// <T> が「型のプレースホルダ（あとで決まる型の仮の名前）」。
// 引数 x の型も T、戻り値の型も「T の配列（T[]）」にしている。
function wrap<T>(x: T): T[] {
  // 受け取った値を、要素1つだけの配列に包んで返す
  return [x];
}

// 呼び出すときに T が自動で決まる（明示しなくてよい）
// T は string → a の型は string[]
const a = wrap("hello");
// T は number → b の型は number[]
const b = wrap(42);
// T は boolean → c の型は boolean[]
const c = wrap(true);

console.log(a, b, c);
// ▼ 実行結果
// [ 'hello' ] [ 42 ] [ true ]

// 型を明示的に指定することもできる（普通は不要）
// T を string と明示
const d = wrap<string>("world");

console.log(d);
// ▼ 実行結果
// [ 'world' ]
```

▼ コードを1つずつ分解して解説

ジェネリックは「型に名前を付けて使い回す」だけの仕組みです。**1つずつ**見ていきましょう。

解説1: `<T>` は「あとで決まる型の名前」

```ts
function wrap<T>(x: T): T[] {
  return [x];
}
```

- 関数名のうしろの `<T>` は「**型のプレースホルダ（仮の名前）**」です。`T` は「あとで決まる型」を表す目印で、`T` という名前自体に決まりはありません（`<U>` でも `<Item>` でもOKです）。

- 引数 `x: T` は「`T` 型の値を受け取る」、戻り値 `: T[]` は「`T` の配列を返す」という意味です。受け取った型と返す型が**連動**しているのがポイントです。
- 中身の `return [x]` は「受け取った値を、要素1つだけの配列に包んで返す」だけのシンプルな処理です。

> **用語: ジェネリック（総称型）** … 型を「あとで決められる変数」のように扱う仕組み。1つの関数を色々な型で使い回せるようになります。

解説2: 呼び出すと型が自動で決まる

```
// a の型は string[]
const a = wrap("hello");
// b の型は number[]
const b = wrap(42);
```

- `wrap("hello")` のように文字列を渡すと、`T` が自動で `string` に決まり、戻り値 `a` の型は `string[]` になります。
- `wrap(42)` なら `T` は `number` になり、`b` は `number[]` です。**渡した値から型が自動で決まる**ので、いちいち型を書く必要はありません。
- もし `any`（何でも型）で書いてしまうと型の情報が失われますが、ジェネリックなら「文字列を渡したら文字列の配列が返る」という対応が**型として保たれる**のが大きな利点です。

解説3: 型を明示的に指定することもできる

```
// T を string と明示
const d = wrap<string>("world");
```

- `wrap<string>(...)` のように `< >` の中に型を書くと、`T` を自分で指定できます。
- 普段は値から自動で決まるので省略しますが、「どうしてもこの型として扱ってほしい」場面では明示できる、と覚えておけば十分です。

練習問題（解答・解説つき）

この章の内容を、手を動かして確かめましょう。どの問題も、冒頭の「この章のサンプルを動かす準備」で作った練習用フォルダで試せます。**まず自分で考えて（できれば実際に打って動かして）から**、後ろの「解答と解説」を見てください。

問題1: 型注釈を付ける（3節）

次の JavaScript の関数に型注釈を付けて、TypeScript の関数にしてください。`price`（単価）と `quantity`（冊数）は数値、`discount`（値引き額）は数値で、**省略したら 0** とします。戻り値も数値です。また、型を付けたあとに `calcTotal("1000", 3)` と呼ぶと、`tsc` は何と言うでしょうか。

```
function calcTotal(price, quantity, discount) {
  return price * quantity - discount;
}
```

問題2: エラーメッセージを読む（8節の冒頭）

次の `q2.ts` を型チェックすると、下のエラーが出ました。(1) 何行目の何文字目が問題か、(2) 何型の値を何型の場所に入れようとしたのか、(3)「入れる場所の型」はどこで決められているか、(4) 直し方を答えてください。

```
interface Book {
  title: string;
  isbn: string;
}

const book: Book = {
  title: "TypeScript入門",
  isbn: 9784000000000,
};
```

```
q2.ts:8:3 - error TS2322: Type 'number' is not assignable to type 'string'.

8   isbn: 9784000000000,
    ~~~~

  q2.ts:3:3 - The expected type comes from property 'isbn' which is declared here on
type 'Book'
    3   isbn: string;
        ~~~~


Found 1 error in q2.ts:8
```

問題3: 参照とコピー（0.10.1）

次のコードを実行すると、3行の表示はそれぞれどうなるでしょうか。実行する前に予想してください。

```ts
const a = { title: "入門", tags: ["TS"] };
const b = a;
const c = { ...a };

b.title = "実践";
c.tags.push("React");

console.log(a.title);
console.log(a.tags);
console.log(a === b, a === c);
```

問題4: ユニオン型の絞り込み（6.1.1）

`string | number | boolean` のどれかを受け取り、次のような表示用の文字を返す関数 `toLabel` を書いてください。

- 文字列なら `文字列: TypeScript` のように、そのまま付ける
- 数値なら `数値: 3.0` のように、小数点以下1桁にする（`toFixed(1)` を使う）
- 真偽値なら `true` は `真偽値: はい`、`false` は `真偽値: いいえ`

問題5: 判別可能なユニオン型と never（6.3・2.9）

本の一覧を読み込む画面の状態を、次の型で表します。

```ts
type BooksState =
  | { status: "loading" }
  | { status: "success"; books: string[] }
  | { status: "error"; message: string };
```

(1) この状態を受け取り、`読み込み中…`、`2冊: TypeScript入門, React実践ガイド`、`エラー: 通信に失敗しました` のような文字を返す関数 `render` を `switch` で書いてください。`default` には、2.9 で学んだ `never` を使った網羅性チェックを入れてください。(2) あとから `BooksState` に `| { status: "empty" }` を追加したのに `render` を直し忘れると、何が起きるでしょうか。

問題6: ジェネリクス（5節）

配列の最後の要素を返す関数 `last` を、ジェネリクスを使って書いてください。空の配列なら `undefined` を返します。そのうえで、`last(["TypeScript入門", "React実践ガイド"])` と `last([2980, 3300, 1980])` の戻り値の型は何になるか答えてください。

問題7: async / await の実行順（7.3.1）

次のコードを実行すると、`A`・`B`・`C`・`D`・`1冊目 完了`・`2冊目 完了` の6行は、どの順番で表示されるでしょうか。

```ts
function wait(ms: number): Promise<void> {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

async function task(name: string, ms: number): Promise<string> {
  await wait(ms);
  console.log(`${name} 完了`);
  return name;
}

async function main(): Promise<void> {
  console.log("A");
  const first = task("1冊目", 200);
  const second = task("2冊目", 100);
  console.log("B");
  const results = await Promise.all([first, second]);
  console.log("C", results);
}

main();
console.log("D");
```

解答と解説

解答1

```ts
// price（単価）・quantity（冊数）・discount（値引き額。省略したら 0）
function calcTotal(
  price: number,
  quantity: number,
  discount: number = 0,
): number {
  return price * quantity - discount;
}

console.log(calcTotal(1000, 3));
console.log(calcTotal(1000, 3, 500));
```

▼ 実行結果（`node q1.ts`。`tsc` はエラーなし）

```
3000
2500
```

`calcTotal("1000", 3)` と呼んだ場合の型チェックの結果は次のとおりです。

```
q1-ng.ts:9:23 - error TS2345: Argument of type 'string' is not assignable to parameter
of type 'number'.

9 console.log(calcTotal("1000", 3));
                        ~~~~~~


Found 1 error in q1-ng.ts:9
```

- 関数の引数は、TypeScript が推論できない代表的な場所なので、必ず型を書きます（3.3）。
- `discount: number = 0` のように `= 値` を付けると「省略されたときの値（デフォルト値）」になり、引数を省略できるようになります。`discount?: number` と書く方法もありますが、その場合は中で `undefined` の対応が必要になるので、ここではデフォルト値のほうが簡単です。
- ちなみにこの間違った呼び出しも `node` で実行すると、`"1000" * 3` が自動で数値として計算されて `3000` と表示されてしまいます。たまたま正しく見えるだけなので、やはり型チェックが頼りです。

解答2

- (1) `q2.ts:8:3` なので、**8行目の3文字目**（`isbn` の部分）です。
- (2) `Type 'number' is not assignable to type 'string'.` なので、**number 型の値**（`9784000000000`）**を、string 型の場所に**入れようとしています。
- (3) 補足の `q2.ts:3:3 - The expected type comes from property 'isbn' which is declared here on type 'Book'` から、**3行目の** `isbn: string;`（`Book` **型の定義**）で決められていると分かります。
- (4) ISBN は「先頭の 0 や `-` を含むことがある、計算には使わない番号」なので、値のほうを文字列 `isbn: "9784000000000",` に直すのが正解です。型を `isbn: number` に変える直し方もエラー自体は消えますが、「番号のように見えても計算しない値（電話番号・郵便番号・ISBN など）は文字列で持つ」のが定石です。

解答3

▼ 実行結果（`node q3.ts`）

```
実践
[ 'TS', 'React' ]
true false
```

- 1行目: `b` は `a` と**同じ荷物の住所**なので、`b.title = "実践"` は `a` の荷物を書き換えます。
- 2行目: `c` は `{ ...a }` で作った**別の荷物**ですが、スプレッドは1段目だけのコピー（浅いコピー）なので、中の配列 `tags` は `a` と**共有**されています。`c.tags.push("React")` で `a.tags` にも `"React"` が増えます（図2-3の③と同じ）。
- 3行目: `a === b` は同じ住所なので `true`、`a === c` は別の住所なので `false` です。

解答4

```ts
function toLabel(value: string | number | boolean): string {
  if (typeof value === "string") {
    // ここでは string
    return `文字列: ${value}`;
  }
  if (typeof value === "number") {
    // ここでは number
    return `数値: ${value.toFixed(1)}`;
  }
  // 残りは boolean だけ
  return `真偽値: ${value ? "はい" : "いいえ"}`;
}

console.log(toLabel("TypeScript"));
console.log(toLabel(3));
console.log(toLabel(false));
```

▼ **実行結果**（`node q4.ts`。`tsc` はエラーなし）

```
文字列: TypeScript
数値: 3.0
真偽値: いいえ
```

- `typeof` で1つずつ調べて `return` するたびに、候補が `string | number | boolean` → `number | boolean` → `boolean` と減っていきます（図2-12と同じ流れ）。
- 最後の行では `value` が `boolean` に確定しているので、条件 `? :`（三項演算子）にそのまま使えます。

解答5

```ts
type BooksState =
  | { status: "loading" }
  | { status: "success"; books: string[] }
  | { status: "error"; message: string };

function render(state: BooksState): string {
  switch (state.status) {
    case "loading":
      return "読み込み中…";
    case "success":
      return `${state.books.length}冊: ${state.books.join(", ")}`;
    case "error":
      return `エラー: ${state.message}`;
    default: {
      // ここに来ることは無いはず。来たら型エラーで教えてもらう
      const unreachable: never = state;
      return unreachable;
    }
  }
}

const loaded: BooksState = {
  status: "success",
  books: ["TypeScript入門", "React実践ガイド"],
};

console.log(render({ status: "loading" }));
console.log(render(loaded));
console.log(render({ status: "error", message: "通信に失敗しました" }));
```

▼ **実行結果**（`node q5.ts`。`tsc` はエラーなし）

```
読み込み中…
2冊: TypeScript入門, React実践ガイド
エラー: 通信に失敗しました
```

(2) `BooksState` に `| { status: "empty" }` を追加して `render` を直さないと、型チェックで次のエラーになります。

```
q5-ng.ts:17:13 - error TS2322: Type '{ status: "empty"; }' is not assignable to type
'never'.

17       const unreachable: never = state;
               ~~~~~~~~~~~


Found 1 error in q5-ng.ts:17
```

- `case "success":` の中では `state` が `success` の形に絞り込まれるので、`state.books` が使えます（判別可能なユニオン型。6.3）。
- 3つの `case` で全部の形を処理しきると、`default` に来る `state` の型は `never`（ありえない）になり、`never` 型の変数に代入できます。
- 新しく `"empty"` が増えると、`default` に `{ status: "empty" }` が残ってしまうので代入できず、**直し忘れを型エラーで教えてくれます**。なお、このとき `node` で実行すると今までどおり動いてしまう（3行とも同じ表示）ので、やはり型チェックが大切です。`case "empty": return "本がありません";` を足せば直ります。
- `default: { ... }` のように `{ }` で囲んでいるのは、`case` の中で `const` を宣言するときの作法です（ほかの `case` と変数名がぶつからないようにするため）。

解答6

```ts
function last<T>(items: T[]): T | undefined {
  // 要素が無ければ undefined、あれば最後の要素を返す
  if (items.length === 0) {
    return undefined;
  }
  return items[items.length - 1];
}

const lastTitle = last(["TypeScript入門", "React実践ガイド"]);
const lastPrice = last([2980, 3300, 1980]);

console.log(lastTitle);
console.log(lastPrice);
console.log(last<string>([]));
```

▼ **実行結果**（`node q6.ts`。`tsc` はエラーなし）

```
React実践ガイド
1980
undefined
```

- `lastTitle` の型は `string | undefined`、`lastPrice` の型は `number | undefined` です（VS Code でホバーすると確かめられます）。渡した配列から `T` が自動で決まります。
- 戻り値に `| undefined` が付くのは、空の配列を渡されると返す要素が無いからです。「無いかもしれない」を型で正直に表しておくと、使う側が `??` や `if` で備えられます。
- `last<string>([1, 2])` のように型と中身が食い違うと、`error TS2322: Type 'number' is not assignable to type 'string'.` が要素ごとに出ます。

解答7

▼ **実行結果**（`node q7.ts`。`tsc` はエラーなし）

```
A
B
D
2冊目 完了
1冊目 完了
C [ '1冊目', '2冊目' ]
```

- `A` → `B` までは `main` の中で順番に実行されます。`task(...)` は呼んだ瞬間にタイマーを動かし始めますが、`await` していないので待たずに `B` へ進みます。
- `await Promise.all(...)` で `main` が一時停止すると、呼び出し元に戻って `D` が表示されます（図2-14と同じ仕組み）。
- タイマーは 100ミリ秒の「2冊目」が先に鳴るので、`2冊目 完了` → `1冊目 完了` の順です。
- `Promise.all` の結果の配列は、**完了した順ではなく渡した順**（`first`、`second`）に並ぶので、`[ '1冊目', '2冊目' ]` になります。

まとめ

この章では、TypeScript の基礎として以下の内容を学びました。

トピック	学んだこと
TypeScript とは	JavaScript のスーパーセットであり、型安全性を提供する
tsc と node の役割	`tsc` は型だけを調べる検査係、`node ファイル名.ts` は型を消して動かす実行係（型の間違いは見ない）
参照とコピー	オブジェクトの変数には「住所（参照）」が入る。`{ ...元 }` は1段目だけのコピー、丸ごとなら `structuredClone`
基本的な型	`string`, `number`, `boolean`, `array`, `tuple`, `object`, `any`, `unknown`, `never`, `void`, `null`, `undefined`
型注釈と型推論	明示的に型を書く方法と、TypeScript が自動で型を判断する仕組み
interface と type	オブジェクトの形を定義する2つの方法とその使い分け
ジェネリクス	型をパラメータ化して再利用性を高める仕組み
ユニオン型とリテラル型	複数の型や特定の値のみを許可する型定義
型の絞り込み	`if` と `typeof` / `=== null` / 型ガード関数で候補を減らしてから使う。型は「値の集まり」で、狭い → 広いは代入できる
tsconfig.json	TypeScript プロジェクトの設定ファイル
非同期処理	`Promise` は「あとで届く約束」。`await` で中身を取り出す（付け忘れると `Promise<T>` のまま）。同時に待つなら `Promise.all`
よくあるエラー	代表的な型エラーの原因と対処法。**場所** → **番号** → **本文** → **補足**の順に読む

次の章へ

次の章では、**React の基礎** を学び、TypeScript と React を組み合わせた開発方法を習得していきます。この章で学んだ型の知識は、React コンポーネントの Props や State の定義で活用されます。